# - TUNE 1 -
# Optimisation Manuelle de Requêtes SQL (ORS) sous Oracle

Gabriel MOPOLO-MOKE

Ancien professeur associé Université Côte d'Azur (UCA) NICE, France
Formateur et consultant indépendant

# Plan Général


- 1. Introduction
- 2. Etapes d'optimisations
- 3. Méthodes d'accès
- 4. Outils d'Optimisations
- 5. Optimiseur Statistique
- 6. Etude de cas
- 7. Annexes

# 1. Introduction

- PLAN

  - **1.1 Objectifs du cours**
  - **1.2 Problématique du Tuning**
  - **1.3 Optimisation sous Oracle**
  - **1.4 Optimisation de l'environnement du SGBD Oracle**
  - **1.5 Conception optimisée d'une base**
  - **1.6 Approches d'optimisation de requêtes**

# 1.1 Objectifs du cours

- **Comprendre le fonctionnement de l'optimiseur Oracle afin d'améliorer les temps de réponses**

- **Etudier les méthodes et chemins d'accès Oracle**

- **Etudier et utiliser les outils d'optimisation**

- **Etudier l'optimiseur de statistique avec des allusions à l'ancien optimiseur de syntaxe**

- **Produire et choisir le meilleur plan d'accès**

- **Ecrire soigneusement les requêtes afin de favoriser ou défavoriser un chemin :**
  **Optimiseur syntaxique et... *Statistique***

# 1.2 Problématique du Tuning

- **Définition**

  - **Le Tuning est l'art de réduire le coût d'accès aux données d'une base de données**

  - **Le coût d'une requête est influencé par:**

    - **Le coût de son traitement en mémoire**

    - **Et le coût du chargement des blocs depuis disques**

  - **COUT = Coût CPU (<10%) + COUT IO (>=90%)**

  - **Le Tuning est l'art de réduire le nombre de blocs à charger !!!**

# 1.2 Problématique du Tuning

- **Problèmes nécessitant le Tuning**
  - **Absence d'indexes**
  - **Requêtes mal écrites**
  - **Sous-occupation des blocs**
  - **Sous-dimensionnement du buffer de données**
  - **Sous-dimensionnement de la zone des requêtes partagées**
  - **Surdimensionnement du buffer redolog**
  - **Sous-dimensionnements des fichiers redolog**
  - **Conflits entre les processus : exemple LGWR et ARCH**

# 1.2 Problématique du Tuning

- **Absence d'indexes**

  - **Considérons la table EMPLOYE :**

    - **Ses colonnes : Empno, ename, adr, job, sal, téléphone**

    - **Voici ses caractéristiques :**
      - **Elle a 100000 Employés, 8000 blocs de 8 K chacun qui sont tous occupés**
      - **Il y a un seul employé de nom KING**
      - **10 ms (Millisecondes) est le coût moyen de chargement d'un bloc**

  - **Considérons la requêtes suivantes**

```
SELECT * FROM EMPLOYE
WHERE ENAME='KING';
```

# 1.2 Problématique du Tuning

- **Absence d'indexes**
  - **Si aucun index n'est posé sur ENAME**
  - **Le SGBD doit charger les 8000 Blocs alors qu'il n' y a qu'un employé de nom KING=> 8000 blocs * 10 ms**
  - **En posant un index sur ENAME**
    - Le cout hors cout d'accès à l'index tombe à 1 bloc=> 1 Bloc * 10 ms

# 1.2 Problématique du Tuning

- **Requêtes mal écrites**

  - **Considérons la table des Employés précédemment décrite, il y a 10 employés sans téléphone, il y a un index sur la colonne téléphone.**

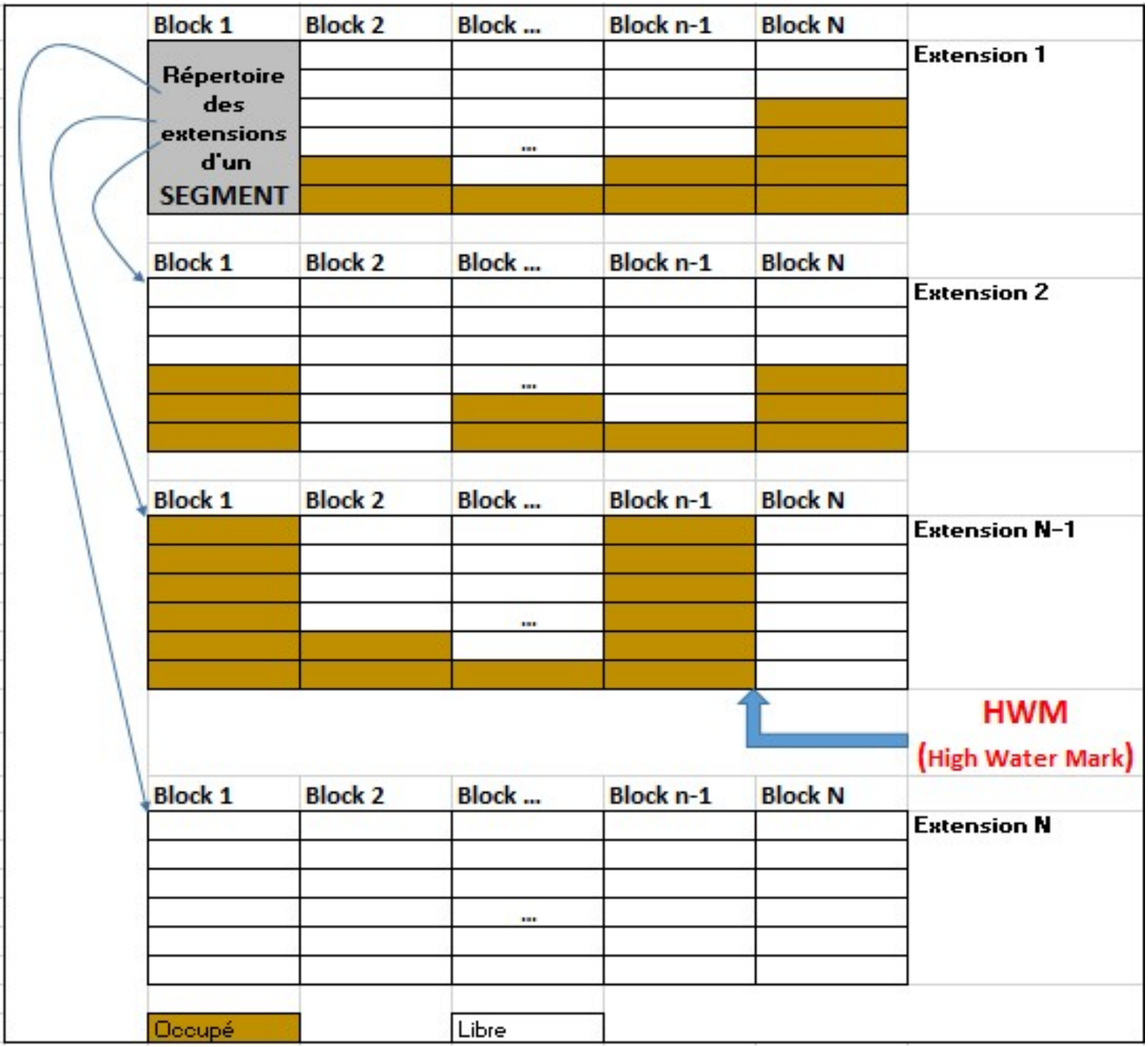
  - **Considérons la requête suivante**

    (1) Select * from Employe where téléphone is null;
    (2) Select * from employe where initcap(ename) = 'King';

    **8000 Blocs seront chargés au lieu de 10 si chaque ligne était dans 1 bloc différent : Car (1) INDEX JAMAIS UTILISE DANS CE CAS (2)Application d'une fonction désactive l'index**

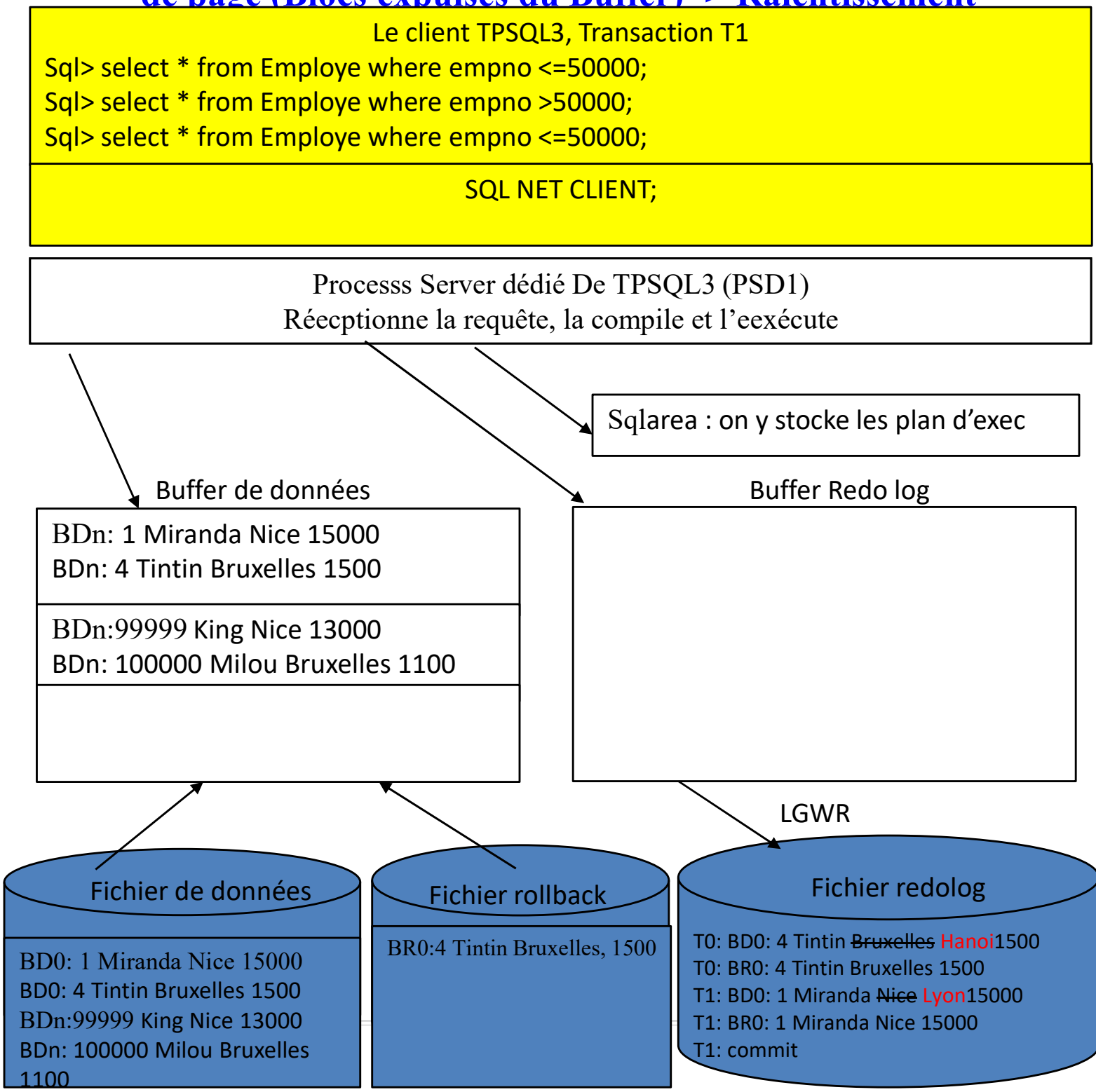
# 1.2 Problématique du Tuning

- **Sous-occupation des blocs**
  - **Trop de blocs à charger pour peu de lignes**
  - **Solution : COMPACTER pour réduire le nombre de blocs à charger**



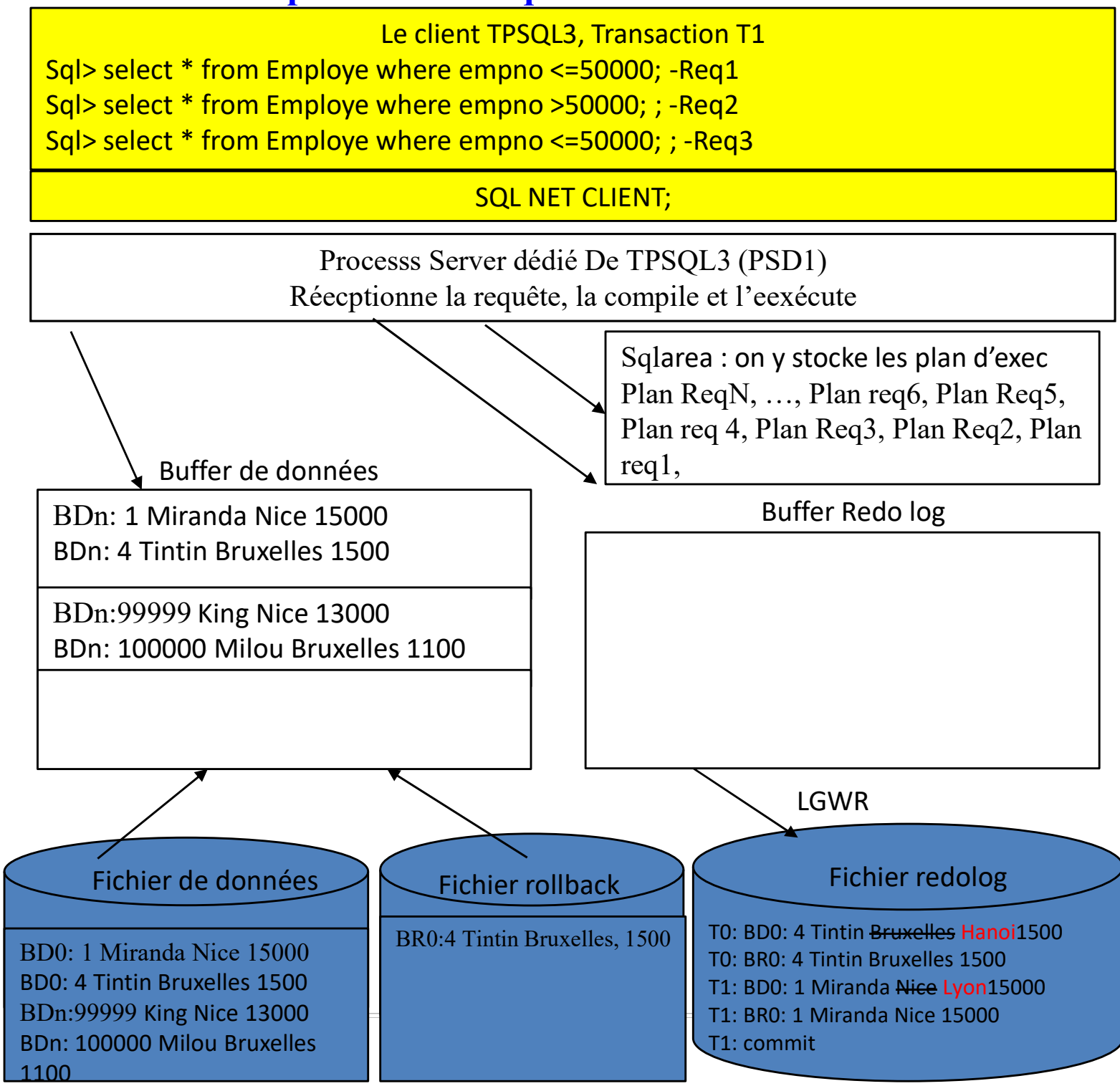
# 1.2 Problématique du Tuning

- **Sous-dimensionnement du buffer de données**
  - **Algorithme LRU**
  - **Si le Buffer de Données est trop petit, il y aura des défauts de page (Blocs expulsés du Buffer)=> Ralentissement**



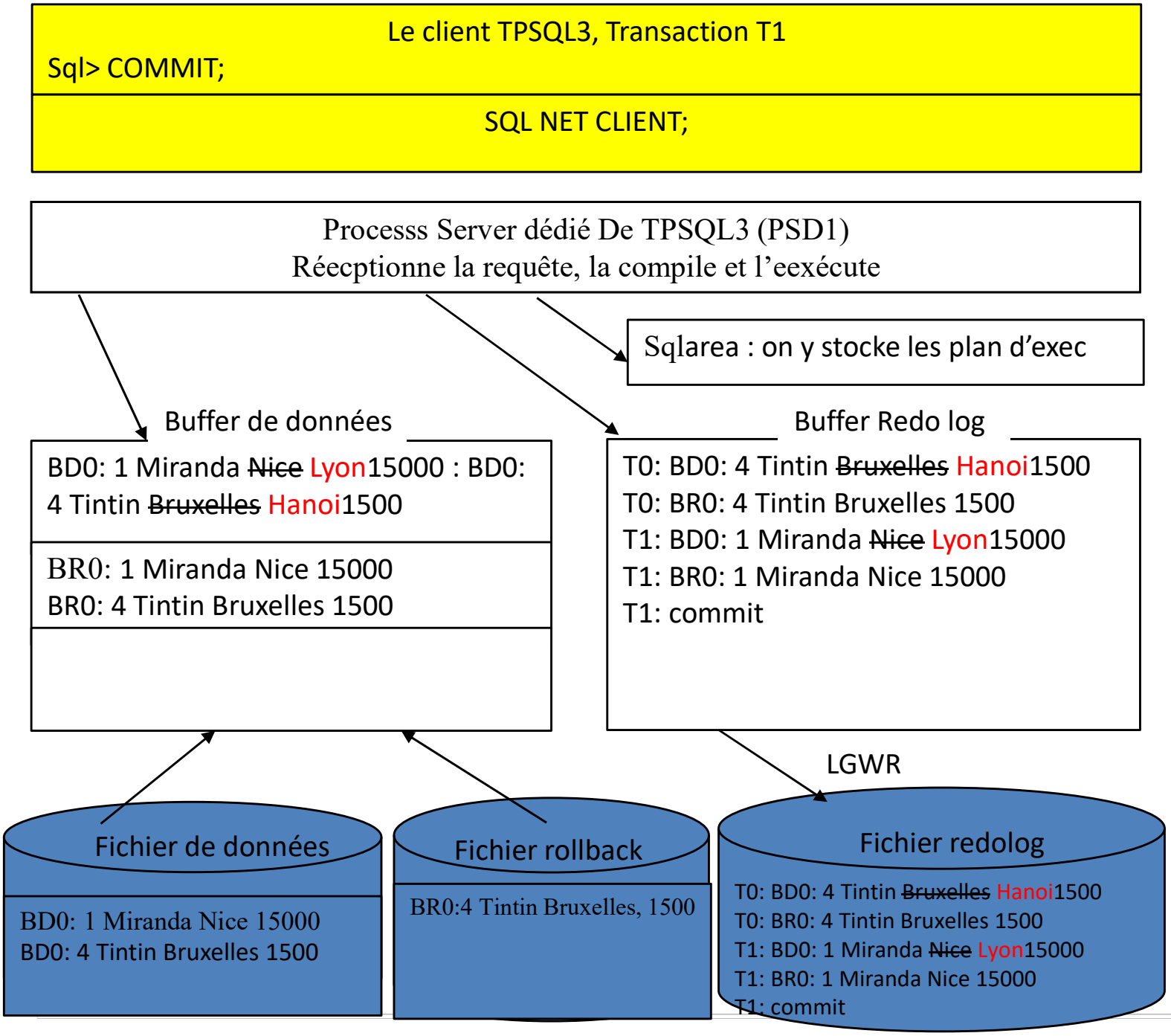
# 1.2 Problématique du Tuning

- **Sous-dimensionnement de la zone des requêtes partagées**
  - **Algorithme LRU. Les requêtes les moins sollicités sont expulsées de la sqlarea**



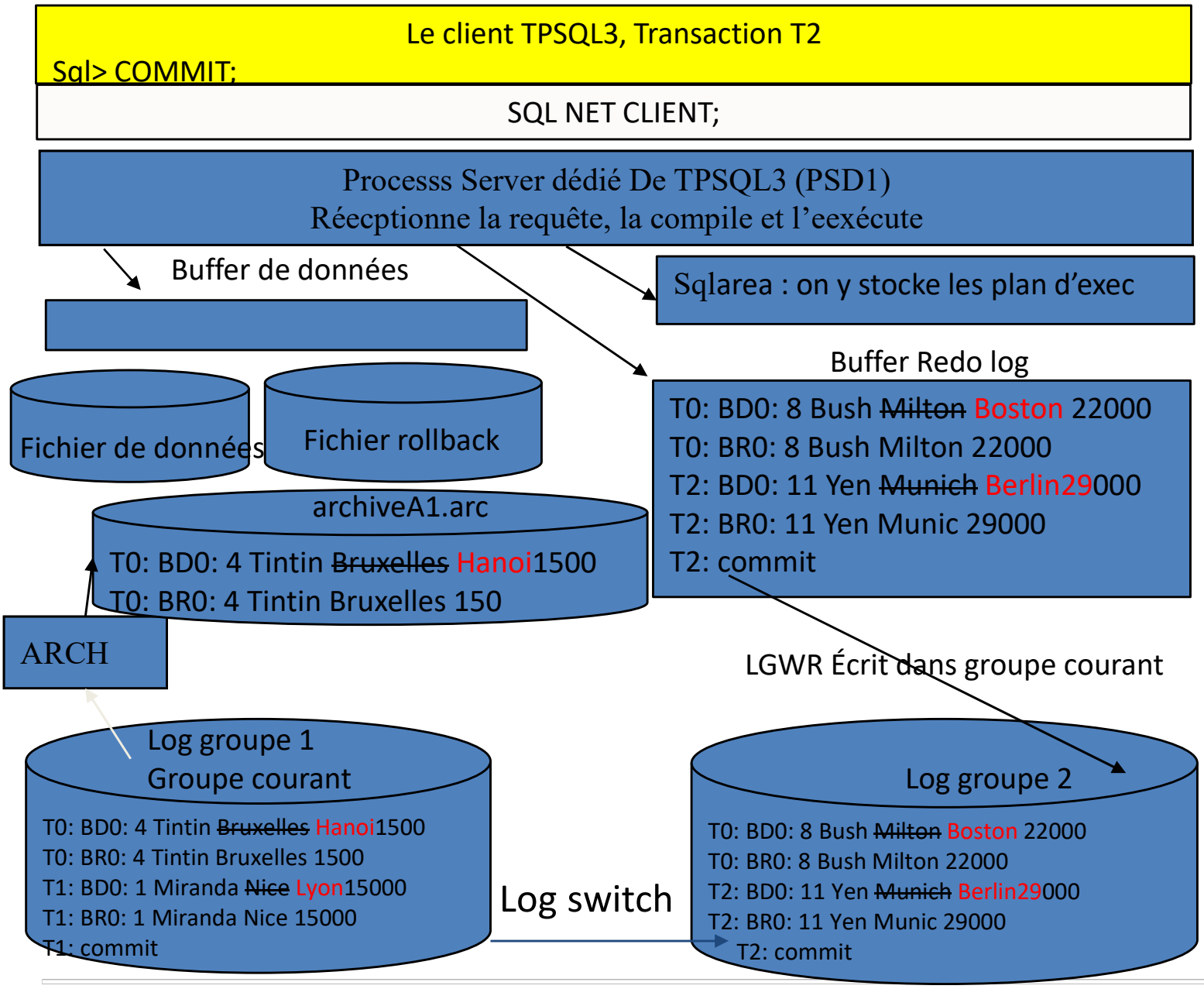
# 1.2 Problématique du Tuning

- **Surdimensionnement du buffer redolog**
  - En case de COMMIT, Aucune activité de Mise à jour n'est possible tant que le contenu du Buffer Redolog n'est pas entièrement recopié vers le bon fichier Redolog
  - Si le Buffer Redolog est LARGE=> ralentissement



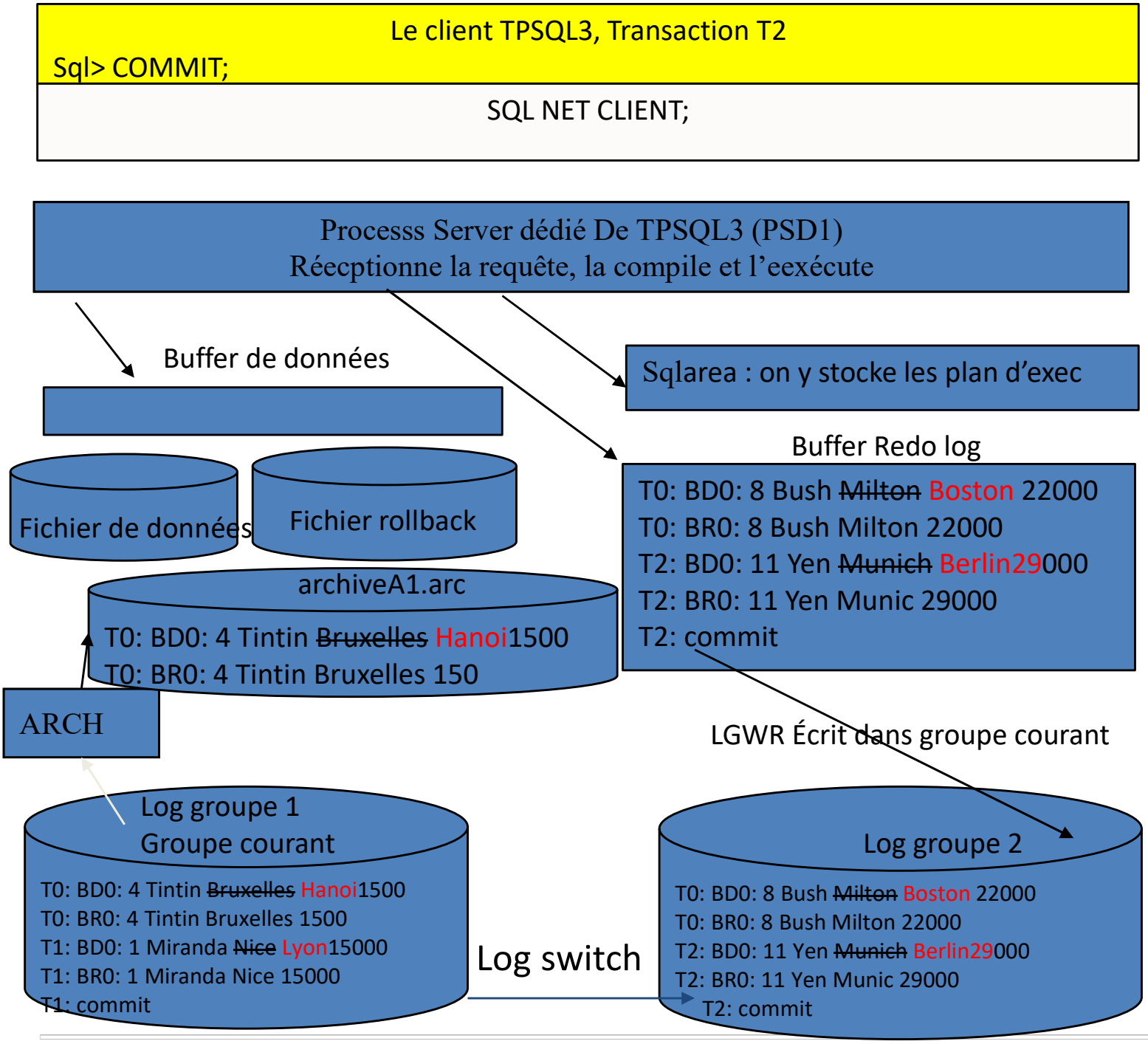
# 1.2 Problématique du Tuning

- **Sous-dimensionnements des fichiers redolog**
  - Si les fichiers Redolog sont trop petit et qu'il y a beaucoup de Mise à jours. LGWR sautera d'un groupe de Log à un autre
  - Provoquant des attentes du é l'écriture des bufferss de données et Redolog vers les bons fichiers => ATTENTES



# 1.2 Problématique du Tuning

- **Conflits entre les processus : exemple LGWR et ARCH**
  - Attentes si LGWR veut écrire dans un GROUPE alors que ARCH n'a pas fini de le sauvegarder !!!



# 1.3 Optimisation sous Oracle

ACTIVITE DANS UNE BASE DE DONNEES ORACLE= EXECUTION DE REQUETES SQL

**COUT EXECUTION = COUT CPU + COUT IO**

**Optimisation de requêtes**

**Optimisation Manuelle** des **Requêtes**
**Cours TUNE1**

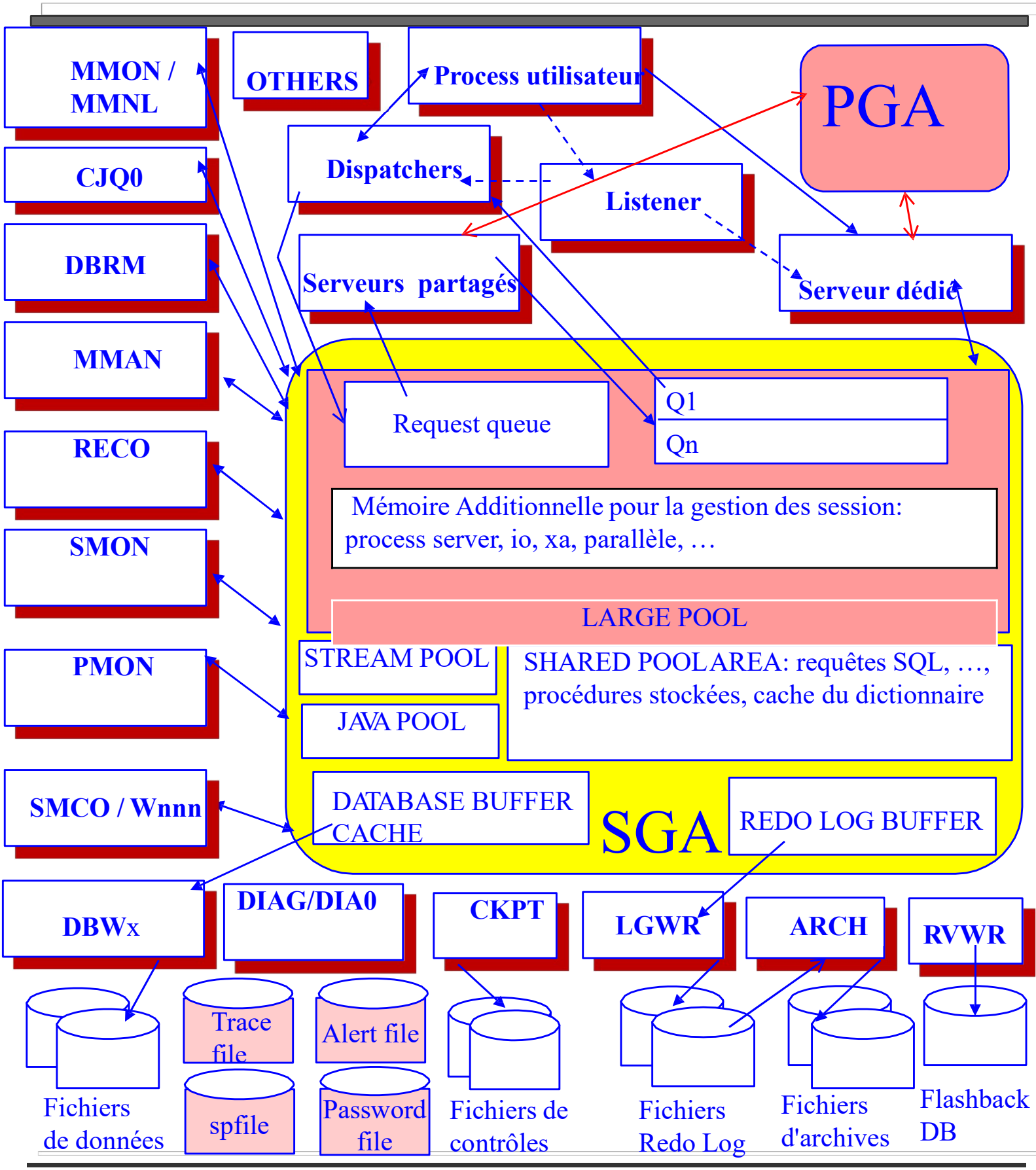
**Optimisation Automatique Des requêtes avec Des conseillers**
**Cours TUNE2**

**Optimisation de l'environnement Du SGBD** : SGA, PGA, évènements, E/S, les fichiers, les Segments, ASM, CPU, …

**Cours TUNE2**

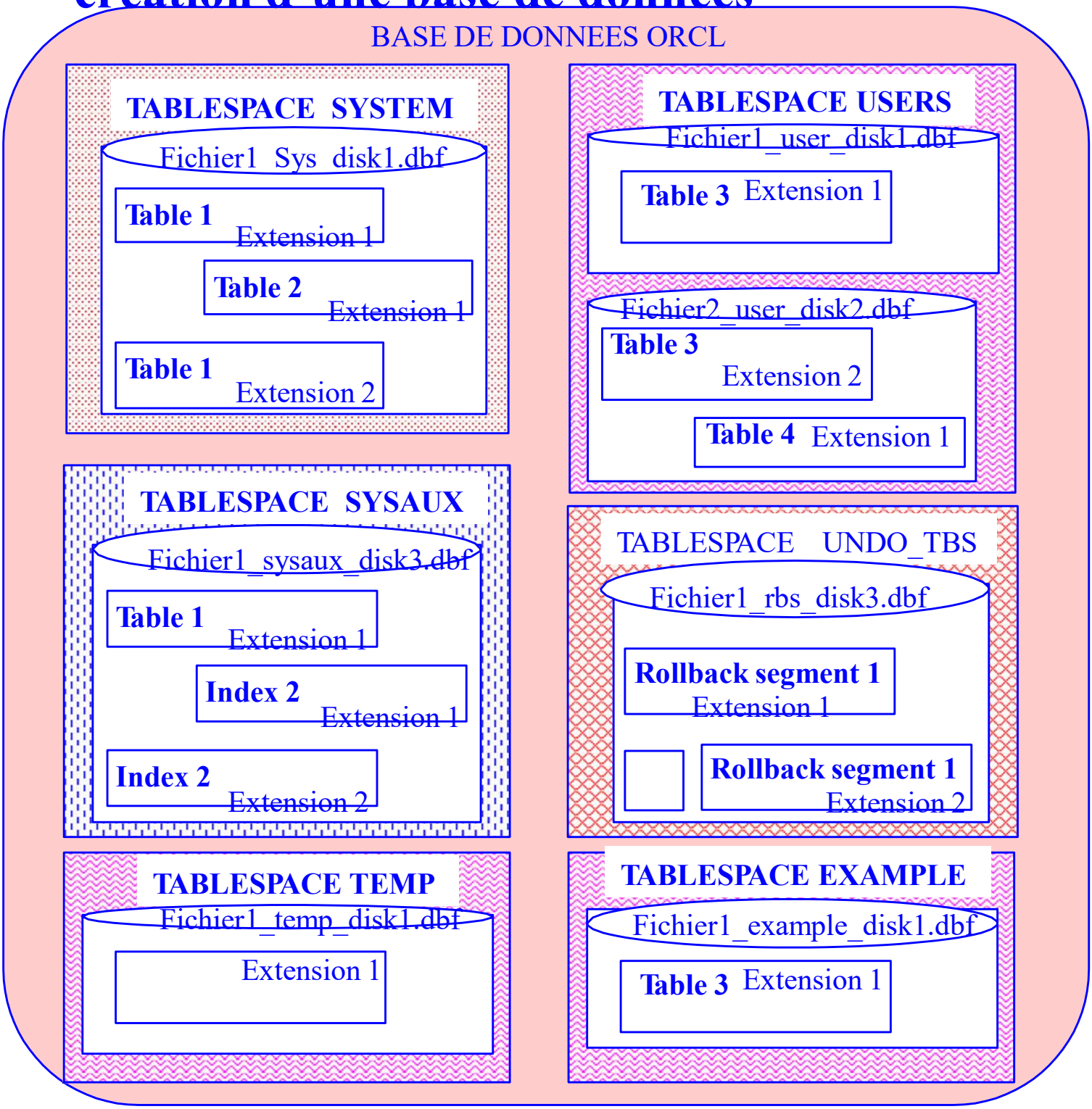
**TUNING ORACLE**
**(OBECTIF : REDUIRE COUTS EXECUTION)**

# 1.4 Optimisation de l'environnement du SGBD



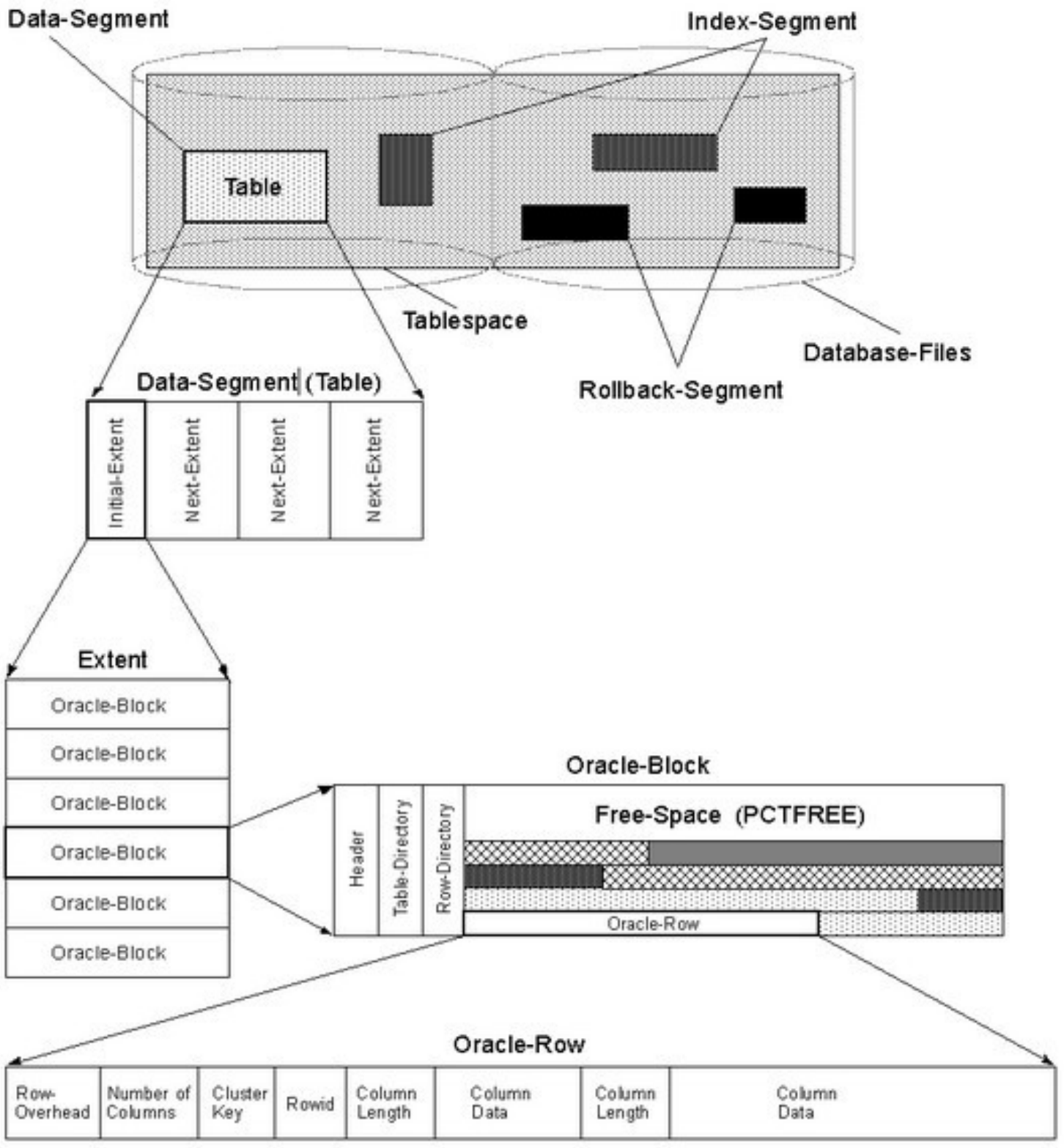
# 1.4 Optimisation de l'environnement du SGBD

- **Tablespaces, Segments, extensions, blocks à la création d'une base de données**



# 1.4 Optimisation de l'environnement du SGBD

## ■ Tablespaces, Segments, extensions, blocks



# 1.4 Optimisation de l'environnement du SGBD Oracle

## ▪ Paramètres d'initialisation importants ayant un impact sur les performances

| Paramètre | Description |
|---|---|
| COMPATIBLE | Permet de tirer parti des dernières améliorations d'une nouvelle version. |
| DB_BLOCK_SIZE | 8192 pour un système OLTP ; taille plus élevée pour un système d'aide à la décision. |
| MEMORY_TARGET | Composants de mémoire dimensionnés automatiquement |
| SGA_TARGET | Gestion automatique des composants SGA |
| PGA_AGGREGATE_TARGET | Gestion de la mémoire PGA automatique |
| PROCESSES | Nombre maximum de processus pouvant être démarrés par l'instance. |
| SESSIONS | A utiliser essentiellement avec un serveur partagé. |
| UNDO_MANAGEMENT | Mode AUTO recommandé. |
| UNDO_TABLESPACE | Tablespace d'annulation devant être utilisé par l'instance. |

# 1.4 Optimisation de l'environnement du SGBD Oracle

## Dimensionnement initial de la mémoire

- Pour effectuer une première estimation de la mémoire nécessaire, utilisez les postulats suivant :
  - Laissez 20 % de la mémoire disponible pour les autres applications.
  - Affectez 80 % de la mémoire à l'instance Oracle.

```
MEMORY_MAX_TARGET=(total_mem*80%)
```

  - Par défaut, la répartition de départ de la mémoire est de 60 % pour la mémoire SGA et 40 % pour la mémoire PGA.
  - La répartition de la mémoire s'adapte à la charge globale.

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■ Fragmentation interne : Eléments à prendre en compte

- Causes possibles :
  - Valeurs inappropriés des paramètres `PCTFREE` et `PCTUSED` pour les segments de "heap"
  - Valeurs inappropriées de `PCTVERSION` et `RETENTION` pour les segments de LOB
  - Faible densité des données dans un segment
  - Chargements de données par chemin direct suivis de suppressions (pas d'insertions)
  - Index sur des tables avec mises à jour aléatoires et suppressions sans insertions
- Solutions :
  - **Récupération en ligne de l'espace dans les segments**
  - Redéfinition en ligne
  - **Opérations `MOVE`**

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■ Taille de bloc : Avantages et inconvénients

| Taille de bloc | Avantages | Inconvénients |
|---|---|---|
| Petite | Petites lignes avec un grand nombre d'accès aléatoires | Surcharge relativement élevée (en-tête de bloc) |
| | Réduit les contentions liées aux blocs (OLTP) | Pour les lignes volumineuses (chaînage) |
| Grande | Charge globale réduite : espace plus important pour les données | Gaspillage de l'espace du cache avec des accès aléatoires à de petites lignes |
| | Ils sont adaptés à l'accès séquentiel. | Contention au niveau des blocs feuille d'index (OLTP) |

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■ Dimensionner le tampon de journalisation

- La taille du tampon de journalisation est déterminée par deux facteurs :
  - Paramètre `LOG_BUFFER`
  - Espace disponible restant dans le granule de la zone fixe

- Valeur par défaut : plus grande des deux valeurs
  2 Mo et 128 ko x `CPU_COUNT`

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■ Dimensionner les fichiers de journalisation

- **La taille des fichiers de journalisation peut avoir un impact sur les performances**.
- Des **fichiers** de **journalisation de grande taille fournissent de meilleures performances**.
- **En général, la taille des fichiers de journalisation doit être comprise entre 100 Mo et quelques gigaoctets. Plutôt quelques Gigaoctets**
- Changez de groupe de fichier de journalisation **toutes les vingt minutes** environ.
- Utilisez la fonction d'information sur la taille des fichiers de journalisation pour dimensionner correctement les fichiers de journalisation.

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■ Améliorer les performances de l'archivage

- Partagez le travail d'archivage au cours d'une augmentation temporaire de la charge globale :

```
ALTER SYSTEM ARCHIVE LOG ALL
TO <log_archive_dest>
```

- Augmentez le nombre de processus d'archivage à l'aide du paramètre LOG_ARCHIVE_MAX_PROCESSES.

- **Multiplexez les fichiers de journalisation et ajoutez des membres supplémentaires.**

- **Modifiez le nombre de destinations d'archivage :**
  - **LOG_ARCHIVE_DEST_*n***

# 1.4 Optimisation de l'environnement du SGBD Oracle

## ■Les 10 principales erreurs détectées dans les systèmes des clients

1. Gestion inappropriée des connexions
2. Utilisation incorrecte des curseurs et de la zone de mémoire partagée
3. **Instructions SQL mal écrites**
4. Utilisation de paramètres d'initialisation non standard
5. Mauvaise configuration des E/S de base de données
6. Problèmes liés à la configuration des fichiers de journalisation
7. Sérialisation des blocs de données dans le cache de tampons
8. Longs balayages complets de table
9. Nombre élevé d'instructions SQL récursives
10. Erreurs de déploiement et de migration

# 1.5 Conception optimisée d'une base

- Choix d'un Index**(clé primaire/étrangère, colonne de sélection/partitionnement,...)**
- Colonnes à ne pas indexer**(si valeurs de la colonne peu distincts)**
- Index non utilisé **(dans certains cas)**
- Choix d'un cluster (**pour accélérer les jointures**)
- Redondance calculée et dé-normalisation **(pour éviter les jointures, ...)**
- Partitionnement des tables**(horizontal)**
- Paramètres de stockage**(pctfree, pctused, ...)**

# 1.6 Approches d'optimisation de requêtes

- **Oracle propose, à partir de la version 7, deux types d'optimiseurs :**
  - l'optimiseur syntaxique basé sur les règles (RBO:Rule Based Optimizer) et l'optimiseur statistique basé sur les coûts (CBO:Cost Based Optimizer). Les versions antérieures ne proposent que l'Optimiseur syntaxique.

- **Optimiseur Syntaxique (RBO)**
  - **S'appuie 15 chemins d'accès** (chemin 1 à 15 : 1 plus rapide et 15 moins rapide)
  - **Un ensemble de règles permettent de sélectionner le chemin d'accès à priori le plus rapide**.
  - **N'a aucune statistique sur les données. Peut donc pour cela faire des choix inefficaces**
  - **Notes :**
    - **Ne doit plus être utilisé**
    - Nous l' évoquerons uniquement à des fins pédagogiques

# 1.6 Approches d'optimisation de requêtes

## ■ Optimiseur Statistique (CBO)

- **S'appuie sur les statistiques sur les tables, les indexes, les clusters et le système** (débit des disques, vitesse de la CPU, etc.)

- **Choisit le ou les chemins d'accès avec le taux de sélectivité le plus faible** (pourcentage de lignes pouvant être ramenées par une requête)

- **Calcule le coût (coût CPU et coût IO) de chaque chemin d'accès et choisit le ou les chemins avec le coût le plus faible**

- **Ainsi, l'optimiseur statistique peut décider d'effectuer un balayage séquentiel alors que l'optimiseur syntaxique choisirait d'utiliser l'index!!!**

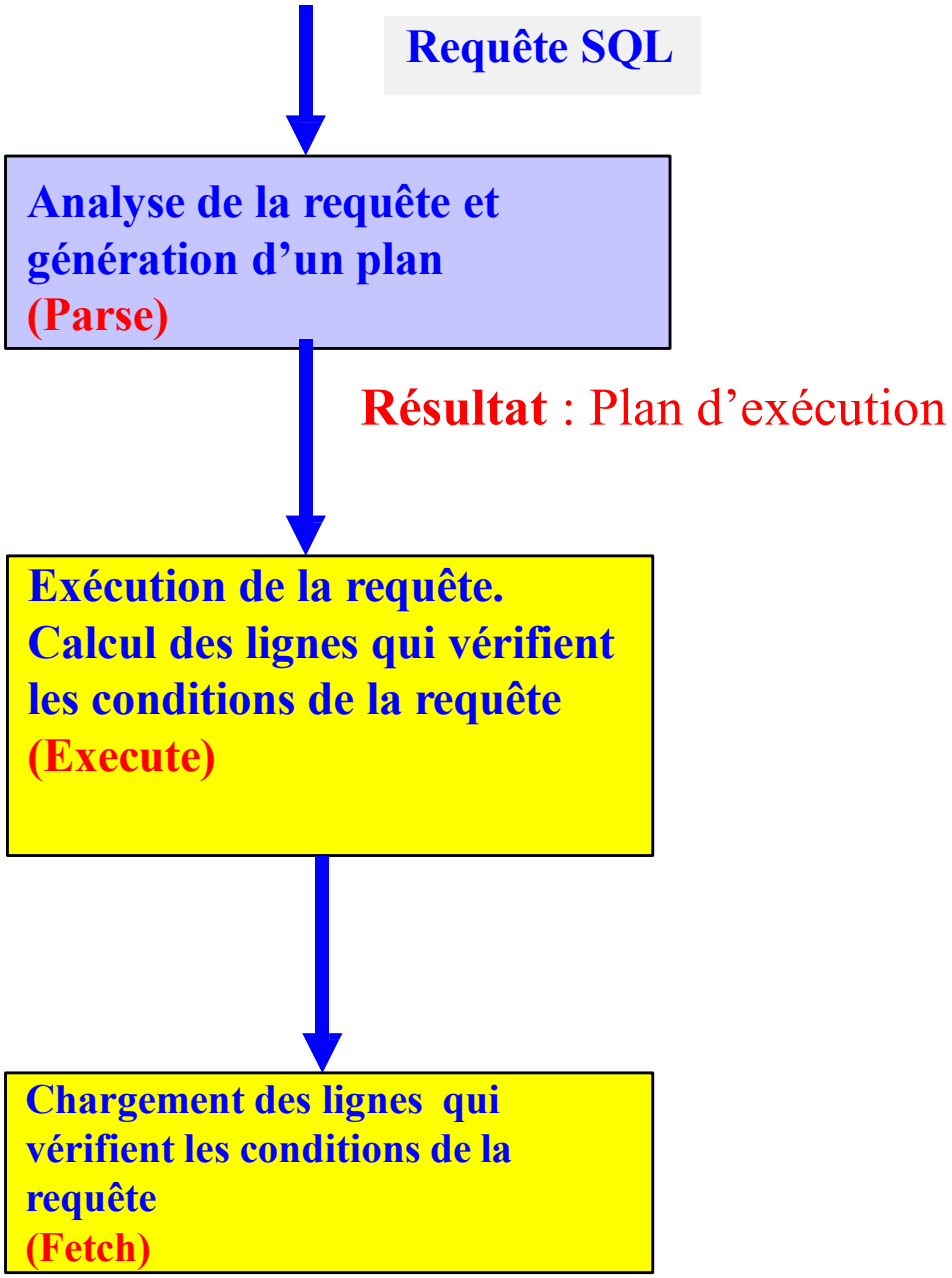
# 2. Etapes d'optimisations

- PLAN

  - **2.1 Démarche générale**
  - **2.2 Evaluation des expressions et constantes**
  - **2.3 Transformation de requêtes**
  - **2.4 Choix d'une approche d'optimisation**
  - **2.5 choix de chemins d'accès**
  - **2.6 Optimisation de la jointure**

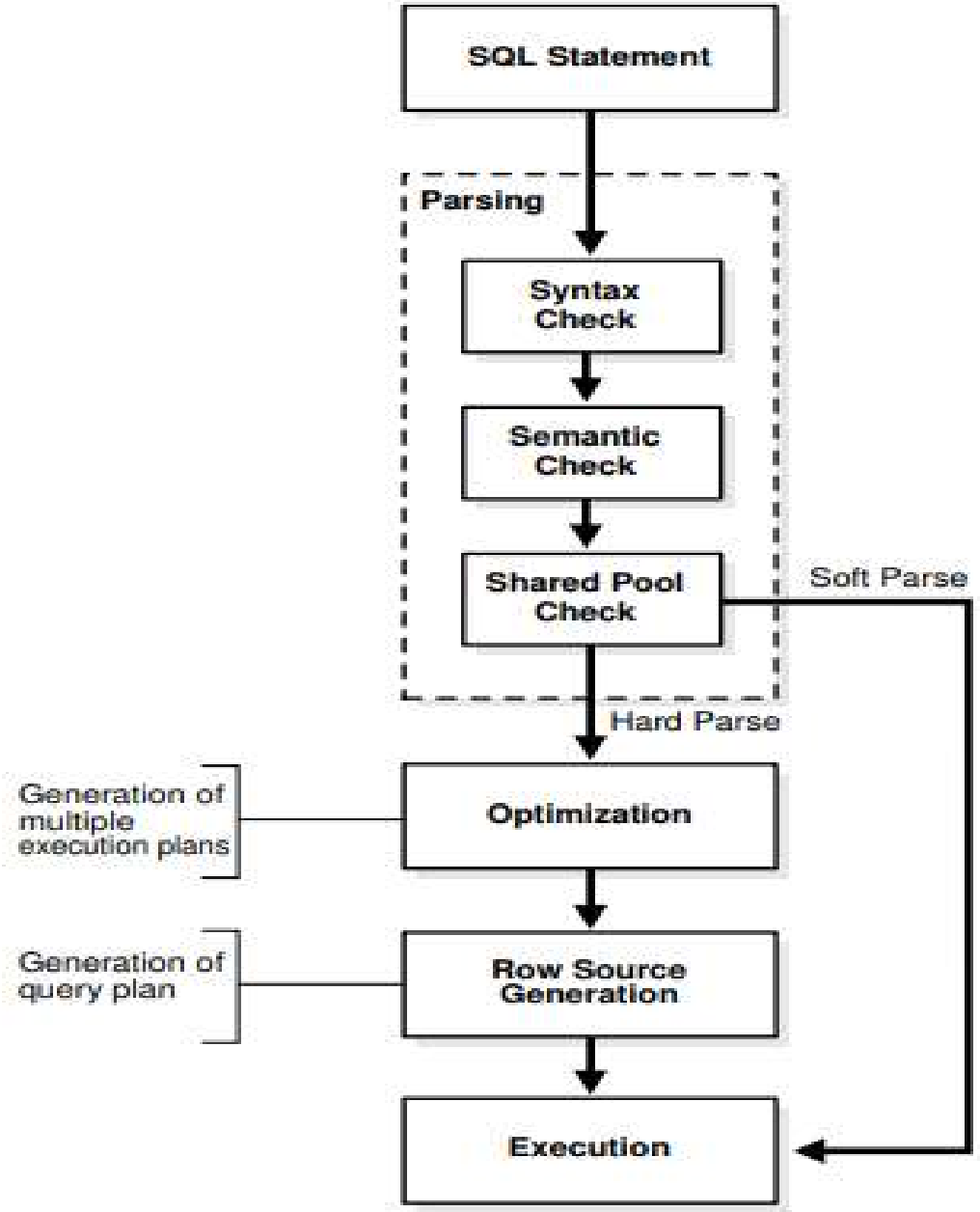
# 2.1 Démarche Générale

- **Les étapes d'exécution d'une requête SQL :**



# 2.1 Démarche Générale

## ■ Les étapes d'exécution d'une requête SQL :




**Semantic check:** vérifie entre autre l'existence des objets (tables, index, …) utilisés dans la requête SQL

# 2.1 Démarche Générale

- **L'optimiseur d'Oracle suit les étapes suivantes afin d'optimiser une requête SQL (Phase PARSE) :**
  - Evaluation des expressions et des constantes,
  - Réécriture des requêtes
  - Choix de l'approche d'Optimisation
  - Choix des chemins d'accès
  - Choix de l'ordre de jointure et des opérations de jointure
- Note :
  - Cette liste d'étapes est effectuées indépendamment de la nature de l'optimiseur.

# 2.2 Evaluation des expressions et constantes

## Evaluation d'expressions et constantes

- Lorsque l'optimiseur Oracle rencontre une expression, il tente de la transformer ou évaluer afin d'éviter la réévaluation à l'exécution.

## *Pré-évaluation des constantes*

- (expr 1) select * from emp where **sal > 24000/12**
- (expr 2) select * from emp where **sal > 2000**
- (expr 3) select * from emp where **sal*12 > 24000**

- **expr 1** *converti en* **expr 2**
- **expr 3** *pas converti en* **exp 2**

- **Note :**
  - réévaluation perpétuelle de expr 3
  - il faut éviter les expressions contenants une colonne.

# 2.3 réécriture de de requêtes

## ■ Transformation des requêtes

- L'optimiseur transforme une expression en une autre expression équivalente.

| Requête source | Requête transformée |
|---|---|
| Exp IN (v1, v2) | Exp=v1 OR Exp=v2 |
| Exp Between (v1 and v2) | exp>=v1 and exp<=v2 |
| Exp>Any ou Some(v1,v2) | Exp>v1 OR exp>v2 |
| Exp>All(v1, v2) | Exp>v1 and Exp>v2 |
| Vue | Remplacement par Table origine |
| Not != | = |
| C1=C2 and c1=exp1 | C1=exp1 and c2=exp1 |
| Résultat stockée dans une vue matérialisée (VM) | Réécriture et accès direct à laVue Matérialisée (VM) |
| | |

# 2.3 réécriture de de requêtes

## ■ Transformation par transitivité

```
ORS1 , orcl >set linesize 200
ORS1 , orcl >select * from emp, dept where emp.deptno=10
  2  and emp.deptno=dept.deptno;


Plan d'exécution
----------------------------------------------------------
Plan hash value: 568005898

-----------------------------------------------------------------------------------------
| Id  | Operation                     | Name    | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT              |         |     3 |   177 |     4   (0)| 00:00:01 |
|   1 |  NESTED LOOPS                 |         |     3 |   177 |     4   (0)| 00:00:01 |
|   2 |   TABLE ACCESS BY INDEX ROWID| DEPT    |     1 |    20 |     1   (0)| 00:00:01 |
|*  3 |    INDEX UNIQUE SCAN          | PK_DEPT |     1 |       |     0   (0)| 00:00:01 |
|*  4 |   TABLE ACCESS FULL           | EMP     |     3 |   117 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("DEPT"."DEPTNO"=10)
   4 - filter("EMP"."DEPTNO"=10)
```

```
ORS1 , orcl >select * from emp, dept where emp.deptno=10
  2  AND dept.deptno=10;


Plan d'exécution
----------------------------------------------------------
Plan hash value: 568005898

-----------------------------------------------------------------------------------------
| Id  | Operation                     | Name    | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT              |         |     3 |   177 |     4   (0)| 00:00:01 |
|   1 |  NESTED LOOPS                 |         |     3 |   177 |     4   (0)| 00:00:01 |
|   2 |   TABLE ACCESS BY INDEX ROWID| DEPT    |     1 |    20 |     1   (0)| 00:00:01 |
|*  3 |    INDEX UNIQUE SCAN          | PK_DEPT |     1 |       |     0   (0)| 00:00:01 |
|*  4 |   TABLE ACCESS FULL           | EMP     |     3 |   117 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("DEPT"."DEPTNO"=10)
   4 - filter("EMP"."DEPTNO"=10)
```

# 2.3 réécriture de de requêtes

- **Transformation par réécriture de requête avec accès à la vue matérialisée**

  - **Les paramètres suivants doivent avoir au moins les valeurs ci-dessous** :
    - job_queue_processes = 10 (doit être >0)
    - optimizer_mode = ALL_ROWS (doit être all_rows, first_rows_n)
    - query_rewrite_enabled =TRUE (doit être TRUE)
    - query_rewrite_integrity =enforced (doit être enforced)

  - **Exemple**

```
// Ecrire une requête qui permet de calculer la somme
// des salaires par nom de département et par métier.

Execute Dbms_stats.gather_schema_stats('MOPOLO');
Set autotrace on;
set linesize 200

select dname, job, sum(sal)
from dept4, emp4
where emp4.deptno=dept4.deptno group by dname, job;
```

# 2.3 réécriture de de requêtes

- **Transformation par réécriture de requête avec accès à la vue matérialisée**
  - **Exemple**
    - **Plan sans vue matérialisée**

```
-----------------------------------------------------------------------------------------
| Id  | Operation                    | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |          |    11 |   407 |  2303   (6)| 00:00:28 |
|   1 |  HASH GROUP BY               |          |    11 |   407 |  2303   (6)| 00:00:28 |
|   2 |   MERGE JOIN                 |          |    11 |   407 |  2302   (6)| 00:00:28 |
|   3 |    TABLE ACCESS BY INDEX ROWID| DEPT4    |     4 |    52 |     2   (0)| 00:00:01 |
|   4 |     INDEX FULL SCAN          | PK_DEPT4 |     4 |       |     1   (0)| 00:00:01 |
|*  5 |    SORT JOIN                 |          |    11 |   264 |  2300   (6)| 00:00:28 |
|   6 |     VIEW                     | VW_GBC_5 |    11 |   264 |  2299   (6)| 00:00:28 |
|   7 |      HASH GROUP BY           |          |    11 |   165 |  2299   (6)| 00:00:28 |
|   8 |       TABLE ACCESS FULL      | EMP4     | 1300K|    18M| 2215   (2)| 00:00:27 |
-----------------------------------------------------------------------------------------
```

    - **Création de la vue matérialisée**

CREATE MATERIALIZED **VIEW mv_masse_salariale**
Refresh complete
enable query rewrite
As select dname, job, sum(sal) from dept4, emp4
where emp4.deptno=dept4.deptno group by dname, job;

# 2.3 réécriture de de requêtes

- **Transformation par réécriture de requête avec accès à la vue matérialisée**
  - **Exemple**
    - **Réécriture avec accès à la vue matérialisée**

Execute
Dbms_stats.gather_schema_stats('MOPOLO');
Set autotrace on;
set linesize 200

select dname, job, sum(sal) from dept4 d, emp4 e
where d.deptno=e.deptno group by dname, job;

```
---------------------------------------------------------------------------------------
| Id  | Operation                | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                 |     9 |   207 |     3  (0)| 00:00:01 |
|   1 |  MAT_VIEW REWRITE ACCESS FULL| MV_MASSE_SALARIALE |     9 |   207 |     3  (0)| 00:00:01 |
---------------------------------------------------------------------------------------
```

**COUT =3 contre 2303 sans réécriture**

# 2.4 Choix de l'optimiseur (règle ou statistique)

- **Le choix de l'optimiseur (règle ou statistique) dépend des facteurs suivants :**
  - La valeur du paramètre OPTIMIZER_MODE
  - Les suggestions (HINTS) dans la requête obligeant l'utilisation d'un optimseur.

- **Le Paramètre OPTIMIZER_MODE**
  - Permet de choisir mode d'optimisation
  - Ce paramètre est modifiable :
    - **Pour une SESSION** à travers l'ordre SQL ALTER SESSION SET optimizer_mode=valeur
    - **Pour une INSTANCE** à travers l'ordre SQL alter system set optmimizer_mode=valeur scope=memory
    - **Pour toutes les nouvelles INSTANCES** à travers l'ordre SQL : Alter system set optmimizer_mode=valeur scope=spfile (édite le fichier spfileSID.ora) ou en éditant manuellement le fichier initSID.ora

# 2.4 Choix de l'optimiseur (règle ou statistique)

- **Le paramètre OPTIMIZER_MODE prendre les valeurs suivantes :**
  - **CHOOSE** : choix automatique entre les règles et les coûts en fonction de la présence ou non des statistiques. A éviter.
  - **ALL_ROWS** : oblige l'optimiseur à utiliser l'approche basée sur les coûts. BUT : temps minimum pour ramener l'ensemble des tuples. C'EST AUSSI SA VALEUR PAR DEFAUT.

    **Alter session set optimizer_mode=all_rows;**
  - **FIRST_ROWS[_n]** : cette valeur oblige l'optimiseur à utiliser l'approche basée sur les coûts pour toute requête en présence ou non de statistiques dans le dictionnaire. But : meilleurs temps de réponse.
    - Nota : n nombre de lignes {1, 10, 100, 1000}

    **Alter session set optimizer_mode=first_rows_1;**
  - **RULE** : cette valeur oblige l'optimiseur à utiliser l'approche basée sur les règles qu'ils y aient ou non des statistiques. A éviter.

    **Alter session set optimizer_mode=RULE;**

# 2.5 Choix de chemins d'accès

## ■ Généralités

- L'accès à un tuple lors de l'exécution d'une requête SQL (indépendance logique/physique) peut se faire à travers **un ou plusieurs chemins d'accès**. **Le rôle de l'optimiseur est de sélectionner le meilleurs d'entre eux**. Pour accéder aux données, Oracle s'appuie sur plusieurs méthodes d'accès de base.

- **Nous survolons ici rapidement :**

  – **Les méthodes d'accès d'Oracle**

  – **Mode de choix d'un chemin d'accès dans l'approche règle (Optimizer_mode=rule)**

  – **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

# 2.5 Choix de chemins d'accès

## Les Méthodes d'accès

- Accès par ROWID
- Accès par Index B-arbre,
- Accès par index **Bitmap (depuis oracle 8)**
- Accès par cluster indexé
- Accès par cluster haché
- Accès séquentiel
- **Table organisée dans un index (Oracle 8)**
- **Index basés sur une fonction**
- **Index de domaines** (ou Index créés par l'utilisateur, non traités dans ce cours…)

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**
  - **Calculer les statistiques permettant de déterminer le coût de chaque chemin :**
    - Statistiques sur les tables et les clusters (nombre de lignes, nombre de blocs utilisés, nombre de blocs jamais utilisés, ...)
    - Statistiques sur les colonnes (facteur de filtrage ou Densité, nombre de valeurs distinctes, …)
    - Statistiques sur sur les index(niveau d'index, nombre de feuilles, ...) ...
    - Les statistiques systèmes (Vitesse de transfert de données, Volume de transfert par seconde, cycle CPU, …)
  - **Les étapes pour choisir un chemin d'accès sont les suivantes :**
    - 1. Déterminer les chemins d'accès valides
    - **2. Estimer le coût de chacun des chemins d'accès ou leur combinaison(plan d'accès).**

      COUT CPU+COUT IO.
    - 3. Choisir le plan avec le(s) plus faible coût(s)

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **Estimer le coût de chacun des chemins d'accès ou leur combinaison(plan d'accès).**

    COUT= COUT CPU+COUT IO. Le meilleur chemin est celui avec le coût le plus faible. Le coût le plus faible est déterminé en fonction de :

    **1. La sélectivité**

    - Privilégie le passage par les indexes des prédicats avec taux de sélectivité faible (peu de blocks chargé : Index+ table, mode monobloc) et que le coût du balayage séquentiel (full) est plus élevé ainsi qu'il n'y a pas de HINTS

    **2. La lecture multiple de blocs**

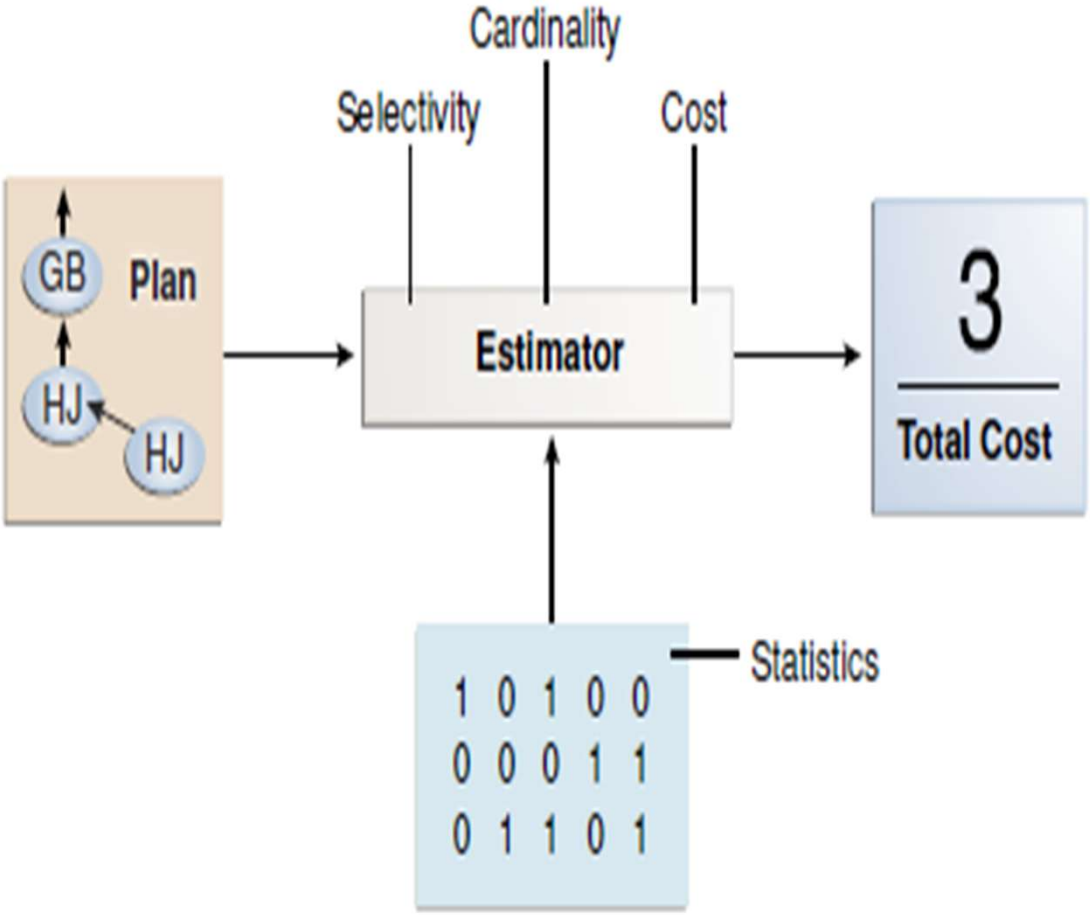
    - Privilégie le balayage séquentiel (full) si le passage par les indexes puis la table (mode monobloc) est plus couteux que la lecture de tous les blocks (mode multi-blocs). Aucun HINT n'est posé

    **3. Les HINTS influençant les chemins d'accès**

    - **Suggestions contraignantes, l'optimiseur suit toujours les HINTS, s'ils sont réalisables**

# 2.5 Choix de chemins d'accès

- **Estimation du coût pour un plan d'exécution**



# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **La sélectivité**

    - **La sélectivité est le pourcentage de lignes pouvant être sélectionnées dans une table par une requête.**

      - Faible pourcentage => bonne sélectivité **car peu de block à charger**
      - Pour cela l'optimiseur privilégie l'accès par index

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **La sélectivité** (Relation entre sélectivités et prédicats)

    – Oracle fait apparaître les filtres utilisés pour calculer le nombre des lignes ramenées dans les plans

    – La valeur du facteur de filtrage (FF) pour une requête utilisant un prédicat qui implique une seule colonne avec condition d'égalité est :

    **FF= 1/NDV** ou NDV correspond au Nombre De Daleurs distinctes. FF est aussi appelé la densité.

  - **Sans variable de liaison, sans histogrammes et si les colonnes sont déclarées non nulles**

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **La sélectivité** (Relation entre sélectivités et prédicats)

| Prédicat | Facteur de Filtrage |
|---|---|
| C1 = valeur | c1.density |
| C1 like valeur | c1.density |
| C1 > valeur | (Hi - valeur) / (Hi - Low) |
| C1 >= valeur | (Hi - valeur) / (Hi - Low) + 1 / c1.num_distinct |
| C1 < valeur | (Valeur - Low) / (Hi - Low) |
| C1 <= valeur | (Valeur - Low) / (Hi - Low) + 1 / c1.num_distinct |
| C1 between val1 and val2 | (Val2 - Val1) / (Hi - Low) + 2 * 1 / c.num_distinct |

**Note : Sans variable de liaison, sans histogrammes et si les colonnes sont déclarées non nulles**

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**
  - **La sélectivité** (Relation entre sélectivités et prédicats)

### – Avec variables de liaison

| Prédicat | Facteur de Filtrage |
|---|---|
| C1 = :b1 | c1.density |
| C1 {like \| < \| <= \|> \| >=} :b1 | { 5.0e-02 \|c1.density } |
| C1 between :b1 and :b2 | 5.0e-02 * 5.0e-0.2 |

### – La combinaison des prédicats

| Prédicat | Facteur de Filtrage |
|---|---|
| prédicat1 and prédicat2 | FF1 * FF2 |
| prédicat1 or prédicat2 | FF1 + FF2 - FF1 * FF2 |

### – Note :

- density, high_value (Hi), low_value (Low), num_distinct sont des colonnes de la vue (dba|all|user)_tab_columns.

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **La sélectivité** (Relation entre sélectivités et prédicats)

  - **Exemple 1**

    SELECT * FROM emp WHERE ename='KING';

    – ***Le nombre de lignes raménées dépend du facteur de filtrage***. Dans notre cas le facteur de filtrage sera :

    Ename.density= 1/User_tab_columns.num_distinct

    S'il y'a 14 noms distincts

    Ename.density=1/14=0,071428571

    Nb de lignes ramenées Par Clé =
    **ename.density*user_tables.num_rows**

    Si user_tables.num_rows=14

    Nb de lignes ramenées Par Clé =0,071428571*14

    Nb de lignes ramenées Par Clé =1

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **Lecture de blocs multiples**

    - L'optimiseur peut décider d'effectuer un accès séquentiel à une table en fonction du facteur de blocage qui permet à Oracle de *charger plusieurs blocs en une lecture*. Le paramètre déterminant ici est :

    - DB_FILE_MULTIBLOCK_READ_COUNT.

    - **Exemple**
      - ✓ select * FROM emp WHERE ename='KING'
      - ✓ Si le **balayage** complet est **plus rapide que le balayage via un Index sur ename**, l'optimiseur choisira un balayage complet.

# 2.5 Choix de chemins d'accès

- **Mode de choix d'un chemin d'accès dans l'approche coûts (optimizer_mode= firts_rows[_n] ou all_rows).**

  - **La HINTS (Suggestions) influençant les chemins d'accès**

    - **L'optimiseur suit toujours les HINTS même s'il y a de meilleurs chemins**

    - **Exemple**

      - **Forcer l'usage d'un Index par exemple**

SELECT /*+ INDEX(emp idx_sex) Utilisation de l'index sur la colonne sex car peu de Masculin*/
* FROM emp
WHERE sex='M';

Note : Les détails sur les HINTS sont développés plus loin avec l'optimiseur statistique.

# 2.6 Optimisation de la jointure

## Les différentes méthodes de Jointure

- Une opération de jointure permet de concaténer les lignes de deux ou plusieurs tables

- Les différentes méthodes de jointures sont:
  - Jointure boucle imbriquée (Nested loop join)
  - Jointure par hachage (Hash join)
  - Jointure tri-fusion (Sort-merge join)
  - Jointure cartésienne (Cartesian Join)
  - Jointure via les clusters indexés ou hachés

- Nota :
  - Nous discuterons des jointures via les clusters indexés ou hachés plus tard

# 2.6 Optimisation de la jointure

## ■ Jointure boucle imbriquée

- Une des deux tables est définie comme table directrice (table externe)
- L'autre table est appelée table interne
- Pour chaque ligne de la table extene, on recherche toutes les lignes qui vérifient la condition de jointure dans la table interne

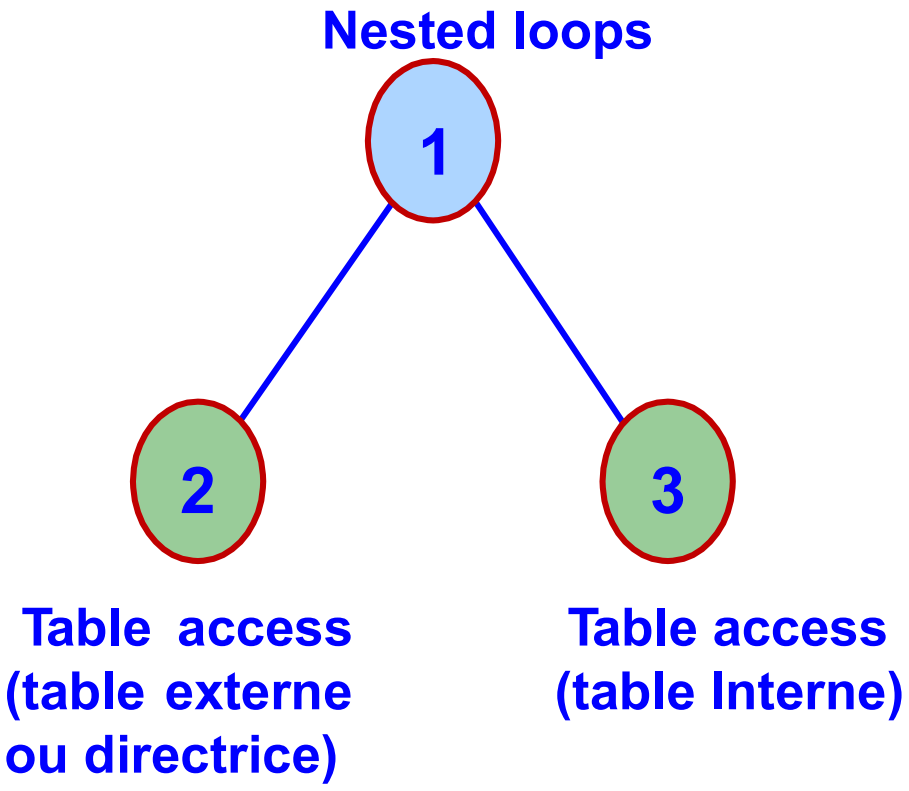
Pour chaque ligne
De la table directrice

Pour chaque ligne de
La table interne

Vérifier que le prédicat
De jointure est vrai

# 2.6 Optimisation de la jointure

## ■ Jointure boucle imbriquée



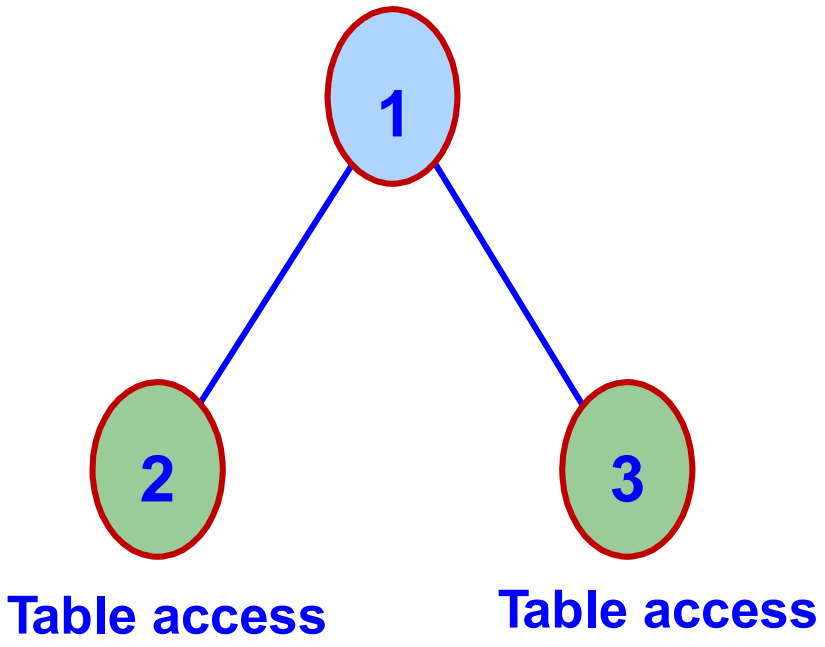

## ■ Quand utiliser les jointures boucle imbriquées

- Sur des tables avec un volume faible de lignes et une bonne condition de jointure
- La ou l'ordre des tables est important
- `Via une suggestion USE_NL(table1 table2)`

# 2.6 Optimisation de la jointure

## ■ Jointure par hachage

- Une jointure par hachage s'effectue comme suit:
  - Les deux tables sont partitionnées en autant de partitions que nécessaire, via accès full aux tables
  - Pour chaque paire de partition, une table de hachage est construite en mémoire sur la petite partition
  - L'autre partition est utilisée pour tester la condition de hachage



# 2.6 Optimisation de la jointure

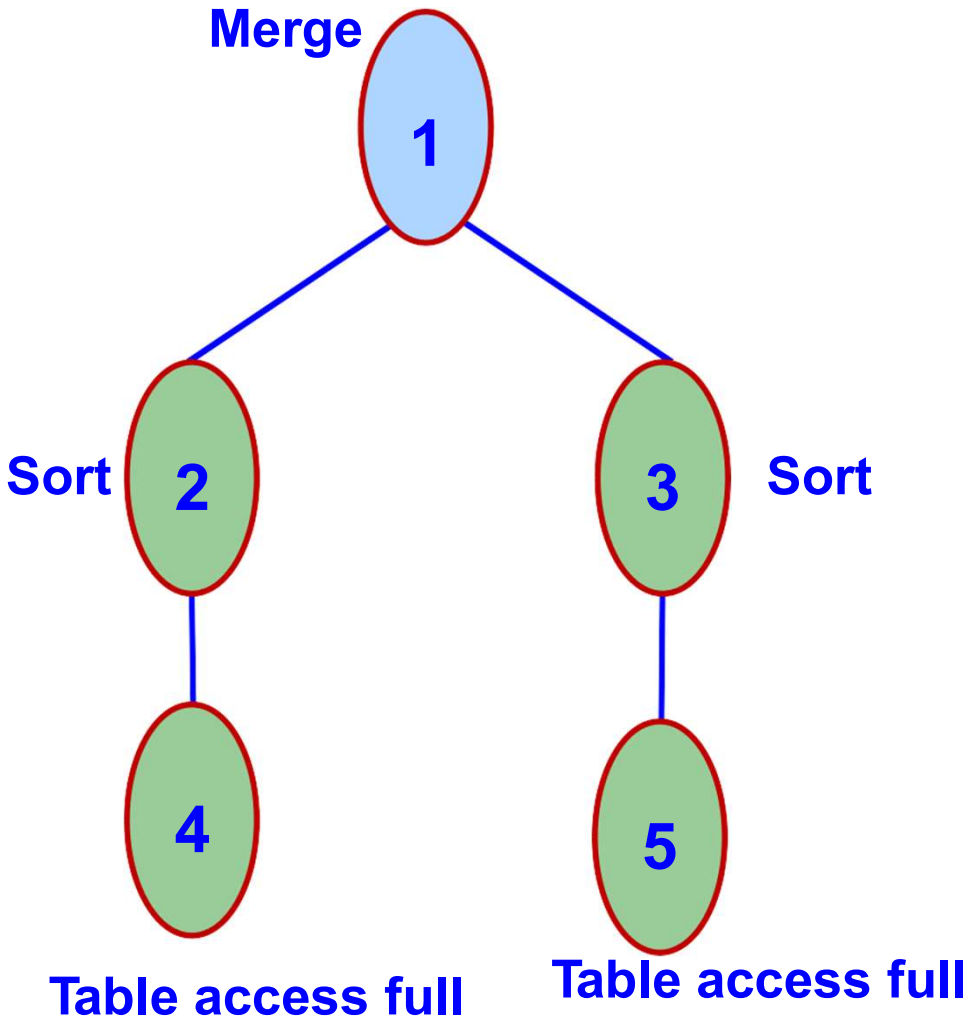
## ■ Jointure par hachage

- Quand oracle utilise les jointures par hachage
  - Jointure des tables ayant un grand volume de données (au moins une des tables est très large).
  - Jointure de grosses fractions de tables
  - Uniquement en cas d'équi-jointure
  - Ou en utilisant la suggestion USE_HASH (table1 table2 …) .

# 2.6 Optimisation de la jointure

## ■ Sort-Merge Joins

- La jointure par tri fusion s'exécute comme suit :
  - 1. Les lignes de chaque table sont triées sur les colonnes de jointure
  - 2. Fusion des deux tables triées.



# 2.6 Optimisation de la jointure

- **La jointure tri-fusion est utilisée par Oracle dans les conditions suivantes :**

  - Il ne s'agit pas d'une équi-jointure (=) ou d'une négation de l'équi-jointure (!=) mais plutôt de (>, >=, <, <=)
  - Les tris sont nécessaires pour d'autres opérations
  - Les colonnes du prédicat de jointure ne sont pas indexées ou les index ne sont pas utilisés
  - Les données sont déjà triées sur les colonnes de jointure
  - En cas de suggestion USE_MERGE(table1 table2 …)

# 2.6 Optimisation de la jointure

- **Jointure cartésienne (Cartesian Join)**

  - Il s'agit d'une jointure pour laquelle aucun prédicat de jointure n'est prévu. Concaténation de toutes lignes de la table 1 avec les ligne de la table 2

    SELECT * FROM EMP, DEPT;

    – A éviter
    – Sql tuning advisor le détecte

- **Jointure cluster indexé (voir chapitre 3)**

- **Jointure cluster haché (voir chapitre 3)**

# 2.6 Optimisation de la jointure

- **Comment l'optimiseur choisi les plans d'exécution pour les jointures? Le choix est fait en fonction de :**
  - Du mode de l'optimiseur (Règle, statistiques)
  - La source des lignes (les tables impliquées)
  - Du type de jointure (Equi-jointure, Jointure externe, Anti-jointure : Not IN, Semi-Jointure : Exist, etc.)
  - De la méthode de jointure (Nested Loop, Merge Join ou Tri fusion, Hash join, Cluster)
  - Des coûts des requêtes :
    - Coûts I/O
    - Coût CPU
  - Des suggestions (HINTS) : USE_NL, USE_MERGE, USE_HASH, …
  - Du facteur de blocage
    - `DB_FILE_MULTIBLOCK_READ_COUNT`

# 2.6 Optimisation de la jointure

- **Le coût d'une jointure imbriquée est déterminé par:**
  - le coût de la lecture séquentielle de la table directrice
  - le coût d'accès aux lignes de la table interne
  - L'optimiseur calcule ces coûts en se basant sur les statistiques.
    - ✓ *Table directrice => celle ramenant le moins de lignes*

- **Coût d'une jointure par tri-fusion**
  - Est déterminé par le coût du chargement et du tri
  - Facteurs supplémentaires pour estimer le coût :
    - ✓ une petite zone de tri pénalise le tri-fusion (contrôle via le paramètre SORT_AREA_SIZE dans init.ora)
    - ✓ un grand facteur de blocage fait décroître le coût de la jointure par tri/fusion. La valeur de DB_FILE_MULTIBLOCK_READ_COUNT peut provoquer à ce que la table non directrice soit accéder séquentiellement alors qu'un Index existe.

# 2.6 Optimisation de la jointure

- **Le coût d'une jointure par hachage est déterminée par :**
  - La place occupée en mémoire par la taille de la petite table qui doit être hachée

# 2.6 Optimisation de la jointure

- *Exemple :*

  - le choix de l'optimiseur statistique n'est pas forcément celui de l'optimiseur syntaxique :

    – Soient les tables Emp et Dept et les index sur ename, deptno dans emp et deptno dans dept

```
SQL> Alter session set OPTIMIZER_MODE=rule;

SQL> SELECT EMPNO, ENAME, dept.DEPTNO,
     DNAME  FROM dept, emp
WHERE emp.deptno=dept.deptno ;
```

```
-----------------------------------------------
| Id  | Operation                     | Name    |
-----------------------------------------------
|   0 | SELECT STATEMENT              |         |
|   1 |  NESTED LOOPS                 |         |
|   2 |   TABLE ACCESS FULL           | EMP     |
|   3 |   TABLE ACCESS BY INDEX ROWID| DEPT    |
|*  4 |    INDEX UNIQUE SCAN          | PK_DEPT |
-----------------------------------------------
```

# 2.6 Optimisation de la jointure

- *Exemple :*

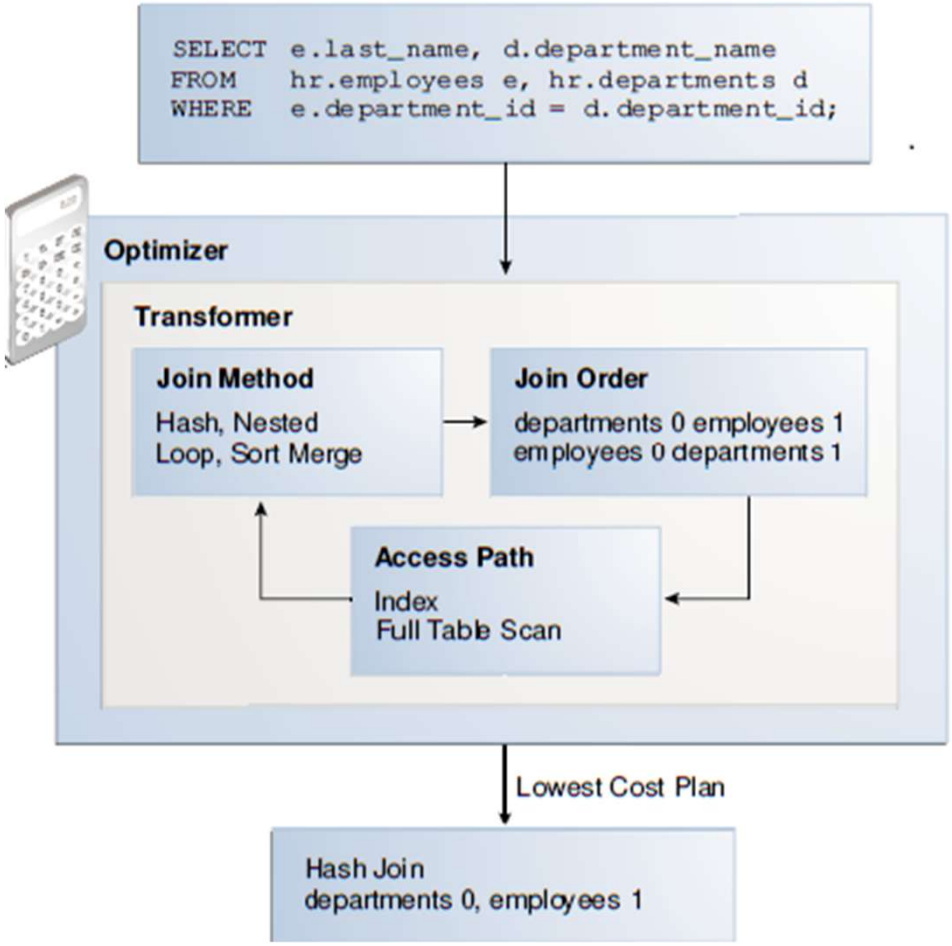
```
SQL> execute dbms_stats.gather_schema_stats('ORS1');
SQL> Alter session set OPTIMIZER_MODE=all_rows;

SQL> SELECT EMPNO, ENAME, dept.DEPTNO,
     DNAME FROM dept, emp
     WHERE emp.deptno=dept.deptno ;
```

```
-----------------------------------------------------------------------------------------
| Id  | Operation                    | Name         | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |              |    14 |   364 |     4   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP          |     4 |    52 |     1   (0)| 00:00:01 |
|   2 |   NESTED LOOPS               |              |    14 |   364 |     4   (0)| 00:00:01 |
|   3 |    TABLE ACCESS FULL         | DEPT         |     4 |    52 |     3   (0)| 00:00:01 |
|*  4 |    INDEX RANGE SCAN          | IDX_EMP_DEPTNO |     5 |       |     0   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------
```

# 2.6 Optimisation de la jointure

- *Exemple de génération d'un plan pour la jointure*



# 3. Méthodes d'accès

- PLAN
  - **3.1 Généralités**
  - **3.2 Accès séquentiel**
  - **3.3 Accès par ROWID**
  - **3.4 Accès indexé (B-arbre, Bitmap)**
  - **3.5 Table organisée dans un index**
  - **3.6 Accès par clusters indexés**
  - **3.7 Accès par clusters Hachés**
  - **3.8 Accès par index basés sur des fonctions**
  - **3.9 Mesures**
  - **3.10 Identification des Indexes non utilizes**
  - **3.11 Condition d'utilisation des indexes par l'optimiseur Oracle**
  - **3.12 Exercices**

# 3.1 Généralités

- **Un SGBD est caractérisé par une *indépendance entre la façon dont les données sont stockées et comment y accéder*.**

- **L'accès à un tuple donné peut se faire suivant différentes méthodes ou combinaison de méthodes d'accès.**

- **Oracle Offre pour cela plusieurs méthodes d'accès aux données :**
  - L'accès par **balayage séquentiel** (FULL TABLE SCAN)
  - L'accès direct par **ROWID**
  - L'accès par **index** (INDEX SEARCH)
  - L'accès par **balayage séquentiel via l'index** (INDEX FAST FULL SCAN)
  - L'accès par **index bitmap**: ***normal, join*** (à partir de V8.0)
  - L'accès par **cluster indexé** (à partir d'Oracle7)
  - **Table organisée dans un Index** (à partir de V8.0)
  - L'accès par **clé de hachage** (à partir d'Oracle7)
  - **Index de domaines** (méthodes d'indexation inventées par l'utilisateur et intégrée au noyau Oracle : non traitées dans ce cours…)

# 3.2 Accès séquentiel

- **La méthode d'accès séquentiel correspond à un balayage complet d'une table (pas d'utilisation d'accélérateurs tels que les index, ...).**

- *Exemple*

  - **select** ename **from** emp ;

```
-----------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |    14 |   518 |     3   (0)| 00:00:01 |
|   1 |  TABLE ACCESS FULL| EMP  |    14 |   518 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------
```

  - *Aucun index n'est utilisé ici*. Nous verrons plus loin comment l'optimiseur décide d'effectuer ou non un balayage séquentielle.

- *Remarque*
  - Un des moyens d'accélérer l'accès aux données en accès séquentiel est d'augmenter le facteur de blocage.
  - db_file_multiblock_read_count

# 3.3 Accès par ROWID

- **Un rowid est l'identifiant unique d'une ligne dans une base Oracle. Toute table Oracle contient une colonne virtuelle appelée rowid.**

- *Rowid sous Oracle 7*

**SQL> select rowid, ename, job from emp;**

| ROWID | ENAME | JOB |
|---|---|---|
| 00000012.0002.0002 | WARD | SALESMAN |
| 00000012.0003.0002 | JONES | MANAGER |
| 00000012.0004.0002 | MARTIN | SALESMAN |
| 00000012.0005.0002 | BLAKE | MANAGER |
| 00000012.0006.0002 | CLARK | MANAGER |
| 00000012.0007.0002 | SCOTT | ANALYST |
| 00000012.0008.0002 | KING | PRESIDENT |
| 00000012.0009.0002 | TURNER | SALESMAN |
| 00000012.000A.0002 | ADAMS | CLERK |
| 00000012.000B.0002 | JAMES | CLERK |
| 00000012.000C.0002 | FORD | ANALYST |
| 00000012.000D.0002 | MILLER | CLERK |
| 00000012.000E.0002 | SMITH | CLERK |

# 3.3 Accès par ROWID

## ■ Rowid sous Oracle 7

| ROWID | ENAME | JOB |
|---|---|---|
| 00000012.0002.0002 | WARD | SALESMAN |
| I1 I2 I3 | | |

- **Avec**
- **I1** : Identificateur logique d'un bloc (4 caractères)
- **I2** : Identificateur logique d'une ligne (2 caractères)
- **I3** : Identificateur logique d'un fichier(2 caractères)

## ■ ROWID à partir de la V8

| ROWID | ENAME | JOB |
|---|---|---|
| AAAAvGAAFAAAACyAAA | SMITH | CLERK |
| I1 I2 I3 I4 | | |

- **I1:** N° de Segment (6 caractères)
- **I2** :N° de fichier(3 caractères)
- **I3** : N° de bloc(6 caractères)
- **I4**: N° de ligne(3 caractères)

# 3.3 Accès par ROWID

- **Un** ROWID **est le moyen le plus rapide d'accéder à un tuple ou ligne.**

  - ***Exemple***

    Select * From emp Where rowid = 'AAAAvGAAFAAAACyAAA';

  - ***Attention***

    Attention, l'écriture ci-dessus n'est pas portable.

# 3.4 Accès par index

## ■ Généralités

- Un index est une **structure optionnelle associée à une table ou un cluster** afin d'accélérer l'accès aux lignes de la table ou du cluster.

- Un **index peut être** (depuis Oracle 7) **crée implicitement** (colonnes clés primaires ou uniques) **ou explicitement** grâce au langage SQL. Les index sont logiquement et physiquement indépendants des données des tables associées.

- Un index **peut être unique ou non unique**. Il peut être **mono-colonne** ou **multi-colonnes**.

- Oracle supporte depuis la version 8 différents types d'index notamment : les Index de **type B-arbre** (normal , descendant, basés sur les fonctions, reverse), de **type Bitmap, tables organisés dans un index**

# 3.4.1 Accès par index B-arbre

## ■ Généralités

- Un index de type B-arbre est un arbre comprenant une **racine**, des **noeuds intermédiaires** et d**es feuilles**.

- La racine du B-arbre pointe vers les noeuds intermédiaires et les noeuds intermédiaires pointent vers les noeuds feuilles. Les noeuds feuilles pointant eux vers les lignes d'une table.

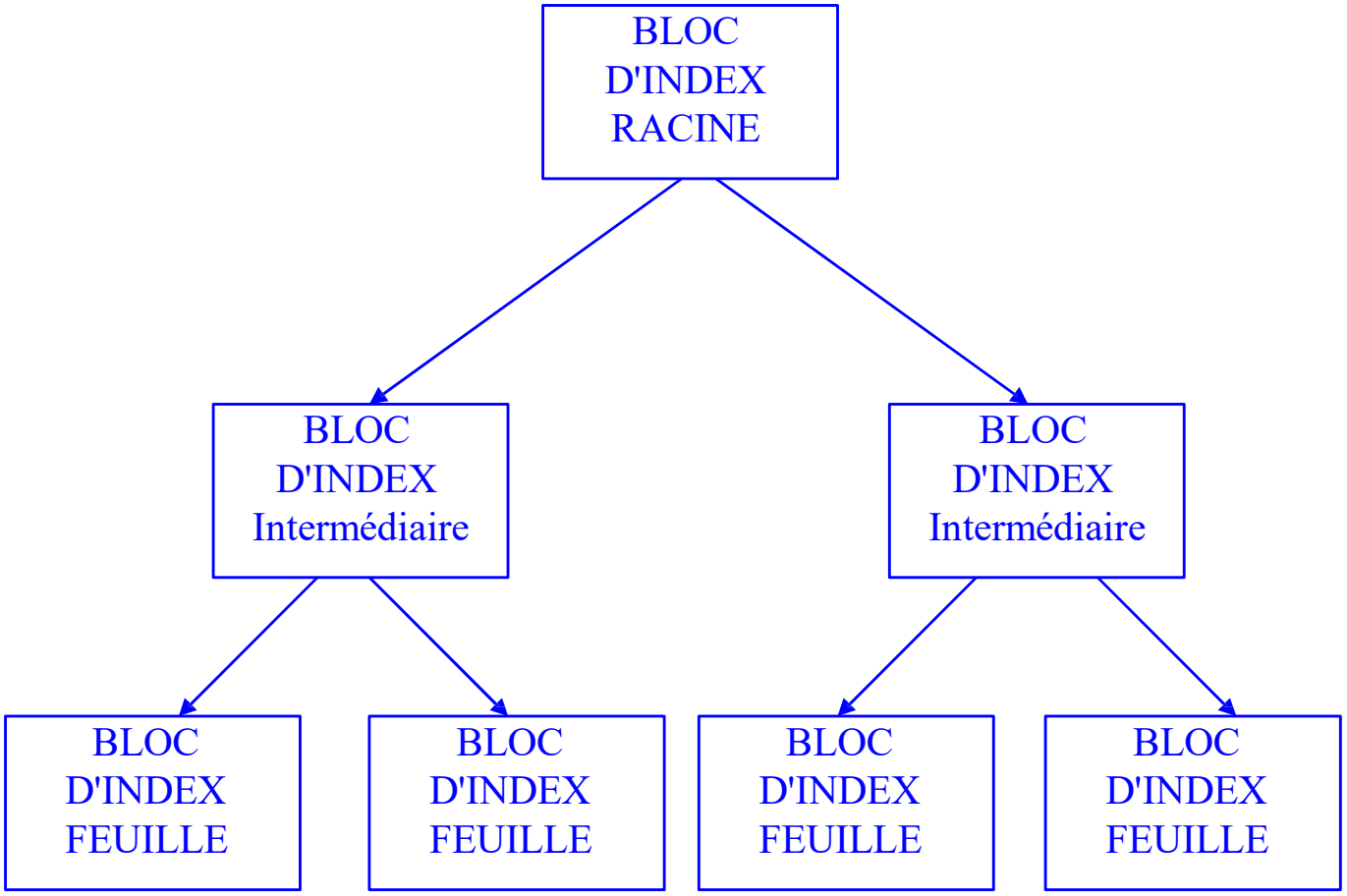
- On parle **de B-arbre ou arbre balancé**. Oracle utilise les arbres dit balancés afin que l'accès à chaque ligne d'une table prenne le même temps.

- Un index B-arbre garanti que **l'accès à chaque ligne d'une table prend le même temps** quelque soit le chemin choisit de la racine du B-arbre vers les feuilles

# 3.4.1 Accès par index B-arbre

## ■ Généralités

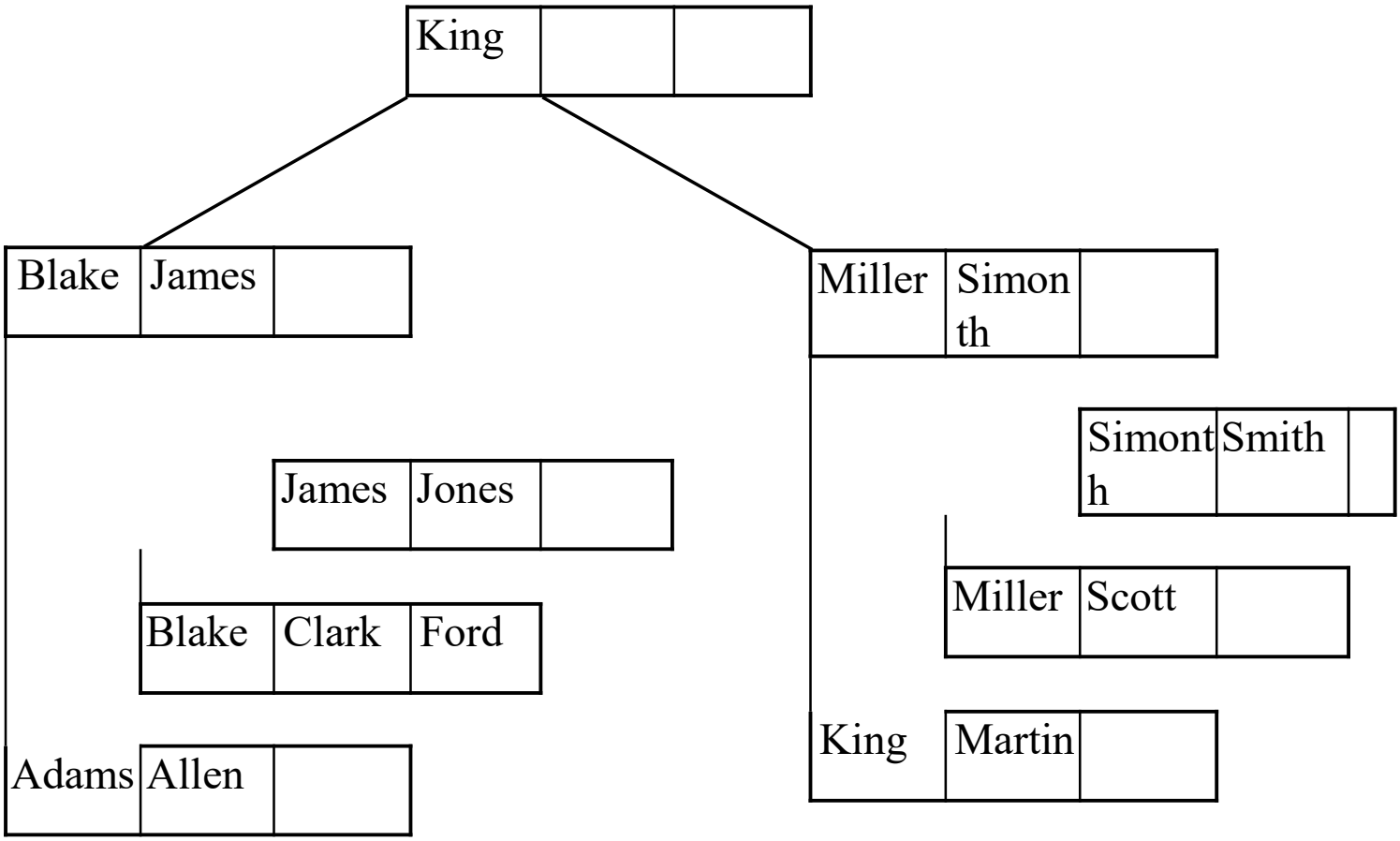
- **Structure d'un B-arbre**



# 3.4.1 Accès par index B-arbre

## ■ Généralités

- **Exemple**




Arbre de Hauteur = 2
A chaque feuille on trouve : la valeur clé et le ROWID (index unique) ou les la liste des ROWID (Index non unique). Par exemple Adams 'AAASsUAAEAAAZqvAAK'.

SELECT * FROM EMP WHERE ENAME='Adams';

# 3.4.1 Accès par index B-arbre

- Correlation entre table et index

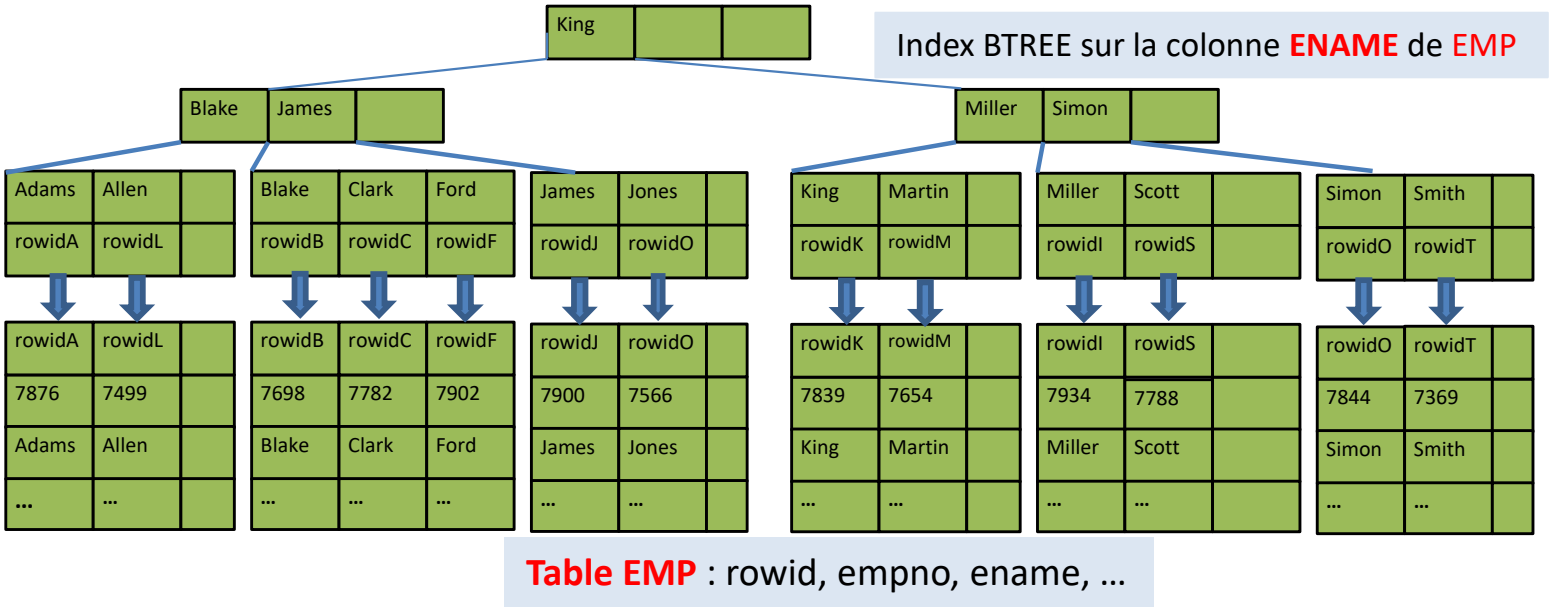



**Table EMP** : rowid, empno, ename, …

# 3.4.1 Accès par index B-arbre

## ■ Généralités

- **Exemple**

```
Plan d'exécution
----------------------------------------------------------
Plan hash value: 3977601012

-------------------------------------------------------------------------------------------
| Id  | Operation                   | Name         | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |              |     1 |    38 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP          |     1 |    38 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_EMP_ENAME |     1 |       |     1   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------
```

- **Accès à la table via les rowid**

select rowid, emp.* from emp order by ename;

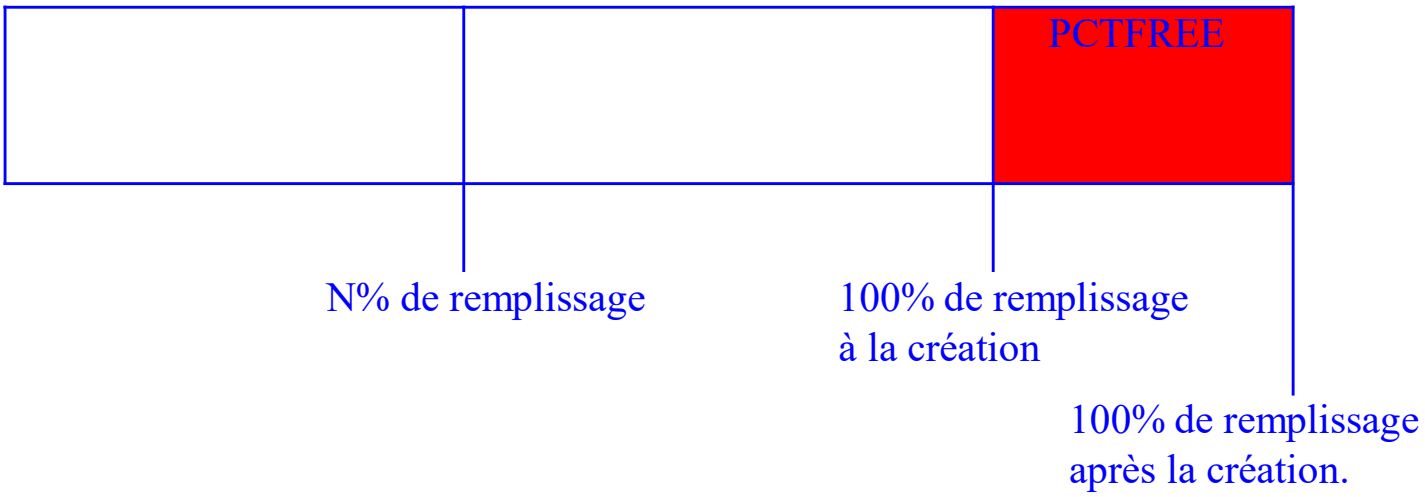
| ROWID | EMPNO | ENAME | JOB | MGR | HIREDATE | SAL | COMM | DEPTNO |
|---|---|---|---|---|---|---|---|---|
| AAASsUAAEAAAZqvAAK | 7876 | ADAMS | CLERK | 7788 | 12/01/83 | 1100 | | 20 |
| AAASsUAAEAAAZqvAAB | 7499 | ALLEN | SALESMAN | 7698 | 20/02/81 | 1600 | 300 | 30 |
| AAASsUAAEAAAZqvAAF | 7698 | BLAKE | MANAGER | 7839 | 01/05/81 | 2850 | | 30 |
| AAASsUAAEAAAZqvAAG | 7782 | CLARK | MANAGER | 7839 | 09/06/81 | 2450 | | 10 |
| AAASsUAAEAAAZqvAAM | 7902 | FORD | ANALYST | 7566 | 03/12/81 | 3000 | | 20 |
| AAASsUAAEAAAZqvAAL | 7900 | JAMES | CLERK | 7698 | 03/12/81 | 950 | | 30 |
| AAASsUAAEAAAZqvAAD | 7566 | JONES | MANAGER | 7839 | 02/04/81 | 2975 | | 20 |
| AAASsUAAEAAAZqvAAI | 7839 | KING | PRESIDENT | | 17/11/81 | 5000 | | 10 |
| AAASsUAAEAAAZqvAAE | 7654 | MARTIN | SALESMAN | 7698 | 28/10/81 | 1250 | 1400 | 30 |
| AAASsUAAEAAAZqvAAN | 7934 | MILLER | CLERK | 7782 | 23/01/82 | 1300 | | 10 |
| AAASsUAAEAAAZqvAAH | 7788 | SCOTT | ANALYST | 7566 | 09/12/82 | 3000 | | 20 |
| AAASsUAAEAAAZqvAAA | 7369 | SMITH | CLERK | 7902 | 17/12/80 | 800 | | 20 |
| AAASsUAAEAAAZqvAAJ | 7844 | TURNER | SALESMAN | 7698 | 08/09/81 | 1500 | 0 | 30 |
| AAASsUAAEAAAZqvAAC | 7521 | WARD | SALESMAN | 7698 | 22/02/81 | 1250 | 500 | 30 |

14 ligne(s) sélectionnée(s).

# 3.4.1 Accès par index B-arbre

## Structure Interne d'un Index B-arbre

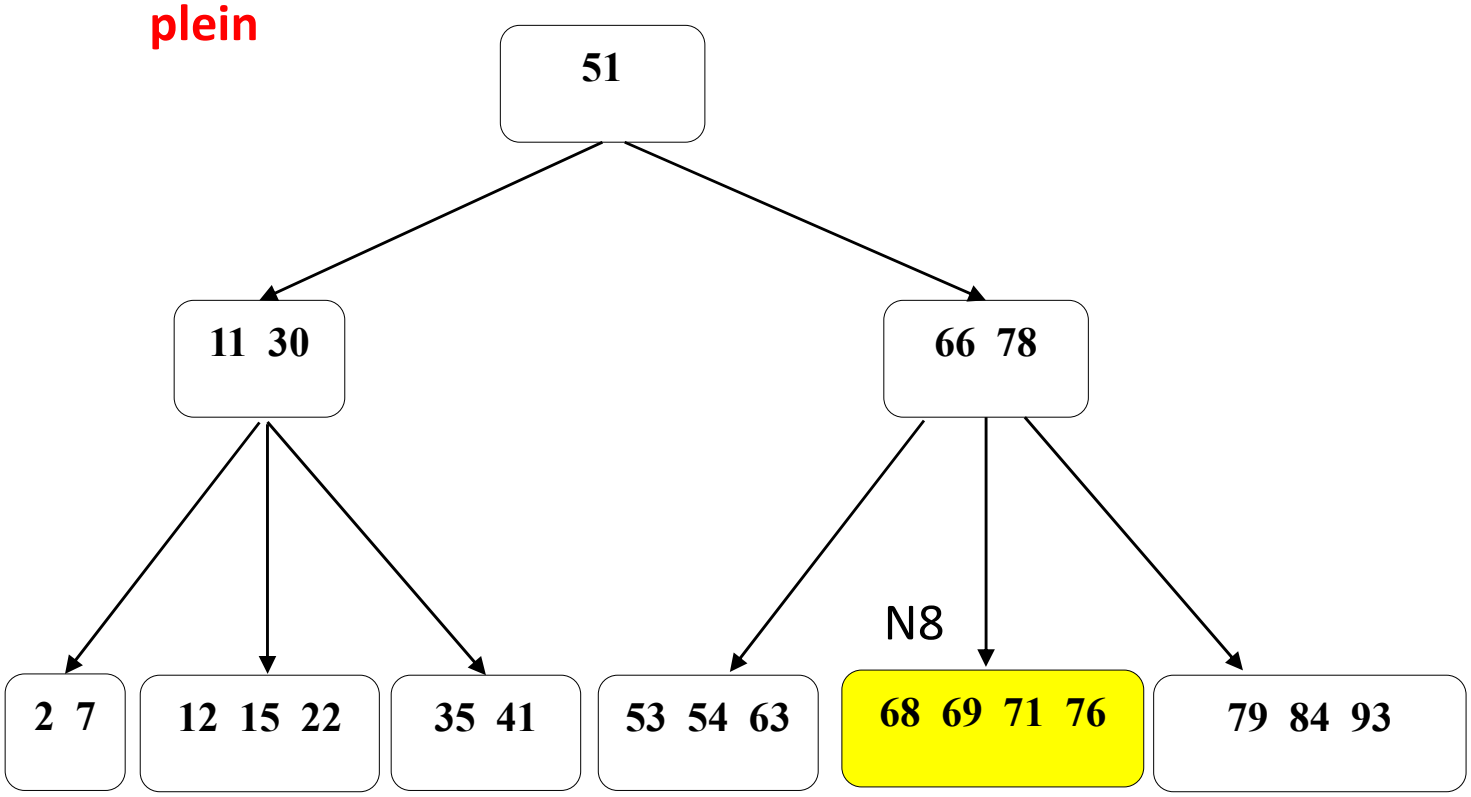
- Condition requis par un B-arbre Oracle. Un arbre de recherche est B*arbre Oracle d'Ordre **m** si :
  - Chaque bloc est rempli au plus à 100%
  - Chaque bloc, sauf la racine, a au moins un taux de remplissage de **2m**
  - Chaque nœud contient k clés triées avec :
    - **m <=k<=2m** (nœud non-racine )
    - et **1 <=k <=2m** (nœud racine)
    - Chaque bloc noeud a **k+1** successeurs, ou **k** est le nombre de clés dans la page
  - Tous les blocs feuilles sont au même niveau.



# 3.4.1 Accès par index B-arbre

## ▪ Ajout d'un élément dans un B-Arbre

Arbre d'ordre m ou m=2. Exemple, insertion de **75 dans un nœud plein**




Rappel : ici nombre de clés par nœud<=4, 75 sera ajouté à N8

**Méthode**

**Eclatement du nœud N8 en deux** :

Les (deux) plus petites clés restent dans le nœud

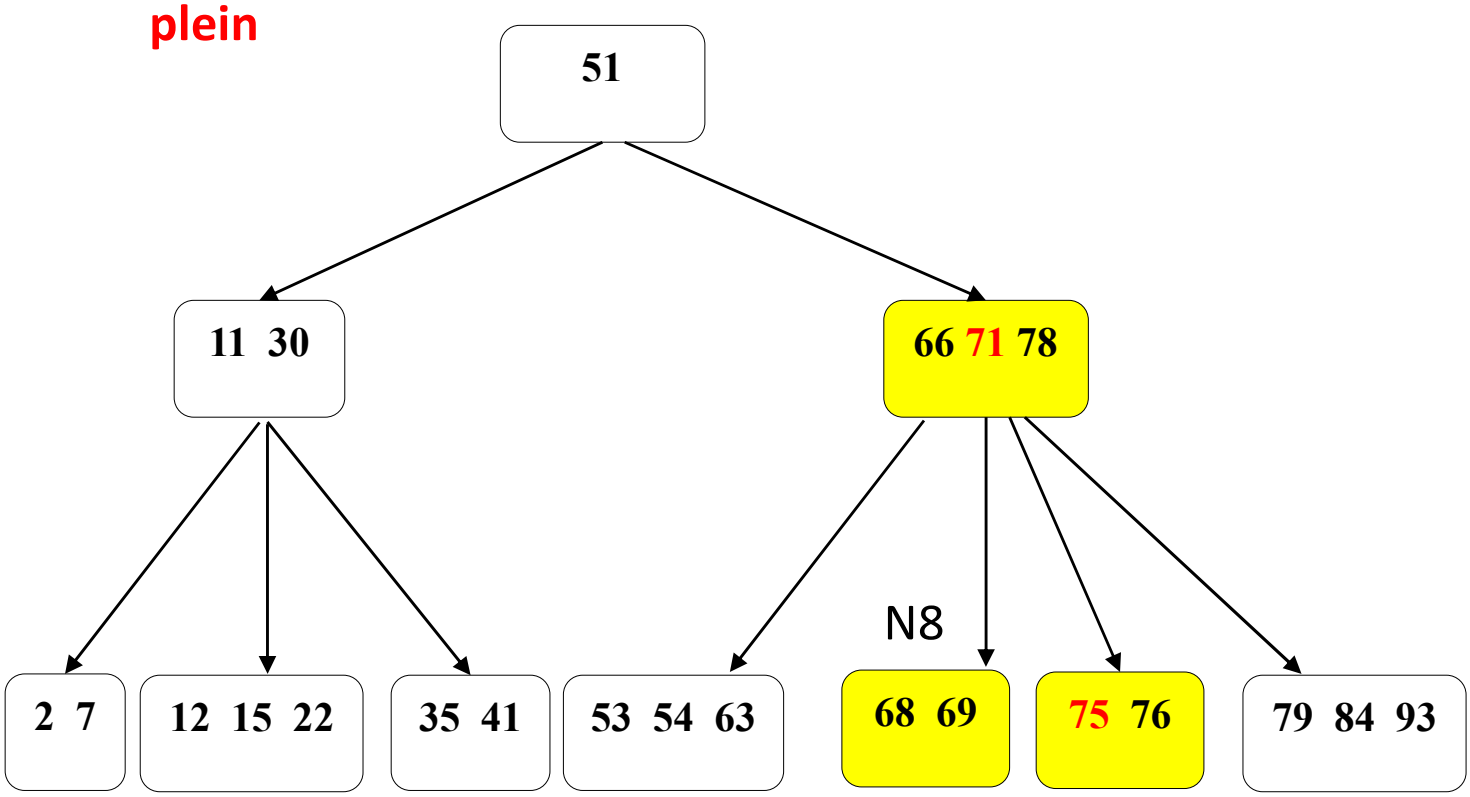
Les (deux) plus grandes clés sont insérées dans un nouveau nœud

**Remontée de la clé médiane dans le nœud père** (e.g. ici 71)

# 3.4.1 Accès par index B-arbre

## ▪ Ajout d'un élément dans un B-Arbre

Arbre d'ordre m ou m=2. Exemple, insertion de **75 dans un nœud plein**




Rappel : ici nombre de clés par nœud<=4, 75 sera ajouté à N8

**Méthode**

**Eclatement du nœud en deux** :

Les (deux) plus petites clés restent dans le nœud

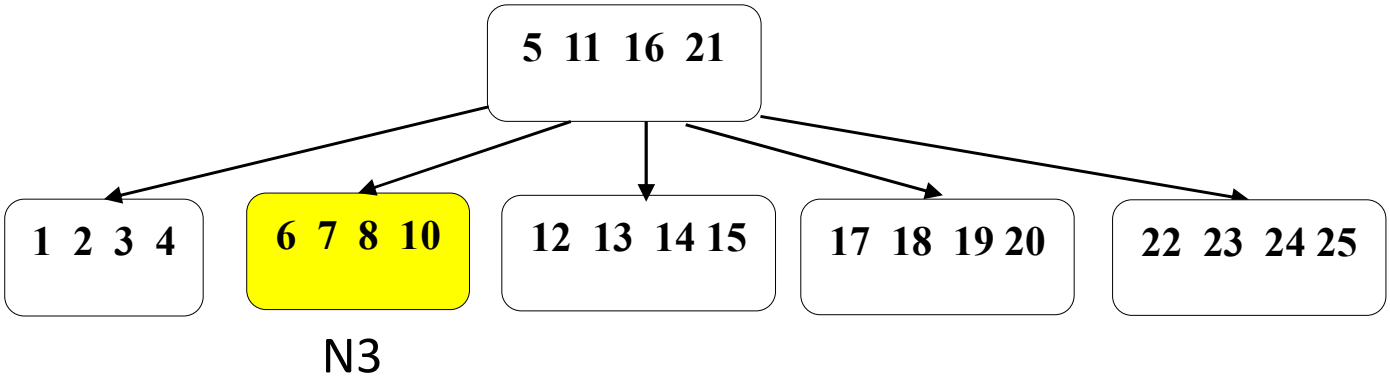
Les (deux) plus grandes clés sont insérées dans un nouveau nœud

**Remontée de la clé médiane dans le nœud père** (e.g. ici 71)

# 3.4.1 Accès par index B-arbre

## ▪ Ajout d'un élément dans un B-Arbre

Exemple, insertion de **9 dans un nœud plein avec éclatement récursif jusqu'à la racine avec augmentation de la hauteur de l'arbre**




**Insertion de la clé 9**

Eclatement du nœud N3 + remonté de la clé 8 au nœud père

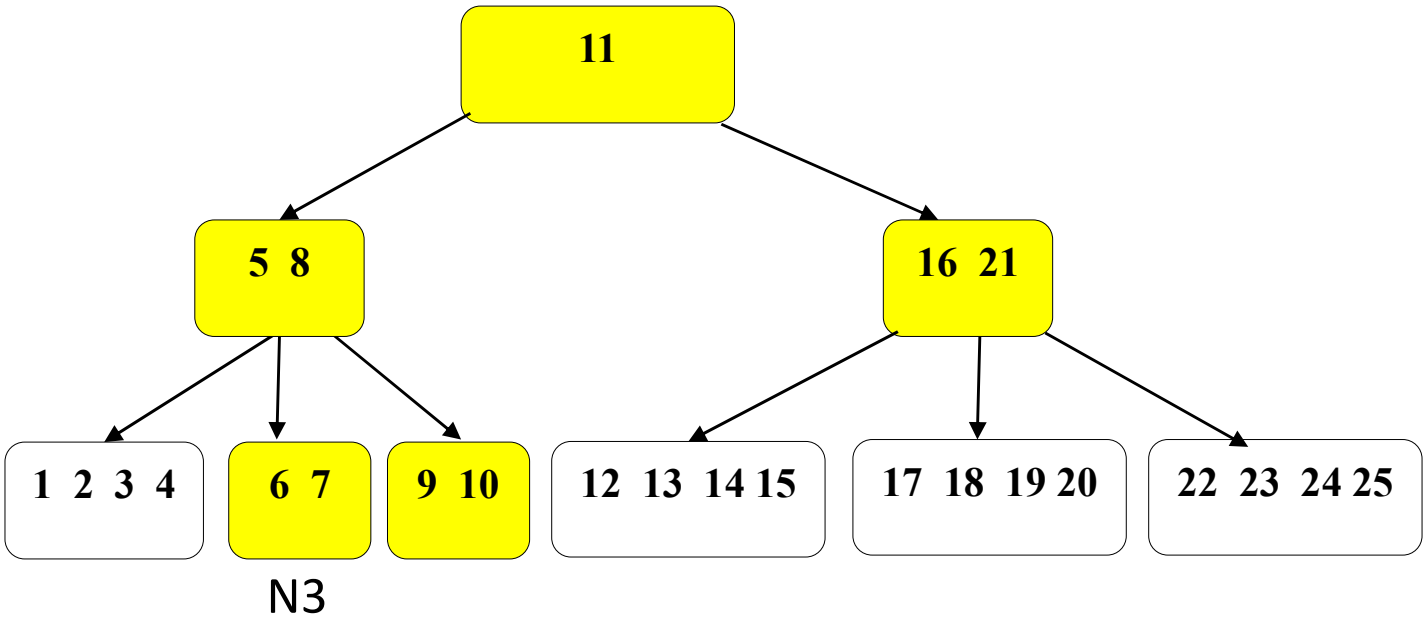
Remonté de la clé 8 au nœud père

Eclatement du nœud père (racine) + création d'une racine ( ce sera la clé médian 11 dans la racine)

# 3.4.1 Accès par index B-arbre

## ▪ Ajout d'un élément dans un B-Arbre

Exemple, insertion de **9 dans un nœud plein avec éclatement récursif jusqu'à la racine avec augmentation de la hauteur de l'arbre**




**Insertion de la clé 9**

Eclatement du nœud N3 + remonté de la clé 8 au nœud père

Remonté de la clé 8 au nœud père

Eclatement du nœud père (racine) + création d'une racine ( ce sera la clé médian 11 dans la racine)

**Augmentation d'une unité de la hauteur de l'arbre**

# 3.4.1 Accès par index B-arbre

## Structure Interne d'un Index B-arbre

- **Avantage d'un B-arbre**
  - Toutes les feuilles de l'arbre ont la même profondeur
  - Le balancement est automatique
  - s'adapte à une variété de requêtes
  - Intéressant pour petites et grandes tables
- **Inconvénient d'un B-arbre**
  - consomme de la place,
  - pénalise la modification et l'insertion

# 3.4.1 Accès par index B-arbre

## ■ Création d'un index

- Pendant qu'une table est associée lors de sa création à un segment de données, un index lui est associé à un segment d'index.

- **Syntaxe de création d'un Index**

**CREATE [UNIQUE] INDEX** nom_index **ON** nom_table
(colonne1**[ASC | DESC]** [, colonne2, ...])
[**INITRANS** valeur] [**MAXTRANS** valeur]
**[TABLESPACE** nom_tablespace]
[**STORAGE** paramètres_de_stockage]
[**PCTFREE** [=] %]
[**NOSORT**] --ASC par défaut
[**PARTITION** clause_de_partionnement]

# 3.4.1 Accès par index B-arbre

## ■ Création d'un index

| Mot clé / paramètre | Description |
|---|---|
| ASC,DESC | Index ascendant ou descendant |
| Initrans | Nombre de transactions concurrentes dont les informations sont directement notées dans l'entête du bloc. |
| Maxtrans | Nombre de transaction concurrentes maximum sur un, bloc. |
| Tablespace | Espace de stockage dans lequel le segement de l'index sera créé. |
| Storage | Définition de la taille du segment |
| Partition | Clause de partitionnement d'un index. |

# 3.4.1 Accès par index B-arbre

## ■ Création d'un index

- **Exemple**

création de l'index idx_emp_ename sur la colonne ename de la table emp.

```
CREATE INDEX idx_emp_ename ON emp (ename)
TABLESPACE USERS
STORAGE(
INITIAL 50K
NEXT 50K
MINEXTENTS 1
MAXEXTENTS 25
PCTINCREASE 0
)
PCTFREE 30;
```

# 3.4.1 Accès par index B-arbre

## ■ Utilisation de l'index

- **Exemple**

Select * from Emp where ename ='KING';

```
---------------------------------------------------------------------------------------
| Id  | Operation                   | Name        | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |             |     1 |    37 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP         |     1 |    37 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_EMP_ENAME |     1 |       |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------------------
```

# 3.4.1 Accès par index B-arbre

- **Index full scan, au lieu d'un balayage séquentiel : quand Oracle l'utilise**

  - Si la ou les colonnes derrières Select sont dans l'index
  - Si la ou les colonnes indexées sont NOT NULL
  - S'il s'agit de l'optimiseur de statistique

```
Alter table EMP
modify ename varchar2(12) not null;
Alter session set optimizer_mode=all_rows;
Select ename from Emp;
```

```
---------------------------------------------------------------------------
| Id  | Operation        | Name        | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT |             |    14 |    84 |     1   (0)| 00:00:01 |
|   1 |  INDEX FULL SCAN | IDX_EMP_ENAME |  14 |    84 |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------
```

# 3.4.1 Accès par index B-arbre

## ■ Modification d'un index

- **Syntaxe**

**ALTER INDEX** nom_index
[REBUILD {
**[TABLESPACE** nom_tablespace]
[**STORAGE** paramètres_de_stockage]
**PCTFREE** [=] %]
**[LOGGING] --génération ou non d'entrées Redolog**
**[NOLOGGING]**
**[REVERSE]**
**[NOREVERSE]}]**
**[RENAME TO nouveau_nom_index]**
**[MODIFY DEFAULT ATTRIBUTES {**
**[TABLESPACE** nom_tablespace]
[**STORAGE** paramètres_de_stockage]
**PCTFREE** [=] %]}]
**[MODIFY PARTITION** nom_partition]
**[RENAME PARTITION** nom_partition]
**[DROP PARTITION** clause_de_partionnement]
**[SPLIT PARTITION** partion_split_clause]
**[REBUILT PARTITION** partition_name]

# 3.4.1 Accès par index B-arbre

## ■ Modification d'un index

- **Exemple**

```
Sql>ALTER INDEX idx_ename REBUILD
TABLESPACE ts_cour_temp;
```

# 3.4.1 Accès par index B-arbre

## ■ Vues du dictionnaire

- ALL_INDEXES, ALL_IND_COLUMNS
- ALL_IND_PARTITIONS,

- USER_INDEXES, USER_IND_COLUMNS
- USER_IND_PARTITIONS, USER_PART_INDEXES

- DBA_INDEXES, DBA_IND_COLUMNS
- DBA_PART_INDEXES, DBA_IND_PARTITIONS

## ■ Consultation des vues du dictionnaire

**sql > select * from all_ind_columns**

**where index_name ='IDX_ENAME';**

```
INDEX_OWNER          : SCOTT
INDEX_NAME           : IDX_ENAME
TABLE_OWNER          : SCOTT
TABLE_NAME           : EMP
COLUMN_NAME          : ENAME
COLUMN_POSITION      :  1
COLUMN_LENGHT        : 10
```

# 3.4.1 Accès par index B-arbre

- **Consultation des vues du dictionnaire**

```
sql> select * from all_indexes
where index_name = 'IDX_ENAME';

OWNER                          : SCOTT
INDEX_NAME                     : IDX_ENAME
TABLE_OWNER                    : SCOTT
TABLE_NAME                     : EMP
TABLE_TYPE                     : TABLE
UNIQUENES                      : NONUNIQUE
TABLESPACE_NAME                : USERS
MAX_TRANS                      : 255
INITIAL_EXTENT                 : 51200
NEXT_EXTENT                    : 51200
MIN_EXTENTS                    : 1
MAX_EXTENTS                    : 25
PCT_INCREASE                   : 0
PCT_FREE                       : 30
STATUS                         : Valid
```

# 3.4.1 Accès par index B-arbre

## Estimation de la taille d'un Index

- Le package DBMS_SPACE possède un certains nombre de fonctions pour calculer la taille des objets
  - CREATE_TABLE_COST : permet de calculer la taille d'une table
  - CREATE_INDEX_COST : permet de calculer la taille d'un index
  - **Exemple**

```
set serveroutput on
Declare
    bytes_utilises number;
    bytes_alloues number;
Begin
    dbms_space.create_index_cost(
        ddl=>'CREATE index idx_emp_sal on emp(sal)',
        used_bytes=>bytes_utilises ,
        alloc_bytes=>bytes_alloues
    );
    dbms_output.put_line('bytes utilises='||bytes_utilises);
    dbms_output.put_line('bytes alloues='||bytes_alloues);
End;
/
```

# 3.4.1 Accès par index B-arbre

## ■ Estimation de la taille d'un Index

- Paramètres de la fonction create_index_cost
  - **Ddl** : commande de creation de l'index
  - **Used_bytes** : nombre d'octets utilisés par l'index
  - **Alloc_bytes** : nombre d'octets alloués pour les extensions de l'index
- Notes
  - La table doit avoir été **créée, peuplée** de lignes et les **statistiques sur la table calculées**

# 3.4.2 Index Bitmap

## ■ Accès par index Bitmap

- Quand utiliser un index Bitmap
- Exemple de colonnes candidates
- Exemple de requêtes
- Création d'un index Bitmap
- Paramètres d'initialisations et valeurs par défaut
- Index bitmap et les valeurs nulles
- Limites des index bitmap
- Join bitmap index
- Comparaison des index Btree et Bitmap

# 3.4.2 Index Bitmap

## ■ Quand utiliser un index Bitmap

- La clause WHERE a plusieurs PREDICAT sur des colonnes de CARDINALITE faible ou moyenne (peut de clés distinctes)
- Chacun des prédicats sur les colonnes sus-mentionnées rend un nombre important de lignes
- Des index Bitmap existent sur une ou plusieurs colonnes sus-mentionnées
- Pour une table de volumétrie forte (PLUS LA TABLE EST VOLUMINEUSE MIEUX ILS SONT)
- Permettent d'ECONOMISER L'espace DISQUE
- Pratique pour les APPLICATIONS de type DataWareHouse (DWH)

# 3.4.2 Index Bitmap

## ■ Exemple de colonnes candidates

| Custid# | Etat_marital | Region | Genre |
|---|---|---|---|
| 101 | célibataire | est | M |
| 102 | marié | centre | F |
| 103 | marié | ouest | F |
| 104 | divorcé | ouest | M |
| 105 | célibataire | centre | F |
| 106 | marié | centre | F |

– NOTE : Les colonnes Etat_marital, Région et Genre sont de cardinalité faible, donc candidats à index bitmap

- Index Bitmap sur la colonne Region

| Custid# | Region=est | Region=centre | region=ouest |
|---|---|---|---|
| 101 | 1 | 0 | 0 |
| 102 | 0 | 1 | 0 |
| 103 | 0 | 0 | 1 |
| 104 | 0 | 0 | 1 |
| 105 | 0 | 1 | 0 |
| 106 | 0 | 1 | 0 |

# 3.4.2 Index Bitmap

## ■ Exemple de requête

- Compter le nombre de clients mariés habitant la région Centre ou Ouest.

```
SELECT COUNT(*) FROM CUSTOMER
WHERE etat_marital = ‘marie'
AND REGION IN ('centre', 'ouest');
```

| Status =marie | | region=centre | | region=ouest | | Résultat |
|---|---|---|---|---|---|---|
| 0 | | 0 | | 0 | | 0 |
| 1 | | 1 | | 0 | | 1 |
| 1 | AND ( | 0 | OR | 1 ) | = | 1 |
| 0 | | 0 | | 1 | | 0 |
| 0 | | 1 | | 0 | | 0 |
| 1 | | 1 | | 0 | | 1 |

– Réponse : 3

# 3.4.2 Index Bitmap

## Création d'un index Bitmap

- **Syntaxe général**

```
CREATE BITMAP INDEX nom_index
ON nom_table
  (colonne [asc | desc] [, colonne [asc | desc]]…)
  [join_clause_for_join_bitmap_index]
  [TABLESPACE nomTablespace]
  [PCTFREE integer]
  [PCTUSED integer]
  [INITRANS integer]
  [MAXTRANS integer]
  [Storage clause]
  [LOGGING | NOLOGGING]
  [NOSORT]
```

## Exemple

```
SQL>CREATE BITMAP INDEX
    bitmap_idx_emp_comm
    ON emp(comm)
    TABLESPACE users;
```

# 3.4.2 Index Bitmap

## ■ Index Bitmap avec jointure

- Dans un contexte de DWH permet d'accélérer les jointures entre une table de fait et les dimensions. LA JOINTURE EST **pré-faite**

- La **table apparaissant derrière la la clause ON est la tables des faits**

- Les **colonnes apparaissant entre parenthèses sont les clés primaires des tables dimensions**. Elles doivent aussi être clés étrangères dans la table de fait

- **Clause de jointure pour un Index bitmap avec jointure**
  CREATE BITMAP INDEX … ON **nom_table**
  **(colonne [asc | desc] [, colonne [asc | desc]]…)**
  join_clause_for_join_bitmap_index ::=
  **From table_name, [table_name,] […]**
  **WHERE join_condition;**

# 3.4.2 Index Bitmap

## ■ Index Bitmap avec jointure

- **Exemple**
  - Supposons qu'il existe la table de fait Vente et les tables de dimension : produit, région et période
  - 

```
SQL>CREATE BITMAP INDEX
    bitmap_idx_ventes_joins
    ON Ventes(produit#, region#, periode#)
    TABLESPACE users
    FROM vente v, produit pr, region r, periode pe
    where v.produit#=pr.produit# and v.region#=r.region#
    and v.periode#=pe.periode#
```

- **Contraintes sur les bitmap Index avec jointure**
  - Pas d'auto-jointure
  - Pas d'Index basé sur les fonctions
  - Les clés primaires des dimensions doivent y apparaître entièement
  - Les tables doivent faire partie de la jointure
  - Ne peut être un Index local (table partionnée)

# 3.4.2 Index Bitmap

## ■ Index Bitmap avec jointure

- **Exemple d'utilisation**

**Ecrire une requête qui renvoi le chiffre d'affaire par produit, region et période**

```
Select v.produit#, v.region#, v.periode#, sum(montant) CA
FROM vente v, produit pr, region r, periode pe
where v.produit#=pr.produit# and v.region#=r.region# and
    v.periode#=pe.periode#
Group by v.produit#, v.region#, v.periode#;
```

Il n'y aura pas jointure car elle est déjà faire dans l'index

# 3.4.2 Index Bitmap

- **Paramètres d'initialisations et valeur par défaut**

  - create_bitmap_area_size = 8388608
    // espace utile pour créer les Bitmap

  - bitmap_merge_area_size=1048576
    // espace utile pour fusionner les Bitmap

# 3.4.2 Index Bitmap

## Index bitmap et les valeurs nulles

- Contrairement aux Index B-tree, les valeursNulles sont stockées dans l'index

```
create bitmap index idx_bitmap_emp_com on
emp(comm);

SELECT count(*)
FROM emp  WHERE comm IS NULL;
```

- La requête précédente exploite l'index bitmap s'il existe

## Limites des index Bitmap

- Le mélange dans une clause WHERE d'un index B-tree et d'un index Bitmap peut être couteux

- Adaptés pour des tables à forte volumétrie et aux colonnes à cardinalités faibles

# 3.4.2 Index Bitmap

## ■ Comparaison B-TREE et BITMAP

| CRITERE | B-TREE | BITMAP |
|---|---|---|
| Cardinalité des colonnes | forte | faible |
| Coût des MAJ | faible | fort |
| Requête avec OR | inefficace | efficace |
| Verrouillage | ligne | segment |
| Consomation d'espace | fort | Faible |
| Domaine d'application | OLTP | Décisionnel |

# 3.5 Table organisée dans index

## Notion

- Une table organisée dans un index (IOT) est une table dont l'ensemble des lignes est stockée dans les feuilles de l'index
- L'organisation est faite en fonction de la colonne clé.

**TABLE et INDEX SEPARES**

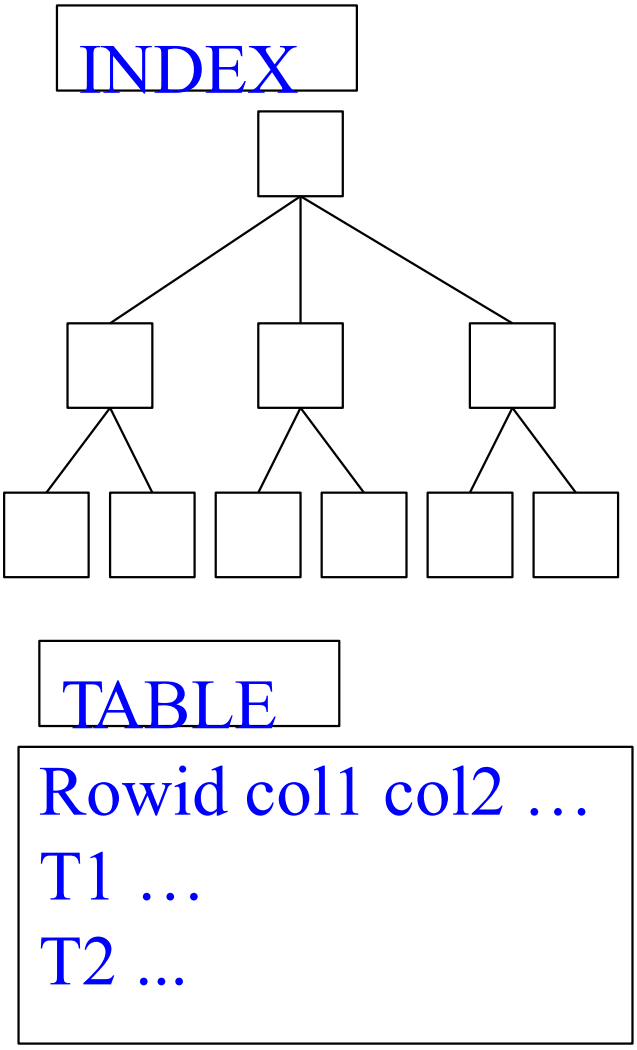
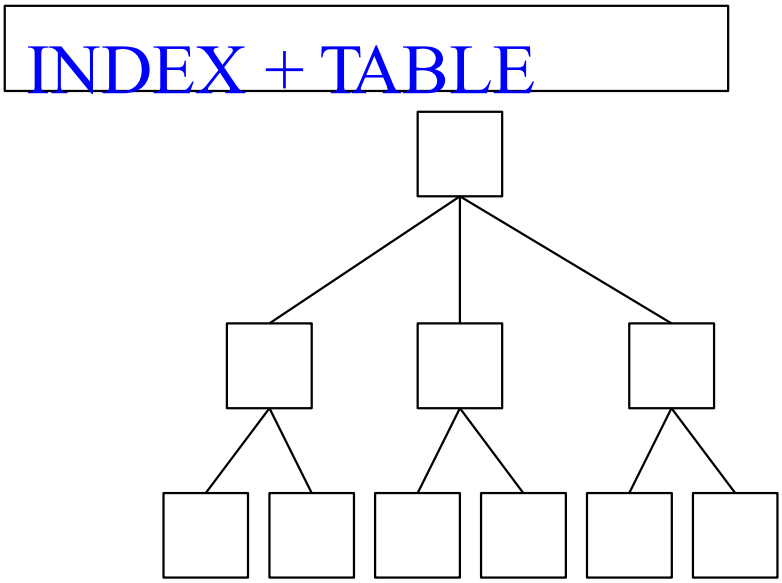



**TABLE et INDEX FUSIONNES**



# 3.5 Table organisée dans index

## Avantages

- Pas de duplication de la valeur de la clé
- L'accès aux données via la clé est rapide
- Adapté pour les applications de type DWH
- Adapté pour les applications de recherche par le contenu (recherche dans un document)
- Adapté pour les applications spatiales

# 3.5 Table organisée dans index

## Inconvénients

- Impossible d'indexer d'autres colonnes car la table n'a pas d'identifiants de lignes (ROWID) : jusqu'à Oracle 8.0 uniquement
- Ne peuvent être mises en CLUSTER
- Ne peuvent contenir de colonnes LONG (par contre les BLOBs sont autorisés)
- Ne peuvent être ni partitionnées ni répliquées (V8.0)

# 3.5 Table organisée dans index

- **Syntaxe de création d'une table Organisée dans un index**

```
CREATE TABLE nomTable(
SpéficationsDesColonnesETdesContraintes
)
ORGANIZATION INDEX
[segment_attributes_clause]
[PCTTHRESHOLD entier]
[[INCLUDING nomColonne]
    OVERFLOW [segment_attributes_clause]];

segment_attributes_clause ::=
        [storage(initial entier next entier minextents ...)]
        [TABLESPACE nomTableSpace]
        [LOGGING | NOLOGGING]
```

# 3.5 Table organisée dans index

- **Syntaxe de création d'une table Organisée dans un index**

| Mot clé ou paramètre | Description |
|---|---|
| Organization | Mot clé indiquant qu'une table est organisée dans un index |
| Pctthreshold *entier* | Spécifie la taille de l'espace réservé à une ligne dans le segment IOT. En cas de dépassement le reste de la ligne sera écrit dans le segment de débordement. Ce paramètre vaut de 0 à 50 |
| Including *nomColonne* | le reste de la ligne à rediriger vers le segment de débordement commence avec la colonne *nomColonne* |
| Overflow | les lignes dépassant **Pctthreshold** sont rejetées dans le segment de débordement spécifié ici |

# 3.5 Table organisée dans index

- **Exemple**

```
CREATE TABLE sales(
    SALESPERSON_ID      NUMBER(4),
    CUSTOMER            VARCHAR2(45),
    PRODUCT             VARCHAR2(30),
    Dachat              DATE,
    AMOUNT              NUMBER,
    PRIMARY KEY(salesperson_id, customer,
                product, dachat))
    ORGANIZATION INDEX
    TABLESPACE users;
```

# 3.5 Table organisée dans index

## Gestion des débordements

- des **lignes trop longues** dans le segment IOT entrainent la baisse des performances d'accès

- Afin de **limiter la taille des lignes** dans le segment de type IOT, il est possible d'orienter une partie d'une ligne vers un segment de débordement.

- Les clauses **OVERFLOW, PCTTHRESHOLD et INCLUDING** permettent de gérer les débordements.

  - **OVERFLOW** : indique le tablespace de rejet

  - **PCTTHRESHOLD val :** permet de spécifier le pourcentage d'espace réservé dans le segment IOT pour un index organisé dans une table. Ce pourcentage doit être suffisant pour stocker au moins la clé primaire. Toutes les colonnes à partir de celle provoquant le dépassement sont stockées dans le segment Overflow. Val = 1..50. Valeur par défaut 50

  - **INCLUDING nomcol :** les colonnes venant après **nomcol** seront stockées dans le segment overflow. Si cette spécification entraine le dépassement de PCTTHRESHOLD, ce paramètre prime.

# 3.5 Table organisée dans index

- **Exemple**

```
CREATE TABLE sales3(
    SALESPERSON_ID          NUMBER(4),
    CUSTOMER                VARCHAR2(45),
    PRODUCT                 VARCHAR2(30),
    Dachat                  DATE,
    AMOUNT                  NUMBER,
    PRIMARY KEY(salesperson_id, customer, dachat))
    ORGANIZATION INDEX
    TABLESPACE users
    INCLUDING product  PCTTHRESHOLD 20
    OVERFLOW TABLESPACE tools;
```

# 3.5 Table organisée dans index

## ■ Consultation du dictionnaire Oracle

- Les informations sur les tables organisées dans l'index sont disponibles dans les vues : *_tables, *_segments, *_indexes

Sql>SELECT table_name, tablespace_name, iot_name, iot_type FROM user_tables ORDER BY table_name;

| Table_name | Tablespace_name | IOT_name | IOT_Type |
|---|---|---|---|
| SALES3 | | | IOT |
| SYS_IOT_OVER_2561 | TOOLS | SALES3 | IOT_overflow |

**SQL> SELECT index_name, index_type, tablespace_name FROM user_indexes ORDER BY index_name;**

| INDEX_NAME | INDEX_TYPE | TABLESPACE_NAME |
|---|---|---|
| SYS_IOT_TOP_2561 | IOT-TOP | USERS |

# 3.5 Table organisée dans index

## ■ Consultation du dictionnaire Oracle

```
Sql> SELECT segment_name, tablespace_name,
     segment_type
FROM user_segments ORDER BY segment_name;
```

| SEGMENT_NAME | TABLESPACE_NAME | SEGMENT_TYPE |
|---|---|---|
| SYS_IOT_OVER_2561 | TOOLS | TABLE |
| SYS_IOT_TOP_2561 | USERS | INDEX |

## ■ NOTES :

- SALES3 table organisée dans un index :
  - ts_cour_users : tablespace de sales3
  - SYS_IOT_TOP_2561 : nom du segment d'index. Le type d'index est IOT-TOP.
- SYS_IOT_OVER_2561 segment de débordement:
  - TOOLS : tablespace de débordement
  - Il est de type TABLE.
  - Son IOT-TYPE est IOT-OVERFLOW.

# 3.5 Table organisée dans index

## ■ Comparaison avec des tables normales

| Critères | Table normale | IOT |
|---|---|---|
| Identification des lignes | rowid | clé primaire |
| Présence de ROWID ! | oui | non |
| Nombres d'index | 0 à N | 1 |
| Accès séquentiel | lignes non ordonnées | ordonnées |
| Consommation d'espace | plus | moins |
| Type de segment | table | index ( table) |
| Réplication/ Partitionnement | oui | non(V8.0) |

# 3.6 Accès par Cluster indexé

## ■ Généralités

- Un cluster (concept introduit avec System*R) est un groupe de tables partageant un même bloc de données. Ces tables ont en commun certaines colonnes et sont souvent accédées ensembles.

- ***Objectifs des clusters***

  – Améliorer les performances des requêtes

  – localisation physique des données de plusieurs tables dans un même bloc.

# 3.6 Accès par Cluster indexé

## ■ Généralités

- un cluster permet de stocker dans un même bloc les données de 2 ou plusieurs tables
- Favorise certains chemins d'accès (jointure physique)
- un cluster peut être indexé ou hashé

Clé cluster Deptno Table Dept et table EMP

| Deptno | dname | loc |
|---|---|---|
| 10 | Sales | Boston |

| Empno | ename | deptno |
|---|---|---|
| 7067 | Jones | 10 |
| 7056 | James | 10 |

| Deptno | dname | loc |
|---|---|---|
| 20 | Info | St Louis |

| Empno | ename | deptno |
|---|---|---|
| 7876 | Adams | 20 |
| 7788 | Scott | 20 |
| 7369 | Smith | 20 |

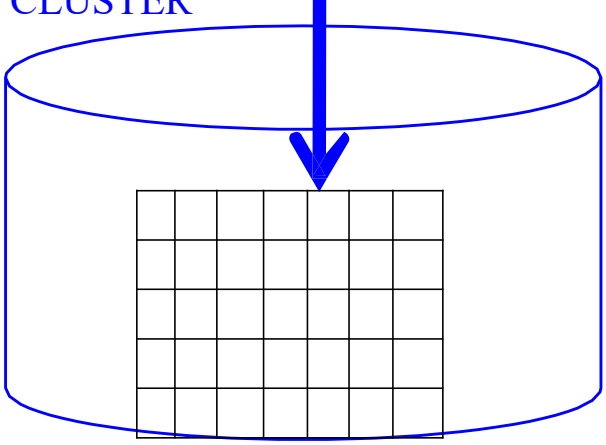
**Table DEPT**

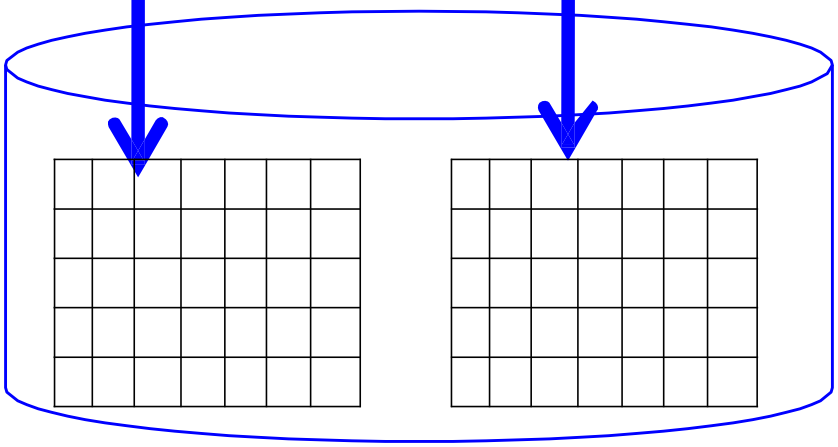
| Deptno | dname | loc |
|---|---|---|
| 10 | Sales | Boston |
| 20 | Info | St Louis |

**Table EMP**

| Empno | ename | deptno |
|---|---|---|
| 7067 | Jones | 10 |
| 7056 | James | 10 |
| 7876 | Adams | 20 |
| 7788 | Scott | 20 |
| 7369 | Smith | 20 |

CLUSTER





# 3.6 Accès par Cluster indexé

## ■ Création d'un cluster

- **Syntaxe**

**CREATE CLUSTER** [schéma.]cluster_name
(column datatype [,column datatype] ...)
[**PCTUSED** integer] [ **PCTFREE** integer]
[**SIZE** integer][K | M]
[**INITRANS** integer] [**MAXTRANS** integer]
[**TABLESPACE** tablespace_name]
[**STORAGE** storage_param]
[**INDEX**]
[[**HAS IS** colonne] **HASHKEYS** integer]

# 3.6 Accès par Cluster indexé

## ■ Exemple de manipulation de Cluster indexé

```
-- Création du cluster
CREATE CLUSTER CLU_IEMP4_IDEPT4 (
DEPTNO NUMBER(2)
);

-- Création de l'index sur la clé de cluster
CREATE INDEX  IDX_CLU_IEMP4_IDEPT4
ON CLUSTER CLU_IEMP4_IDEPT4;

-- Création de la table IDEPT4 dans le cluster
CREATE TABLE IDEPT4(
DEPTNO1 NUMBER(2) CONSTRAINT  pk_Idept4
   PRIMARY KEY,
DNAME VARCHAR2(14),
LOC VARCHAR2(13)
 ) CLUSTER CLU_IEMP4_IDEPT4 (DEPTNO1);
```

# 3.6 Accès par Cluster indexé

## ■ Exemple de manipulation de Cluster indexé

```
-- Création de la table EMP4 dans le cluster
CREATE TABLE IEMP4(
EMPNO NUMBER(4) constraint pk_emp4 primary key,
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
MGR NUMBER(4),
HIREDATE DATE,
SAL NUMBER(7,2),
COMM NUMBER(7,2),
DEPTNO2 NUMBER(2) CONSTRAINT
   fk_emp4_deptno REFERENCES IDEPT4 (DEPTNO1)
)CLUSTER CLU_IEMP4_IDEPT4 (DEPTNO2) ;


-- Insertion de lignes à partir de la table DEPT
INSERT INTO IDEPT4 SELECT * FROM DEPT;

-- Insertion de lignes à partir de la table EMP
INSERT INTO IEMP4 SELECT * FROM EMP;

COMMIT;
```

# 3.6 Accès par Cluster indexé

## ■ Exemple du cluster indexé

- L'accès aux tables stockées **dans un cluster est efficace seulement si les données sont accédées via la clé du cluster**. la requête suivante est par exemple pénalisée

```
-- création de l'index sur le cluster indexé
Create index idx_iemp4_deptno on iemp4(deptno);
```

```
Select iemp.* From idept, iemp
Where iemp.deptno=idept.deptno;
```

- Certaines **requêtes de consultation sont pénalisées.** La requête ci-dessous par exemple prend plus de temps car on parcours plus de blocs

```
select idept.* from idept ;
```

- **L'insertion et la mise à jour des données peuvent être également pénalisées**

# 3.6 Accès par Cluster indexé

- **Exemple** de **manipulation de Cluster indexé**

```
-- Jointure avec tables en cluster (RBO)

Set autotrace traceonly explain
Alter session set optimizer_mode=rule;
SELECT dname, loc, ename, job, hiredate
FROM IEMP4 e, IDEPT4 d
WHERE e.deptno=d.deptno;
```

```
-----------------------------------------
| Id  | Operation                | Name  |
-----------------------------------------
|   0 | SELECT STATEMENT         |       |
|   1 |  NESTED LOOPS            |       |
|   2 |   TABLE ACCESS FULL      | DEPT4 |
|   3 |   TABLE ACCESS CLUSTER   | EMP4  |
-----------------------------------------
```

# 3.6 Accès par Cluster indexé

- Exemple de manipulation de Cluster indexé

```
-- Jointure avec tables en cluster (CBO)

Alter session set optimizer_mode=all_rows;
Execute Dbms_stats.gather_schema_stats('ORS1');
SELECT dname, loc, ename, job, hiredate
FROM EMP4 e, DEPT4 d
WHERE e.deptno=d.deptno;
```

```
-----------------------------------------------------------------------------------------
| Id  | Operation            | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                 |    14 |   630 |     6   (0)| 00:00:01 |
|   1 |  NESTED LOOPS        |                 |    14 |   630 |     6   (0)| 00:00:01 |
|   2 |   TABLE ACCESS FULL  | DEPT4           |     4 |    80 |     3   (0)| 00:00:01 |
|   3 |   TABLE ACCESS CLUSTER| EMP4           |     4 |   100 |     1   (0)| 00:00:01 |
|*  4 |    INDEX UNIQUE SCAN | IDX_CLU_EMP4_DEPT4 |     1 |       |     0   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------
```

# 3.7 Accès par Cluster haché

## ■ Généralités

- La technique de hachage consiste à répartir les, enregistrements d'une ou plusieurs tables dans les mêmes blocs en fonction de la valeur d'une clé. Une fonction de hachage h fait correspondre à une valeur de la clé c un entier e, de 0 à N.

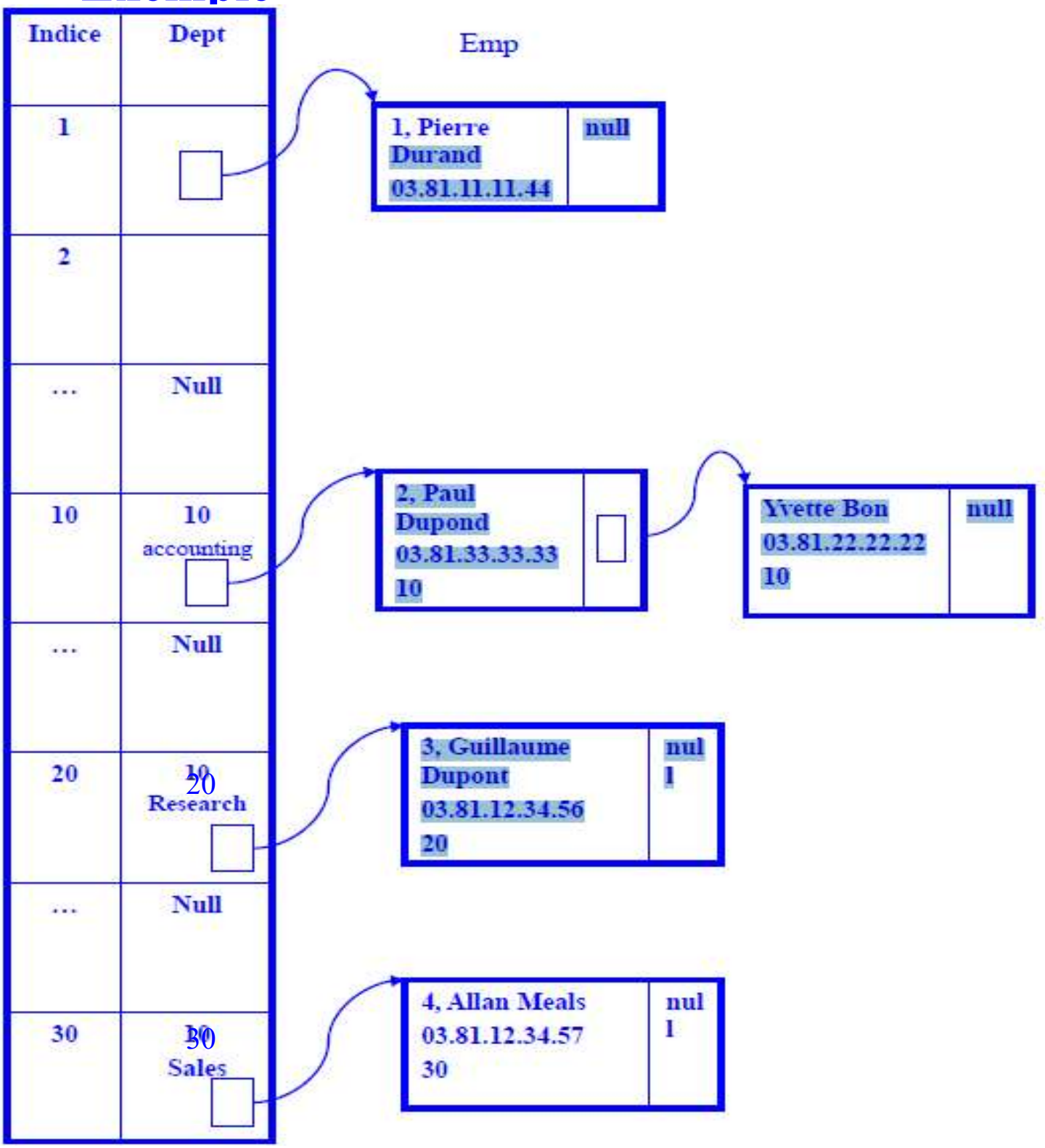
- La recherche d'un enregistrement va consister à appliquer la fonction de hachage pour retrouver le bloc auquel appartient la clé puis de rechercher l'enregistrement dans ce bloc.

- Sous Oracle le balayage par hachage est utilisé pour stocker des lignes (enregistrements) dans des clusters dit hachés.

- **La fonction de hachage fonctionne entre autre selon Algorithme de type modulo.** Si la clause HASH KEY IS n'est pas utilisée, Oracle utilise une fonction de hachage interne. A tout les coûts, la valeur de la clé est divisée par la valeur de HASHKEYS

# 3.7 Accès par Cluster haché

## ■ Exemple



# 3.7 Accès par Cluster haché

- **Exemple** 1 de **manipulation de Cluster haché**

```
-- Création du cluster
CREATE CLUSTER HCLU_HEMP4_HDEPT4 (
DEPTNO NUMBER(2)
) SIZE 500 HASHKEYS 100;

-- Création de la table HDEPT4 dans le cluster
CREATE TABLE HDEPT4(
DEPTNO NUMBER(2) CONSTRAINT  pk_hdept4
   PRIMARY KEY,
DNAME VARCHAR2(14),
LOC VARCHAR2(13) )
CLUSTER HCLU_HEMP4_HDEPT4 (DEPTNO);
```

# 3.7 Accès par Cluster haché

## ■ Exemple 1 de manipulation de Cluster haché

```
-- Création de la table HEMP4 dans le cluster
CREATE TABLE HEMP4(
EMPNO NUMBER(4) constraint pk_hemp4 primary key,
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
MGR NUMBER(4),
HIREDATE DATE,
SAL NUMBER(7,2),
COMM NUMBER(7,2),
DEPTNO NUMBER(2) CONSTRAINT
   fk_hemp4_deptno REFERENCES HDEPT4 (DEPTNO)
)CLUSTER HCLU_HEMP4_HDEPT4 (DEPTNO) ;

-- Insertion de lignes à partir de la table DEPT
INSERT INTO HDEPT4 SELECT * FROM DEPT;

-- Insertion de lignes à partir de la table EMP
INSERT INTO HEMP4 SELECT * FROM EMP;

COMMIT;
```

# 3.7 Accès par Cluster haché

## ■ Exemple 1 de manipulation de Cluster haché

```
-- Jointure avec tables en cluster (RBO)

Set autotrace traceonly explain
Alter session set optimizer_mode=rule;

Create index idx_hemp4_deptno on hemp4(deptno);

SELECT dname, loc, ename, job, hiredate
FROM hEMP4 e, hDEPT4 d
WHERE e.deptno=d.deptno;
```

```
-----------------------------------------
| Id  | Operation              | Name    |
-----------------------------------------
|   0 | SELECT STATEMENT       |         |
|   1 |  NESTED LOOPS          |         |
|   2 |   TABLE ACCESS FULL| HDEPT4 |
|*  3 |   TABLE ACCESS HASH| HEMP4  |
-----------------------------------------
```

# 3.7 Accès par Cluster haché

## ■ Exemple 1 de manipulation de Cluster haché

```
-- Jointure avec tables en cluster (CBO)

Alter session set optimizer_mode=all_rows;
Execute Dbms_stats.gather_schema_stats('ORS1');
SELECT dname, loc, ename, job, hiredate
FROM hEMP4 e, hDEPT4 d
WHERE e.deptno=d.deptno;
```

```
---------------------------------------------------------------------------
| Id  | Operation          | Name   | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |        |    14 |   630 |     4   (0)| 00:00:01 |
|   1 |  NESTED LOOPS      |        |    14 |   630 |     4   (0)| 00:00:01 |
|   2 |   TABLE ACCESS FULL| HDEPT4 |     4 |    80 |     4   (0)| 00:00:01 |
|*  3 |   TABLE ACCESS HASH| HEMP4  |     4 |   100 |            |          |
---------------------------------------------------------------------------
```

# 3.7 Accès par Cluster haché

## ■ Exemple 2 de manipulation de Cluster haché

```
CREATE CLUSTER HCLU_HEMP_HDEPT (
DEPTNO NUMBER(2))
SIZE 500 HASHKEYS 100
HASH IS MOD(deptno, 101);

-- Création de la table HDEPT et HEMP dans le cluster
CREATE TABLE HDEPT(
DEPTNO NUMBER(2) CONSTRAINT  pk_hdept  PRIMARY KEY,
DNAME VARCHAR2(14),
LOC VARCHAR2(13) )
CLUSTER HCLU_HEMP_HDEPT (DEPTNO);

CREATE TABLE HEMP(
EMPNO NUMBER(4) constraint pk_hemp primary key,
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
MGR NUMBER(4),
HIREDATE DATE, SAL
NUMBER(7,2), COMM
NUMBER(7,2),
DEPTNO NUMBER(2) CONSTRAINT fk_hemp_deptno
REFERENCES HDEPT (DEPTNO)
)CLUSTER HCLU_HEMP_HDEPT (DEPTNO) ;
```

# 3.7 Accès par Cluster haché

## ■ Exemple 2 de manipulation de Cluster haché

```
-- Insertion de lignes à partir de la table DEPT
INSERT INTO HDEPT SELECT * FROM DEPT;

-- Insertion de lignes à partir de la table EMP
INSERT INTO HEMP SELECT * FROM EMP;

COMMIT;

Execute Dbms_stats.gather_schema_stats('MOPOLO');
Set autotrace on;
set linesize 200

SELECT dname, loc, ename, job, hiredate
FROM HEMP e, HDEPT d
WHERE e.deptno=d.deptno and e.deptno=10;
```

| Id | Operation | Name | Rows | Bytes | Cost (%CPU) |
|---|---|---|---|---|---|
| 0 | SELECT STATEMENT | | 5 | 225 | 0 (0) |
| 1 | NESTED LOOPS | | 5 | 225 | |
| * 2 | TABLE ACCESS HASH | HDEPT | 1 | 20 | |
| * 3 | TABLE ACCESS HASH | HEMP | 5 | 125 | |

# 3.7 Accès par Cluster haché

## ■ Avantages et Utilisation des clusters hachés

- bien choisir HASH IS, SIZE et HASHKEYS
- convient pour les requêtes ou il ya souvent l'égalité sur la clé de hachage
- convient pour des tables dont la taille est statique (pas d'évolution sensible de leur taille) car risque de chaînage
- un moyen de représenter un tableau de lignes avec HASH IS colonne, HASHKEYS n et SIZE m

## ■ Inconvénients des Clusters hachés

- si on accède souvent séquentiellement aux tables
- si la condition dans la clause where n'est pas '='
- si la table croît continuellement
- si on ne peut prévoir d'avance l'espace que demande le cluster.
- Si l'on modifie souvent la clé de hachage

# 3.8 Accès par index basé sur des fonctions

## ■ Problématiques

- L'application des fonctions sur des colonnes indexées d'une table inhibe les indexes

```
Select * from emp where UPPER(ename)
  ='MILOU';
```

- Afin de forcer l'utilisation de l'INDEX il est possible de créer DES INDEX BASES SUR DES FONCTIONS

- Sont adaptés pour accélérer les **requêtes impliquant des calculs intensifs**

# 3.8 Accès par index basé sur des fonctions

## ■ Exemple

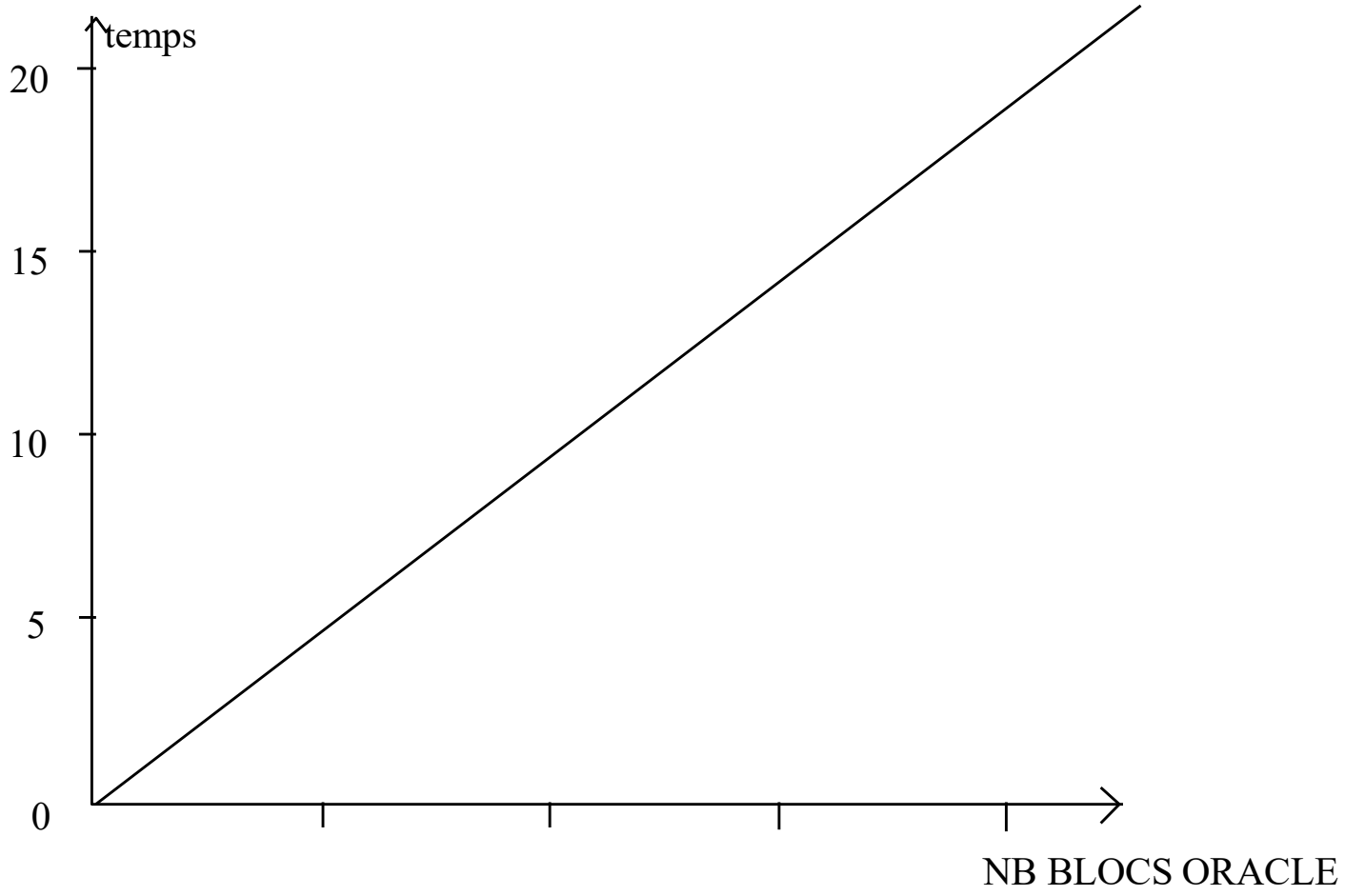
- CREATE INDEX idx_capital_ename ON emp(UPPER(ename));

- Select * from emp where UPPER(ename) ='MILOU';

| Id | Operation | Name |
|---|---|---|
| 0 | SELECT STATEMENT | |
| 1 | TABLE ACCESS BY INDEX ROWID EMP | |
| 2 | INDEX RANGE SCAN | IDX_FCT_CAPITAL_ENAME |

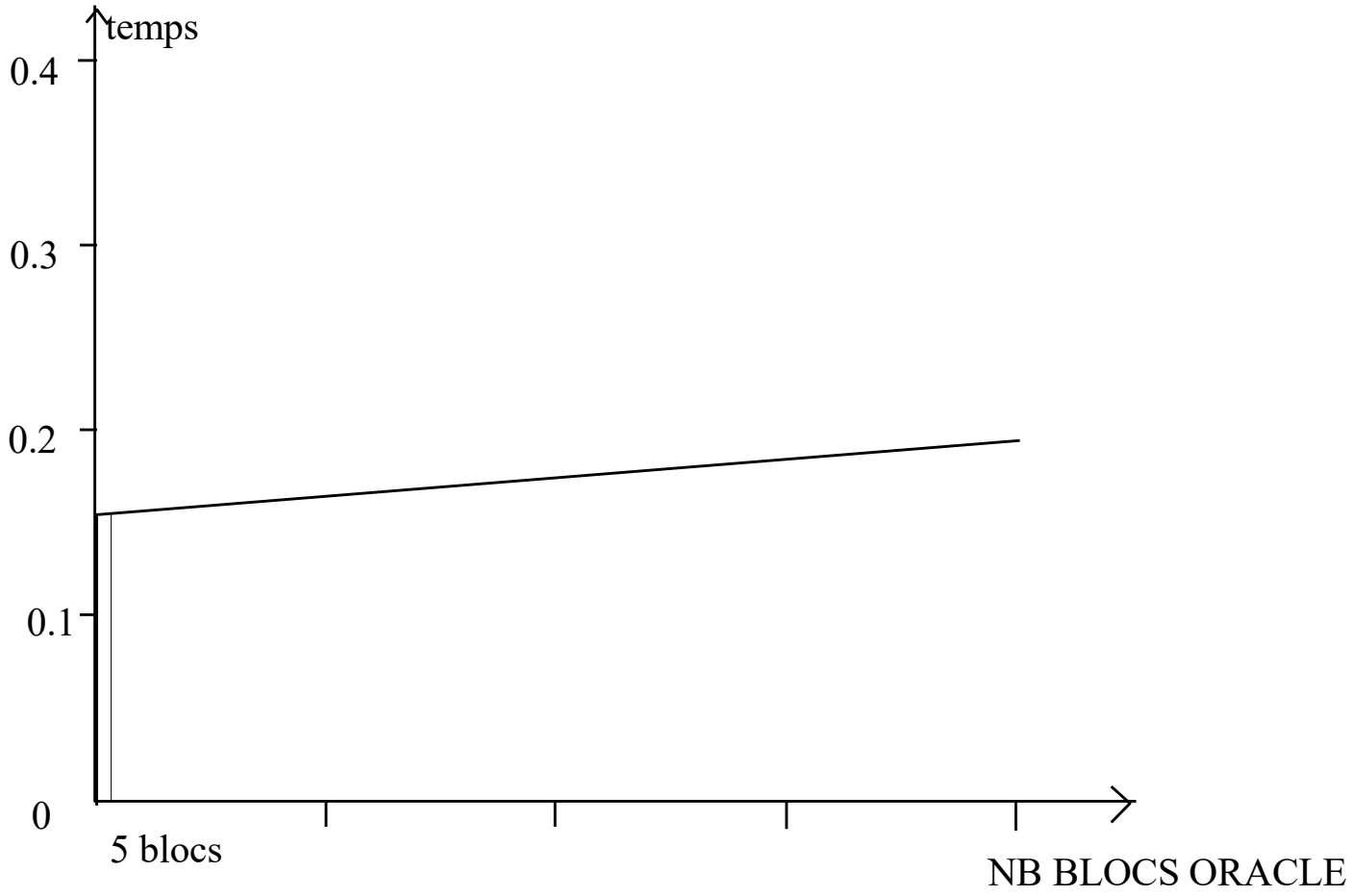
# 3.9 Mesures

## ■ Accès sans index



# 3.9 Mesures

## ■ Accès avec index B-arbre




Note :

- Baisses des performances en cas de mise à jour
- **Accélération des consultations**

# 3.10 Identification des Index non utilisés

- **Oracle permet de recolter des statistiques sur les Index afin d'identifier les Index non utilisés**

- **Intérêts :**
  - Eviter de gaspiller de la place
  - Augmenter les performances en éliminant les pertes de temps unitiles (dispersion des données …)

- **Démarrage du monitoring d'un index**
  - ALTER INDEX idx_emp_ename MONITORING USAGE;
- **Arrêt du monitoring d'un index**
  - ALTER INDEX idx_emp_ename NOMONITORING USAGE;

- **Consultation des informations sur les Index non utilisés**
  - La vue V$OBJECT_USAGE contient ces informations
  - select index_name, used from V$OBJECT_USAGE;

# 3.11 Condition d'utilisation des indexes par l'optimiseur Oracle

## ■ Conditions nécessaires

- L'index a été créé
- L'optimiseur Oracle décide de l'utiliser

# 3.11 Condition d'utilisation des indexes par l'optimiseur Oracle

## ■ Conditions d'utilisation d'indexes

.

| **Expressions contenant des colonnes Indexées** | **Exemple** | **Possibilité d'utiliser l'Index?** |
|---|---|---|
| **Index et Op(Opérateur)** : =, >, <, Between, IN<br><br>ColIndexé **Op** Const; | Select empno, ename FROM emp where job='CLERK'; | OUI |
| **Index et LIKE**<br>ColIndexé **Op** Const;<br>Op(Opérateur) :<br>LIKE | select ename, job from emp where job LIKE 'C%'; | OUI au 1er caractère renseigné |
| | select ename, job from emp where job LIKE '%C%'; | NON |
| **Index et Fonction**<br><br>**Fonction(ColIndexé)** =Const | select * from emp where to_char(hiredate,'dd/mm/yy')='23/01/82'; | NON<br>Résultat du calcul pas dans l'index |
| **Index et Fonction**<br><br>ColIndexé= **Fonction(Const)** | Select * from emp where hiredate=to_date('23/01/82', 'dd/mm/yy'); | OUI |

# 3.11 Condition d'utilisation des indexes par l'optimiseur Oracle

## ■ Conditions d'utilisation d'indexes

| **Expressions contenant des colonnes Indexées** | **Exemple** | **Possibilité d'utiliser l'Index?** |
|---|---|---|
| **Index et IS NULL**<br><br>ColIndexé **IS NULL** | Select empno, ename FROM emp where comm IS NULL; | NON<br>Pas de null dans l'index |
| **Index et is not null**<br><br>ColIndexé **IS NOT NULL** | Select empno, ename FROM emp where comm IS NOT NULL; | NON si RBO<br><br>Possible si CBO |
| **Index et !=**<br><br>ColIndexé **!= Const** | select loc from dept where deptno != 30; | NON si RBO<br><br>Possible si CBO |
| **Index concaténé**<br>ColIndex1=const1 and ColIndex2=const2 and ColIndex3=const3 …<br><br>Create index idx_nom On **EMP(ename, job, deptno)** | select job, ename, deptno from emp where job='CLERK' and **ename='SMITH'** and deptno=20; | OUI Si première colonne dans l'expression NON Sinon. |

# 3.11 Condition d'utilisation des indexes par l'optimiseur Oracle

## ■ Conditions d'utilisation d'indexes

| **Expressions contenant des colonnes Indexées** | **Exemple** | **Possibilité d'utiliser l'Index?** |
|---|---|---|
| **Index et tri : order by** ColIndexé not null<br>**ORDER BY** ColIndexé | SELECT DNAME, loc FROM DEPT ORDER BY DNAME; | OUI possible |
| **Index et Op: (MAX, MIN, AVG, SUM)**<br>Select **Op(ColIndexé)** From nomTable; | Select min(SAL) FROM emp; | OUI index Full scan |
| **Index et OR**<br>Select … From Table Where col1=const1 **OR** Col2=const2 | select ename from emp where empno=7469 OR SAL=2000; | OUI si Col1 et Col2 sont indexés. Sinon NON |
| **Index et OPset (Opérateurs ensemblistes)**<br>Select .. From tab1 where Col1=const **Opset** Select .. From tab1 where Col2=const2; | select * from emp where empno = 7839 UNION select * from emp where sal=3000; | OUI si col1 Indexé et/ou Col2 Indexé. Mettre index sur col1 et un sur col2 |

# 3.11 Condition d'utilisation des indexes par l'optimiseur Oracle

## ■ Conditions d'utilisation d'indexes

| **Expressions contenant des colonnes Indexées** | **Exemple** | **Possibilité d'utiliser l'Index?** |
|---|---|---|
| **Index Group by**<br><br>En cas de group by Elimination de lignes en priorité via WHERE que HAVING | SELECT deptno, ename FROM emp **WHERE deptno > 10 GROUP BY** deptno, ename; | OUI |
| | SELECT deptno, ename FROM emp **GROUP BY** deptno, ename **having deptno > 10**; | NON car construction de groupes puis élimination |

# 3.12 Exercices

## Exercice 3.1

- Créer un index Btree non unique sur la colonne ENAME de la table EMP. L'index doit être localisé dans le tablespace USERS

- Créer un index concaténé sur les colonnes Job et Sal de la table EMP. L'index doit être localiser dans le tablespace USERS. Faites aussi en sorte que 5 transactions puissent être inscrites en même temps

- Créer un index Bitmap sur la colonne COMM (commission) de la table EMP. Le localiser dans le tablespace USERS.

- Calculer la volumétrie des trois index de la table EMP.

- Ecrire une requête qui renvoie des informations sur le job et le salaire pour tous les salaires supérieurs à 7000 et le job égal à CLERK : d'abord en ramenant toutes les colonnes puis en ramenant uniquement le job et le salaire.
  - Avant de lancer les requêtes faire :
    - Set AUTOTRACE ON pour visualiser les plans

- Ecrire une requête qui permet de visualiser les informations sur les employés dont le numéro est supérieur 7839 et dont la commission est supérieure à 0
  - Avant de lancer les requêtes faire :
    - Set AUTOTRACE ON pour visualiser les plans

- Supprimer tous les index et relancer les requêtes précédentes

# 4. Outils d'optimisation

## ■ PLAN

- Généralités
- TKPROF
- EXPLAIN PLAN
- Affichage des plans d'exécutions

# 4.1 Généralités

- **Oracle fournit des utilitaires d'aide à l'optimisation de requêtes SQL. Ce sont :**

  - TKPROF :outil d'analyse de fichiers trace
  - EXPLAIN:donne le plan d'exécution d'un Ordre SQL
  - **Affichage des plans d'exécutions**
  - SQL TUNING ADVISOR : Fournit des conseils rapides et efficaces sur l'optimisation des requêtes
  - SQL ACCESS ADVISOR : fournit des recommandations sur la création des vues matérialisées, d'index …
  - NOTA : Les 2 derniers outils seront étudiés dans un TUNE 2.

# 4.2 TKPROF

- **TKPROF est l'outil d'Oracle qui permet de fournir des statistiques sur les temps d'exécution des requêtes SQL,le nombre de blocs recherchés en mémoire et sur disque ainsi que facultativement le plan d'exécution de l'ordre.**

- **Activation du mode trace**

  - Paramètres à positionner dans le fichier init.ora

    TIMED_STATISTICS= {TRUE | FALSE}
    /* default = TRUE*/

    max_dump_file_size ={integer |unlimited}
    /* default=unlimited*/

    USER_DUMP_DEST=chemin
    /* default= %ORACLE_BASE%\ADMIN\SID\UDUMP */

    **STATISTICS_LEVEL = {BASIC|TYPICAL|ALL}**

    SQL_TRACE=TRUE pour 1 instance avant 10g

# 4.2 TKPROF

- **Activation du mode trace au niveau session**

  - SQL_TRACE peut être activé uniquement pour une session (via la commande SQL alter session) :

    SQL> **alter session set** sql_trace = true;

    ou en exécutant la fonction set_sql_trace du package car SQL_TRACE est déprécié depuis Oracle 10.2 dbms_session

    SQL> EXECUTE sys.dbms_session.set_sql_trace(true);

- Activation du mode trace pour une autre session via l'appel de fonction

  - **Syntaxe**

    EXECUTE sys.dbms_system.set_sql_trace_in_session(
    Session_id, serial_id, {true| false})

  - **Exemple**

    – EXECUTE
    sys.dbms_system.set_sql_trace_in_session(7, 34, true);

# 4.2 TKPROF

## ■ Analyse d'un fichier TKPROF

- Le positionnement des paramètres précédents provoque à chaque lancement d'une requête, la création d'un fichier trace : *.trc

- **Syntaxe**

  $ TKPROF *fichier_trace fichier_sortie*
  [EXPLAIN=username/passwd] [SORT=*sortoption*]

| **Mots clés / Paramètres** | **Description** |
|---|---|
| *fichier_trace* | fichier d'entrée contenant les traces |
| *fichier_sortie* | fichier de sortie |
| *EXPLAIN* | oblige tkprof à lancer EXPLAIN |
| *sortoption* | option de tri dans fichier_sortie :<br>execpu : coût CPU<br>fchcpu :coût de chargement de tuples<br>exedsk : NB de lectures physiques sur disque |

...

# 4.2 TKPROF

## ■ Résultat de l'Analyse

- Le résultat de l'analyse comporte pour chaque Ordre SQL, les informations sur sa compilation, son exécution et les accès aux données.

| | COUNT | CPU | ELAPSED | DISK | QUERY | CURRENT | ROWS |
|---|---|---|---|---|---|---|---|
| **PARSE** | NB Parse | Temp Cpu Parse | Temp Cpu écoulé Parse | Nb Lecture Disque lors du Parse | Nb blocs eu En Lecture | Nb blocs eu En maj | NB de lignes rendues |
| **Execute** | NB Execute | Temp Cpu Execute | Temp Cpu écoulé Execute | Nb Lecture Disque lors du Execute | Nb blocs eu En Lecture | Nb blocs eu En maj | NB de lignes rendues |
| **FETCH** | NB Fetch | Temp Cpu Fetch | Temp Cpu écoulé Fetch | Nb Lecture Disque lors du Fetch | Nb blocs eu En Lecture | Nb blocs eu En maj | NB de lignes rendues |

# 4.2 TKPROF

## ■ Résultat de l'Analyse

| Statistique | Description |
|---|---|
| count | nombre de fois ou un parse, execute ou fetch a été réalisé |
| **cpu** | **temps CPU en seconde pour tous les parse, execute et fetch de la requête** |
| elapsed | temps écoulé en seconde pour tous les parse, execute et fetch de la requête |
| **disk** | **nombre de blocks de données lus sur disque pour tous les parse, execute et fetch** |
| query | nombre total de blocs lu en mode consistant pour tous les parse, execute et fetch (query) |
| **current** | **nombre total de blocs lu en mode courant pour tous les parse, execute et fetch (usuellement en INSERT, UPDATE et DELETE)** |
| rows | nombre total de lignes traitées par la requête (n'inclus pas les lignes traitées par les sous-requêtes). En selection le nombre de lignes apparait sur la ligne du FETCH. En UPDATE, DELETE et INSERT apparait sur la ligne du EXECUTE. |

**querry + current :** Nombre total de blocs accédés

# 4.2 TKPROF

- **Exemple d'activation de tkprof**

```
sql> EXECUTE sys.dbms_session.set_sql_trace(true);
sql> select * from emp where ename ='KING';
sql> EXECUTE sys.dbms_session.set_sql_trace(false);
Sql> select vs.username, vp.spid
from v$process vp , v$session vs
where vp.addr=vs.paddr
and vs.username ='NOMUSER';
sql>exit

$ tkprof …/admin/udump/ora_spid.trc trc1.prf
   EXPLAIN=nomuser/passwd SORT = execpu, fchcpu
    … [sys=yes|no]
```

Cette commande lance tkprof sur les statistiques contenues dans le fichier mytrc.trc. tkprof doit générer les statistiques dans un fichier exploitable appelé trc1.prf en ordonnant le résultat selon les valeurs de tri execpu+fchcpu.

**$ emacs trc1.prf**

/* consultation dans votre éditeur préféré */

# 4.2 TKPROF

## Exemple de fichier résultat

- TKPROF: Release 10.2.0.1.0 - Production on Jeu. Mars 8 00:28:43 2007
- Copyright (c) 1982, 2005, Oracle. All rights reserved.
- Trace file: orcl_ora_2916.trc
- Sort options: execpu fchcpu
- **********
  count = number of times OCI procedure was executed
- cpu = cpu time in seconds executing
- elapsed = elapsed time in seconds executing
- disk = number of physical reads of buffers from disk
- query = number of buffers gotten for consistent read
- current = number of buffers gotten in current mode (usually for update)
- rows = number of rows processed by the fetch or execute call
- **********a
  alter session set sql_trace=true

| call | count | cpu | elapsed | disk | query | current | rows |
|---|---|---|---|---|---|---|---|
| Parse | 0 | 0.00 | 0.00 | 0 | 0 | 0 | 0 |
| Execute | 1 | 0.11 | 0.92 | 0 | 0 | 0 | 0 |
| Fetch | 0 | 0.00 | 0.00 | 0 | 0 | 0 | 0 |
| total | 1 | 0.11 | 0.92 | 0 | 0 | 0 | 0 |

Misses in library cache during parse: 0

Misses in library cache during execute: 1

Optimizer goal: ALL_ROWS

Parsing user id: 5

**********

# 4.2 TKPROF

## Exemple de fichier résultat

```
select * from  emp where ename = 'KING'
call     count       cpu    elapsed       disk      query    current        rows
------- ------  -------- ---------- ---------- ---------- ----------  ----------
Parse        1      0.01       0.09          0          0          0           0
Execute      1      0.02       0.13          0          0          0           0
Fetch        1      0.05       0.51          0          1          3           1
------- ------  -------- ---------- ---------- ---------- ----------  ----------
total        3      0.08       0.73          0          1          3           1

Misses in library cache during parse: 0
Optimizer goal: ALL_ROWS
Parsing user id: 5
********************************************************************************
alter session set sql_trace=false
call     count       cpu    elapsed       disk      query    current        rows
------- ------  -------- ---------- ---------- ---------- ----------  ----------
Parse        1      0.02       0.07          0          0          0           0
Execute      1      0.01       0.01          0          0          0           0
Fetch        0      0.00       0.00          0          0          0           0
------- ------  -------- ---------- ---------- ---------- ----------  ----------
total        2      0.03       0.08          0          0          0           0

Misses in library cache during parse: 1
Optimizer goal: ALL_ROWS

Parsing user id: 5
```

# 4.2 TKPROF

## Exemple de fichier résultat

```
********************************************************************************
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS

call     count       cpu    elapsed       disk      query    current        rows
------- ------  -------- ---------- ---------- ---------- ----------  ----------
Parse        3      0.03       0.17          0          0          0           0
Execute      4      0.14       1.06          0          0          0           0
Fetch        2      0.05       0.51          0          1          3           2
------- ------  -------- ---------- ---------- ---------- ----------  ----------
total        9      0.22       1.74          0          1          3           2

Misses in library cache during parse: 1
Misses in library cache during execute: 1

OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS

call     count       cpu    elapsed       disk      query    current        rows
------- ------  -------- ---------- ---------- ---------- ----------  ----------
Parse        0      0.00       0.00          0          0          0           0
Execute      0      0.00       0.00          0          0          0           0
Fetch        0      0.00       0.00          0          0          0           0
------- ------  -------- ---------- ---------- ---------- ----------  ----------
total        0      0.00       0.00          0          0          0           0
```

# 4.2 TKPROF

## Exemple de fichier résultat

```
Misses in library cache during parse: 0

  3  user  SQL statements in session.
  8  internal SQL statements in session.
 11  SQL statements in session.
  1  statement EXPLAINed in this session.

********************************************************************************
********************************************************************************
Trace file: orcl_ora_2916.trc
Trace file compatibility: 10.01.00
Sort options: execpu  fchcpu
      3  sessions in tracefile.
      3  user  SQL statements in trace file.
      8  internal SQL statements in trace file.
     11  SQL statements in trace file.
      4  unique SQL statements in trace file.
      1  SQL statements EXPLAINed using schema:
         SYSTEM.prof$plan_table
           Default table was used.
           Table was created.
           Table was dropped.
    172  lines in trace file.
      0  elapsed seconds in trace file.
```

# 4.3 La commande Explain Plan

- **La commande EXPLAIN** PLAN affiche le plan d'exécution d'une requête SQL**(select, update, insert et delete).**

- **Le plan d'exécution est la séquence d'opérations dans l'ordre telque exécuté par Oracle.**

- **L'affichage du plan par EXPLAIN permet de voir si les accélérateurs (index, cluster, hash, ...) ont été utilisés ou non.**

- **Syntaxe**

```
sql> EXPLAIN PLAN
[SET STATEMENT_ID='BINOMEx']
[INTO PLAN_TABLE]
FOR SELECT DISTINCT(ENAME) FROM emp;
```

# 4.3 La commande Explain Plan

## Syntaxe

| Mot clé/Paramètre | Description |
|---|---|
| Statement_id | identifiant d'un plan dans la table plan_table |
| Plan_table | table contenant le plan d'exécution d'une requête |

## Installation de la table Plan_table

•sous NT

sql>@%ORACLE_HOME%\rdbms\admin\utlxplan.sql

•sous UNIX

sql>@$ORACLE_HOME\rdbms\admin\utlxplan.sql

# 4.3 La commande Explain Plan

## ■ Structure de la table plan_table

SVRMGR> desc plan_table

| Column Name | Null? | Type |
|---|---|---|
| STATEMENT_ID | | VARCHAR2(30) |
| TIMESTAMP | | DATE |
| REMARKS | | VARCHAR2(80) |
| OPERATION | | VARCHAR2(30) |
| OPTIONS | | VARCHAR2(30) |
| OBJECT_NODE | | VARCHAR2(128) |
| OBJECT_OWNER | | VARCHAR2(30) |
| OBJECT_NAME | | VARCHAR2(30) |
| OBJECT_INSTANCE | | NUMBER(38) |
| OBJECT_TYPE | | VARCHAR2(30) |
| OPTIMIZER | | VARCHAR2(255) |
| SEARCH_COLUMNS | | NUMBER |
| ID | | NUMBER(38) |
| PARENT_ID | | NUMBER(38) |
| POSITION | | NUMBER(38) |
| COST | | NUMBER(38) |
| CARDINALITY | | NUMBER(38) |
| BYTES | | NUMBER(38) |
| OTHER_TAG | | VARCHAR2(255) |
| PARTITION_START | | VARCHAR2(255) |
| PARTITION_STOP | | VARCHAR2(255) |
| PARTITION_ID | | NUMBER(38) |
| OTHER | | LONG |

# 4.3 La commande Explain Plan

## ■ Structure de la table plan_table

| COLONNES | LIBELLES |
|---|---|
| **STATEMENT_ID** | Identifiant de l'analyse d'une requête |
| **OPERATION** | Nom de l'opération interne |
| **OPTIONS** | détail de l'opérations |
| **OBJECT_NAME** | nom de l'objet accédé (nom table, …) |
| **ID** | numéro de l'étape |
| **PARENT_ID** | numéro de l'étape précédente |
| **POSITION** | ordre d'exécution des étapes par rapport à un noeud TIMESTAMP date et l'heure d'analyse |
| REMARKS | Mise à jour SQL possible |
| OBJECT_NODE | nom du lien utilisé pour référencer un objet (table et vue) |
| OBJECT_OWNER | Nom du propriétaire de l'objet |
| OBJECT_INSTANCE | Indique la position originelle de l'objet avant analyse |
| OBJECT_TYPE | complément d'informations sur l'objet traité |
| SEARCH_COLUMNS | Non utilisée pour cette version |
| Partition_id | Nr. étape ayant calculée partition_start /_stop |
| Partition_start ou stop | Borne d'une partition si index partitionné OTHER informations spécifiques à l'exécution d'une étape |

# 4.3 La commande Explain Plan

- **Exemple**

```
SQL>    EXPLAIN PLAN
        SET STATEMENT_ID='PTEST'
        FOR SELECT distinct(ename) from emp;
```

•Le résultat à exploiter se trouve dans PLAN_TABLE

•Voici comment l'exploiter

```
COL OP FORMAT A20
COL OPTIONS FORMAT A20
SET LINESIZE 200
select LPAD ('  ', 2*LEVEL) ||
OPERATION "OP" , OPTIONS, OBJECT_NAME, ID,
PARENT_ID
FROM PLAN_TABLE
WHERE STATEMENT_ID='PTEST'
CONNECT BY PRIOR ID=PARENT_ID
AND STATEMENT_ID='PTEST' START
WITH ID=1
AND STATEMENT_ID='PTEST'
ORDER BY ID;
```

# 4.3 La commande Explain Plan

## ■ Exemple

| OPERATION | OPTIONS | OBJECT_NAME | ID | PARENT |
|---|---|---|---|---|
| SORT | UNIQUE | | 1 | |
| TABLE ACCESS | FULL | EMP | 2 | 1 |

# 4.3 La commande Explain Plan

## ■ Liste des opérations

| Opération | Option | Description |
|---|---|---|
| Aggregate | group by | |
| And-equal | | Intersection de ROWID |
| Bitmap | conversion | Conversion d'une bitmap vers des rowid |
| | index | TO ROWID ou FROM ROWID<br>SINGLE VALUE ou RANGE SCAN ou FULL SCAN |
| | merge | Fusion de plusieurs bitmap en une seule |
| | minus | Différence entre bitmap |
| | or | OR entre deux BITMAP |
| Connect by | | recherche basée sur une arborescence |
| concatenation | | union de toutes les sources |
| count | stopkey | compte le nombre de ligne d'une table<br>Comptage limité par rownum |
| filter | | restriction sur les lignes ramenées |
| first row | | extraction de la première ligne |
| for update | | extraction avec verrouillage de lignes |

# 4.3 La commande Explain Plan

## ■ Liste des opérations

| Opération | Option | Description |
|---|---|---|
| Index | Unique scan | Valide pour une colonne unique ou primary key |
| | Range scan | peut ramèner N lignes pour une même clé |
| | range scan Descending : par cours descendant | |
| | Skip scan : | permet de parcours 1 sous index en cas d'index composé |
| | Full scan : | parcours complet de l'index |
| | Fast Full scan | Parcours de l'index et récupération du résultat final dans l'index |
| Inlist iterator | concatenated | Itère sur l'opération précédente selon les valeurs dans Inlist |
| Intersection | | |
| Merge Join | | |
| | Outer | |
| Minus | | recherche de lignes de la table1 absentes dans la table2 |
| Nested loop | | opération de Jointure imbriquée outer opération de jointure externe imbriquée |

# 4.3 La commande Explain Plan

## ■ Liste des opérations

| Opération | Option | Description |
|---|---|---|
| partition | concatenated | concaténation des résultats des patitions accédées |
| | single | une seule partition est accédée. Cette partition est déterminée à l'exécution |
| | empty | pas de partition accédée |
| projection | | extraction d'un sous-ensemble de colonnes |
| remote | | extraction dans une autre base |
| sequence | | opération avec génération de séquences |
| sort | unique | tri de lignes avec élimination de duplicats |
| | group by | tri pour grouper les lignes |
| | join | tri de lignes avant une jointure par fusion |
| | order by | tri de lignes avec une clause order by |
| table access | full | extraction de toutes les lignes (séquentiel) |
| | cluster | extraction à partir d'une clé cluster+ index |
| | hash | extraction à partir d'une clé de hachage |
| | by index rowid | accès à une table de la base par rowid |
| | By user rowid | |
| | By global index rowid | accès à partir d'un index global sur des données partitionnées |
| | By local index rowid | accès à partir d'un local (partition) |

## ■ Liste des opérations

| Opération | Option | Description |
|---|---|---|
| union | | extraction(union) sans doublon |
| view | | extraction à partir d'une vue |

# 4.4 Affichage des plans d'exécution

- **Les différentes possibilités pour visualiser les plans d'exécutions dépendent du lieu de localisation des plans :**
  - Dans la table PLAN_TABLE via l'exécution de la commande EXPLAIN PLAN. L'affichage se fera :
    - Avec la fonction DISPLAY du package DBMS_XPLAN
    - Ou manuellement via une requête sur plan_table
  - Dans le cache des curseurs (V$SQL_PLAN et `V$SQL_PLAN_STATISTICS_ALL`) pour les requêtes en cours d'exécutions. L'affichage se fera avec:
    - La fonction DISPLAY_CURSOR de DBMS_XPLAN
    - Ou SQL*PLUS AUTOTRACE
    - Ou TKPROF avec l'option EXPLAIN activée
  - Dans les SQL Tuning Sets. Via la fonction. DISPLAY_SQLSET de DBMS_XPLAN
  - Dans les statistiques capturées par AWR (Automatic Workload Repository) via la fonction DISPLAY_AWR de DBMS_XPLAN

# 4.4 Affichage des plans d'exécution

## ■ Le package DBMS_XPLAN

- Le package DBMS_XPLAN fournit les fonctions utiles pour visualiser les plans issus de :
  - La commande EXPLAIN PLAN et la table Plan_table =>DISPLAY
  - Automatic Workload Repository (AWR) =>DISPLAY_AWR (voir Tune 2)
  - Un SQL Tuning Set =>DISPLAY_SQLSET (voir Tune 2)
  - Du cache des curseurs et donc des vues V$SQL_PLAN et V$SQL_PLAN_STATISTICS_ALL =>DISPLAY_CURSOR
- Le package DBMS_XPLAN contient plusieurs tables fonctions qui renvoient des tables contenant un plan d'exécution:
  - **DISPLAY** : Affiche le contenu d'un plan stocké dans plan_table
  - **DISPLAY_CURSOR** : Affichage d'un plan d'un curseur chargé dans le cache des curseurs
  - **DISPLAY_AWR** : Affiche le contenu d'un plan stocké dans AWR
  - **DISPLAY_SQLSET**:Affiche le contenu d'un plan stocké dans un SQL Tuning Set

# 4.4 Affichage des plans d'exécution

## ■ Affichage des plans stockés dans PLAN_TABLE

- Utilisation de la fonction **DISPLAY** de DBMS_XPLAN

```
Set AUTOTRACE OFF

EXPLAIN PLAN FOR SELECT * FROM EMP WHERE
   ENAME='KING';

SELECT PLAN_TABLE_OUTPUT FROM
   TABLE(DBMS_XPLAN.DISPLAY());
```

PLAN_TABLE_OUTPUT

Plan hash value: 3977601012

```
---------------------------------------------------------------------------------------
| Id  | Operation                    | Name         | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |              |     1 |    37 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP          |     1 |    37 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN           | IDX_EMP_ENAME |     1 |       |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("ENAME"='KING')

14 ligne(s) sélectionnée(s).
```

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (`V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL`) avec `DISPLAY_CURSOR`**

  - `La vue V$SQL_PLAN` permet d'examiner les plans d'exécutions pour les curseurs récemment exécutés dans le cache des curseurs

    – Un plan y est mis lors de l'exécution exacte d'une requête

    – Les informations dans cette vue sont similaires à celles fournis par la commande Explain Plan dans Plan_table. EXPLAIN PLAN **montre le plan théorique qui pourra être utilisé pour exécuter une requête. `V$SQL_PLAN`** contient le plan actuel utlisé lors de l'exécution réel de la requête

  - `La vue V$SQL_PLAN_STATISTICS_ALL` forunie les statistiques d'exécution actuelles.

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (V$SQL_PLAN, V$SQL_PLAN_STATISTICS) avec DISPLAY_CURSOR**

  - **Lancer la requête sans Explain Plan**

SELECT * FROM EMP WHERE ENAME='KING';

SELECT PLAN_TABLE_OUTPUT FROM TABLE(
DBMS_XPLAN.DISPLAY_CURSOR());
-- afficher le plan du dernier curseur stocké dans le cache des curseurs

SELECT PLAN_TABLE_OUTPUT FROM TABLE(
DBMS_XPLAN.DISPLAY_CURSOR('1uugk2mqgkgbf'));
Afficher le plan du curseur connaissant son id. Attention : L'utilisateur doit avec le droit d'exécuter la fonction

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------
SQL_ID  1uugk2mqgkgbf, child number 0
-------------------------------------
SELECT * FROM EMP WHERE ENAME='KING'

Plan hash value: 3977601012

-----------------------------------------------------------------------------------------------
| Id  | Operation                   | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |               |       |       |     2 (100)|          |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP           |     1 |    37 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_EMP_ENAME |     1 |       |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("ENAME"='KING')


19 ligne(s) sélectionnée(s).
```

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (`V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL`) `avec SQL*PLUS AUTOTRACE`**

  - Le role plus_trace permet en une seule action d'obtenir le plan d'exécution et les temps mis.

  - Contrairement à EXPLAIN PLAN, la réquête est exécutée avec ou sans affichage du résultat (TRACEONLY).

  - **Création du rôle si SET AUTOTRACE** échoue :

    sql>connect sys/change_on_install@cour
    sql>@c:\orant\sqlplus\admin\plustrce.sql

  - **Visualisation du SQL*PLUS autotrace actuel**
    **SHOW AUTOTRACE**

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (`V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL`) `avec SQL*PLUS AUTOTRACE`**

  - **Activation de SQL*PLUS autotrace**
    - Syntaxe

    SET AUTOTRACE {OFF | ON | TRACE[ONLY]}
    [[EXPLAIN] [STATISTICS]]

    sql> set AUTOTRACE ON
    -- fournit : le résultat, le plan d'exécution et les statistiques
    Show autotrace
    autotrace ON EXPLAIN STATISTICS

    sql> set AUTOTRACE TRACEONLY
    -- pas de résultat uniquement le plan et les statistiques
    Show autotrace
    autotrace TRACEONLY EXPLAIN STATISTICS

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (`V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL`) `avec SQL*PLUS AUTOTRACE`**

  - **Activation de SQL*PLUS autotrace**
    - Syntaxe

    SET AUTOTRACE {OFF | ON | TRACE [ONLY]}
    [[EXPLAIN] [STATISTICS]]

    sql> set AUTOTRACE ON EXPLAIN
    -- fournit : le résultat, le plan d'exécution pas de statistiques
    Show autotrace
    autotrace ON EXPLAIN

    sql> set AUTOTRACE TRACEONLY STATISTICS
    -- pas de résultat uniquement, pas de plan uniquement les statistiques
    Show autotrace
    autotrace TRACEONLY STATISTICS

# 4.4 Affichage des plans d'exécution

- **Affichage des plans stockés dans le cache des curseurs (V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL ) avec SQL*PLUS AUTOTRACE**

  - **Exemple**

    SET AUTOTRACE ON

    SELECT * FROM EMP WHERE ENAME='KING';

```
     EMPNO ENAME      JOB              MGR HIREDATE        SAL       COMM     DEPTNO
---------- ---------- --------- ---------- -------- ---------- ---------- ----------
      7839 KING       PRESIDENT            17/11/81       5000                    10


Plan d'exécution
----------------------------------------------------------
Plan hash value: 3977601012

-------------------------------------------------------------------------------------------
| Id  | Operation                   | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |               |     1 |    37 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP           |     1 |    37 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_EMP_ENAME |     1 |       |     1   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("ENAME"='KING')


Statistiques
----------------------------------------------------------
          0  recursive calls
          0  db block gets
          3  consistent gets
          0  physical reads
          0  redo size
        834  bytes sent via SQL*Net to client
        381  bytes received via SQL*Net from client
          2  SQL*Net roundtrips to/from client
          0  sorts (memory)
          0  sorts (disk)
          1  rows processed
```

# 4.4 Affichage des plans d'exécution

## Exercice 4.1

- 4.1.1 Visualiser avec les différentes méthodes les plans d'exécution de la requête suivante :

  select * from emp where empno=7934;

  a) Utiliser le package DBMS_XPLAN en lisant le plan dans PLAN_TABLE
  – exécuter EXPLAIN PLAN d'abord
  – Utiliser la fonction DISPLAY(…)

  b) Utiliser la vue V$SQL_PLAN
  – Lancer la requête ci-dessus sans EXPLAIN PLAN
  – Afficher le plan en utilisant DISPLAY_CURSOR sans puis avec le paramètre sql_id

  c) Utiliser SQL*Plus `AUTOTRACE`
  - Tester toutes les variantes
  - Et chaque fois lancer la requête ci-dessus

  d) Ecrire soit même la requête qui accède à plan_table
  – exécuter EXPLAIN PLAN d'abord
  – Puis écrire la requête qui consulte plan_table

- 4.1.2 Utiliser TKPROF pour calculer les temps d'exécution de la requête précédente
  – Générer le fichier de trace
  – Bien identifier votre fichier et générer avec TKPROF les temps mis

# 5. L'optimiseur statistique

- PLAN

  - **5.1 Généralités**
  - **5.2 Estimation des coûts**
  - **5.3 LES HINTS ou suggestions**
  - **5.4 Calcul des statistiques**

# 5.1 Généralités

- A la différence de l'optimiseur Syntaxique (qui se fondent uniquement sur la syntaxique des requêtes SQL), l'optimiseur statistique lui s'appuie sur les statistiques sur les objets du schéma qui permettent de calculer le *taux de sélectivité* d'une requête.

- *Le choix d'un plan d'accès se fera entre autre en fonction du taux de sélectivité*.

- *Avantages de l'optimiseur statistique*
  - produit dans de nombreux cas, de meilleurs plans d'exécutions que l'optimiseur syntaxique(surtout pour des requêtes de *grandes tailles à jointures multiples*).

  - dans la plupart des cas, il n'est pas nécessaire de régler manuellement des requêtes : cela permet d'*augmenter la productivité des applications*

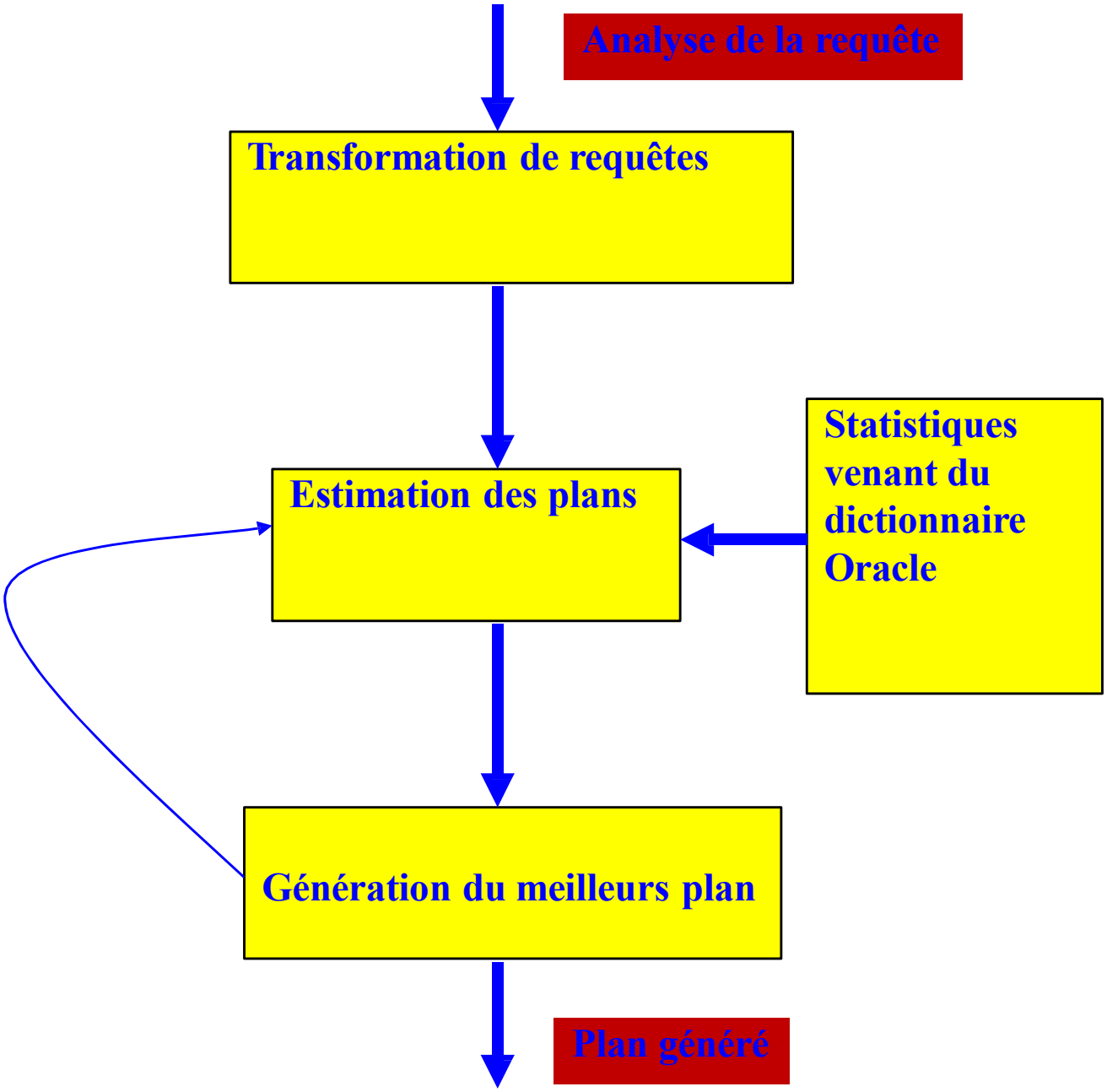
# 5.1 Généralités

- *Avantages de l'optimiseur statistique*

  - laisse le choix au développeur de décider de l'approche d'optimisation : les règles et/ou les coûts : l'intérêt est de *maintenir la compatibilité ascendante* avec les applications développées sous les versions précédentes d'Oracle. Le choix du mode règle peut aussi être fait pour certaine requête ou l'optimiseur statistique ne génère pas de meilleur plan

# 5.1 Généralités

- **L'optimiseur d'Oracle suit les étapes suivantes afin d'optimiser une requête SQL :**



# 5.1 Généralités

## ■ Activation de l'optimiseur de statistique

- L 'activation se fait en modifiant la valeur du paramètre OPTIMIZER_MODE directement ou à travers un HINT ( suggestions)

- OPTIMIZER_MODE doit prendre les valeurs ALL_ROWS ou FIRST_ROWS_n (n=1, 10, 100 ou 1000)

- **Directement pour une session**

Alter session set OPTIMZER_MODE=ALL_ROWS;
Alter session set
OPTIMZER_MODE=FIRST_ROWS_1;

**•Via un HINT pour une seule requête**

SELECT /*+ALL_ROWS */ empno, ename, sal, job
FROM emp WHERE empno=7566;

SELECT /*+FIRST_ROWS(1) */ empno, ename, sal, job
FROM emp WHERE empno=7566;

# 5.2 Estimation des coûts

## ■ Comment Oracle estime-t-il les coûts:

**(1) cost** = (#SRDS * sreadtm + #MRDS * mreadtm
+ #CPUCycles/cpuspeed) / sreadtm

où

**#SRDS** : est le nombre des lectures E/S en mode mono bloc

**Sreadtm** : est le temps nécessaire à la lecture d'un bloc

**#MRDS** : est le nombre des lectures en mode multiblocs

**mreadtm** : est le temps de lecture en mode multiblocs

**CPUCycles** : est le nombre de cycles CPU

**cpuspeed** : est le nombre de cycles CPU par seconde

Cette équation peut être réécrite comme suit :

(1a) **cost** = #SRDS + (#MRDS * mreadtm / sreadtm)
+ #CPUCycles / (cpuspeed * sreadtm)

Elle peut se décliner aussi comme suit:

**cost=cout_io+cout_cpu**

**cout_io**=#SRDS + (#MRDS * mreadtm / sreadtm)

**cout_cpu**=#CPUCycles / (cpuspeed * sreadtm)

# 5.2 Estimation des coûts

## Comment Oracle estime-t-il les coûts:

- Le modèle de coût décrit précédemment existe depuis oracle 9i. Il reste valable sous 10g, 11g et 12c

- Avant Oracle 9i l'optimiseur ne faisait pas de différence de lecture en mode multiblocs et en mode monobloc. L'équation se réduisait à :

  - (1b) Cost=#RDS
    - #RDS inclue les lectures en mode multibloc ou monbloc sans distinctions

- En 9i si les statistiques systèmes ne sont pas calculées, c'est l'équation (1b) qui est utilisée

- A partir de 10g c'est toujours l'équation (1a) qui est utilisée car les statistiques sont toujours initialisées au démarrage d'une base.

# 5.2 Estimation des coûts

## ■ Comment Oracle estime-t-il les coûts:

- **Les statistiques systèmes initialisées au démarrage de la base à partir de 10g sont** :
  - **ioseektim** : temps de recherche d'un bloc sur le disque en ms (milliseconds), valeur par défaut 10
  - **iotfrspeed** : la vitesse de transfert de données en Bytes par ms, valeur par défaut 4096
  - **cpuspeedNW** : le nombre moyen de cycles CPU par seconde, valeur par défaut qui dépende du système.
  - **MBRC** : le nombre des blocs lus en moyenne dans une opération E/S en mode multiblocs

- **Les valeurs de ces statistiques sont ensuite utilisées pour calculer les composants nécessaires à l'équation de coût:**

**sreadtm** = ioseektim + (db_block_size /iotfrspeed)

**mreadtm** = ioseektim+(MBRC *db_block_size/iotfrspeed)

**cpuspeed** = cpuspeedNW

# 5.2 Estimation des coûts

## ■ Comment Oracle estime-t-il les coûts:

- Consultation des valeurs des statistiques systèmes

```
select sname, pname, pval1
from sys.aux_stats$
where sname = 'SYSSTATS_MAIN';
```

| SNAME | PNAME | PVAL1 |
|---|---|---|
| SYSSTATS_MAIN | CPUSPEED | |
| SYSSTATS_MAIN | CPUSPEEDNW | 1219.94751 |
| SYSSTATS_MAIN | IOSEEKTIM | 10 |
| SYSSTATS_MAIN | IOTFRSPEED | 4096 |
| SYSSTATS_MAIN | MAXTHR | |
| SYSSTATS_MAIN | MBRC | |
| SYSSTATS_MAIN | MREADTIM | |
| SYSSTATS_MAIN | SLAVETHR | |
| SYSSTATS_MAIN | SREADTIM | |

9 rows selected.

# 5.2 Estimation des coûts

## ■ Comment Oracle estime-t-il les coûts:

- **Consultation des valeurs des statistiques systèmes avec la procédure dbms_stats.get_system_stats**

```
set serveroutput on
declare
        status varchar2(1000);
    DSTART                  DATE  ;
    DSTOP                   DATE ;
    PNAME VARCHAR2(1000):='CPUSPEEDNW';
    PVALUE                  NUMBER  ;
begin
    dbms_stats.get_system_stats(status, DSTART, DSTOP,
      pname, PVALUE);
    dbms_output.put_line('DSTART='||DSTART);
    dbms_output.put_line('DSTOP='||DSTOP);
    dbms_output.put_line('status='||status);
    dbms_output.put_line(pname||'='||pvalue);
end;
/
DSTART=26-AUG-08
DSTOP=26-AUG-08
status=COMPLETED
CPUSPEEDNW=1219.94750656168
Procédure PL/SQL terminée avec succès.
```

# 5.2 Estimation des coûts

## ■ Comment Oracle estime-t-il les coûts:

- Mise à jour de certaines statistiques systèmes sans attendre leur collecte (en cas de besoin uniquement)

cpuspeed: le nombre des cycles CPU par seconde: 1220

sreadtim: le temps de lecture en mode mono-block: 5 ms

mreadtim: le temps de lecture en mode multi-block: 30 ms

MBRC: le nombre des block lus en mode multi-block : 16

```
Begin
   DBMS_STATS.SET_SYSTEM_STATS('cpuspeed',122
   0);
   DBMS_STATS.SET_SYSTEM_STATS('sreadtim',5.0);
   DBMS_STATS.SET_SYSTEM_STATS('mreadtim',30.
   0);
   DBMS_STATS.SET_SYSTEM_STATS('mbrc',16);
end;
/
```

Procédure PL/SQL terminée avec succès.

# 5.2 Estimation des coûts

- **Exemple de calcul du coût en balayage full**

Dbms_stats.gather_table_stats('ORS1', 'bigemp4');

Soit la requête suivante qui effectue un balayage complet :
Select * from BIGEMP4;

**–Rappel de la formule de calcul du coût (1a) cost=cout_io+cout_cpu**
**cout_io=**#SRDS + (#MRDS * mreadtm / sreadtm)
**cout_cpu=**#CPUCycles / (cpuspeed * sreadtm)

#SRDS est égal à 0 (balayage complet de la table)
#MRDS = Blocks / MBRC =Blocks/16
MBRC=db_file_multiblock_read_count=16
Si ce paramètre a été modifié via alter system et fixé ici à 16 sinon
MBRC= _db_file_optimizer_read_count parameter interne Oracle
valable si =db_file_multiblock_read_count a sa valeur par défaut

BLOCKS est le nombre de blocs lus.
CPUCycles : est le nombre de cycles CPU = colonne cpu_cost de la table plan_table

sreadtm = ioseektim + (db_block_size /iotfrspeed)
sreadtm = 10 + (8192 / 4096) = 12
mreadtm = ioseektim+(MBRC * db_block_size/iotfrspeed)
mreadtm = 10 + (16 * 8192 / 4096) = 42
cpuspeed = cpuspeedNW =1219.95=1220s*1000=1220000ms

# 5.2 Estimation des coûts

- **Exemple de calcul du coût pour un balayage complet**

**cost=cout_io+cout_cpu**

cout_io= 0 + (Blocks/16 * 42 / 12) +1(_table_scan_cost_plus_one)

cout_cpu=#CPUCycles / (1220000 * 12)

– **Récupération de blocks**

Select table_name, num_rows, blocks, avg_row_len
From user_tables Where table_name ='BIGEMP4';

| TABLE_NAME | NUM_ROWS | BLOCKS | AVG_ROW_LEN |
|---|---|---|---|
| BIGEMP4 | 1326797 | 8057 | 39 |

– **Récupération de CPUCycles**

Explain plan set statement_id='ptest' for select * from BIGEMP4;

select id, cost, cpu_cost, io_cost from plan_table where statement_id = 'ptest';

| ID | COST | CPU_COST | IO_COST |
|---|---|---|---|
| 0 | 1793 | 429185572 | 1764 |
| 1 | 1793 | 429185572 | 1764 |

# 5.2 Estimation des coûts

- **Exemple de calcul du coût pour un balayage complet**

**(Rappel 1a)**

**cost=cout_io+cout_cpu**

cout_io= 0 + (Blocks/16 * 42 / 12) +1 (_table_scan_cost_plus_one)

cout_cpu= #CPUCycles / (1220000 * 12)

Cout_io= 0+ (8057 /**16***42/12)+1= 1763.46875

Cout_cpu= 429185572 /(1220000*12)= 29.3159544

Cost= 1763.46875+ 29.3159544= 1792.7847

**Cost=1793**

**A comparer avec :** set autotrace traceonly

select * from BIGEMP4;

```
---------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      | 1282K|    47M|  1793   (2)| 00:00:22 |
|   1 |  TABLE ACCESS FULL| EMP4 | 1282K|    47M|  1793   (2)| 00:00:22 |
---------------------------------------------------------------------
```

# 5.2 Estimation des coûts

- **Synthèse de calcul du coût pour un balayage complet.** db_file_multiblock_read_count modifié

| | | | |
|---|---|---|---|
| cpuspeed= | CPUSPEEDNW= | 1219,95 | Le nombre moyen de cycles CPU par seconde, valeur par défaut qui dépende du système. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOSEEKTIM= | IOSEEKTIM= | 10 | Temps de recherche d'un bloc sur le disque en ms (milliseconds), valeur par défaut 10. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOTFRSPEED= | IOTFRSPEED= | 4096 | La vitesse de transfert de données en Bytes par ms, valeur par défaut 4096. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| db_block_size= | db_block_size= | 8192 | Taille d'un block; show parameter db_block_size |
| MBRC= | db_file_multiblock_read_count= | 16 | Facteur de blocage |
| Blocks= | Blocks= | 8057 | Nombre de blocs à charger. Il fut calculer les stats puis. Select table_name, num_rows, blocks, avg_row_len From user_tables Where table_name ='BIGEMP4'; |
| CPUCycles= | CPUCycles= | 429185572 | Est le nombre de cycles CPU. Pour l'obtenir il faut prendre la valeur de cpu_cost.Explain plan set statement_id='ptest' for select * from BIGEMP4;<br>select id, cost, cpu_cost, io_cost from plan_table where statement_id = 'ptest'; |
| #SRDS= | #SRDS= | 0 | Est le nombre des lectures E/S en mode mono bloc (0 si multiblocks) |
| | _table_scan_cost_plus_one | 1 | Paramètre interne |
| #MRDS = | Blocks / MBRC= | 503,5625 | Est le nombre des lectures en mode multiblocs |
| sreadtm= | ioseektim + (db_block_size /iotfrspeed)= | 12,00 | Est le temps nécessaire à la lecture d'un bloc en mode mono bloc |
| mreadtm= | ioseektim+(MBRC *db_block_size/iotfrspeed)= | 42 | Est le temps de lecture en mode multiblocs |
| cout_io= | #SRDS + (#MRDS * mreadtm / sreadtm) +_table_scan_cost_plus_one= | 1763,46875 | Coût IO |
| cout_cpu= | #CPUCycles / (cpuspeed * sreadtm)= | 29,3171559 | Coût CPU |
| cost= | cout_io+cout_cpu= | 1792,78591 | Coût Total |
| cost : | Avec SET AUTOTRACE TRACEONLY | 1793 | set autotrace trace only; select * from bigemp4; |

# 5.2 Estimation des coûts

- **Synthèse de calcul du coût pour un balayage complet.** db_file_multiblock_read_count modifié

| | | | |
|---|---|---|---|
| cpuspeed= | CPUSPEEDNW= | 1219,95 | Le nombre moyen de cycles CPU par seconde, valeur par défaut qui dépende du système. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOSEEKTIM= | IOSEEKTIM= | 10 | Temps de recherche d'un bloc sur le disque en ms (milliseconds), valeur par défaut 10. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOTFRSPEED= | IOTFRSPEED= | 4096 | La vitesse de transfert de données en Bytes par ms, valeur par défaut 4096. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| db_block_size= | db_block_size= | 8192 | Taille d'un block; show parameter db_block_size |
| MBRC= | db_file_multiblock_read_count= | 16 | Facteur de blocage |
| Blocks= | Blocks= | 8057 | Nombre de blocs à charger. Il fut calculer les stats puis. Select table_name, num_rows, blocks, avg_row_len From user_tables Where table_name ='BIGEMP4'; |
| CPUCycles= | CPUCycles= | 429185572 | Est le nombre de cycles CPU. Pour l'obtenir il faut prendre la valeur de cpu_cost.Explain plan set statement_id='ptest' for select * from BIGEMP4; select id, cost, cpu_cost, io_cost from plan_table where statement_id = 'ptest'; |
| #SRDS= | #SRDS= | 0 | Est le nombre des lectures E/S en mode mono bloc (0 si multiblocks) |
| | _table_scan_cost_plus_one | 1 | Paramètre interne |
| #MRDS = | Blocks / MBRC= | 503,5625 | Est le nombre des lectures en mode multiblocs |
| sreadtm= | ioseektim + (db_block_size /iotfrspeed)= | 12,00 | Est le temps nécessaire à la lecture d'un bloc en mode mono bloc |
| mreadtm= | ioseektim+(MBRC *db_block_size/iotfrspeed)= | 42 | Est le temps de lecture en mode multiblocs |
| cout_io= | #SRDS + (#MRDS * mreadtm / sreadtm) + _table_scan_cost_plus_one= | 1763,46875 | Coût IO |
| cout_cpu= | #CPUCycles / (cpuspeed * sreadtm)= | 29,3171559 | Coût CPU |
| cost= | cout_io+cout_cpu= | 1792,78591 | Coût Total |
| cost : | Avec SET AUTOTRACE TRACEONLY | 1793 | set autotrace trace only; select * from bigemp4; |

# 5.2 Estimation des coûts

- **Synthèse de calcul du coût pour un balayage complet.** db_file_multiblock_read_count avec valeur par défaut

| | | | |
|---|---|---|---|
| cpuspeed= | CPUSPEEDNW= | 1461 | Le nombre moyen de cycles CPU par seconde, valeur par défaut qui dépende du système. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOSEEKTIM= | IOSEEKTIM= | 11 | Temps de recherche d'un bloc sur le disque en ms (milliseconds), valeur par défaut 10. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOTFRSPEED= | IOTFRSPEED= | 4096 | La vitesse de transfert de données en Bytes par ms, valeur par défaut 4096. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| db_block_size= | db_block_size= | 8192 | Taille d'un block; show parameter db_block_size |
| MBRC= | _db_file_optimizer_read_count= | 8 | c'est le paramètre interne utilisé à la place de db_file_multiblock_read_count |
| Blocks= | Blocks= | 8057 | Nombre de blocs à charger. Il faut calculer les stats puis. Select table_name, num_rows, blocks, avg_row_len From user_tables Where table_name ='BIGEMP4'; |
| CPUCycles= | CPUCycles= | 434381502 | Est le nombre de cycles CPU. Pour l'obtenir il faut prendre la valeur de cpu_cost.Explain plan set statement_id='ptest' for select * from BIGEMP4;<br>select id, cost, cpu_cost, io_cost from plan_table where statement_id = 'ptest'; |
| #SRDS= | #SRDS= | 0 | Est le nombre des lectures E/S en mode mono bloc (0 si multiblocks) |
| | _table_scan_cost_plus_one | 1 | Paramètre interne |
| #MRDS = | Blocks / MBRC= | 1007,125 | Est le nombre des lectures en mode multiblocs |
| sreadtm= | ioseektim + (db_block_size /iotfrspeed)= | 13,00 | Est le temps nécessaire à la lecture d'un bloc en mode mono bloc |
| mreadtm= | ioseektim+(MBRC *db_block_size/iotfrspeed)= | 27 | Est le temps de lecture en mode multiblocs |
| cout_io= | #SRDS + (#MRDS * mreadtm / sreadtm) +_table_scan_cost_plus_one= | 2092,72115 | Coût IO |
| cout_cpu= | #CPUCycles / (cpuspeed * sreadtm)= | 22,8706103 | Coût CPU |
| cost= | cout_io+cout_cpu= | 2115,59176 | Coût Total |
| cost : | Avec SET AUTOTRACE TRACEONLY | 2116 | set autotrace trace only; select * from bigemp4; |

# 5.2 Estimation des coûts

- **Synthèse de calcul du coût pour un balayage complet.** db_file_multiblock_read_count avec valeur par défaut

| | | | |
|---|---|---|---|
| cpuspeed= | CPUSPEEDNW= | 1461 | Le nombre moyen de cycles CPU par seconde, valeur par défaut qui dépende du système. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOSEEKTIM= | IOSEEKTIM= | 11 | Temps de recherche d'un bloc sur le disque en ms (milliseconds), valeur par défaut 10. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| IOTFRSPEED= | IOTFRSPEED= | 4096 | La vitesse de transfert de données en Bytes par ms, valeur par défaut 4096. Select sname, pname, pval1 from sys.aux_stats$ where sname = 'SYSSTATS_MAIN'; |
| db_block_size= | db_block_size= | 8192 | Taille d'un block; show parameter db_block_size |
| MBRC= | _db_file_optimizer_read_count= | 8 | c'est le paramètre interne utilisé à la place de db_file_multiblock_read_count |
| Blocks= | Blocks= | 8057 | Nombre de blocs à charger. Il fut calculer les stats puis. Select table_name, num_rows, blocks, avg_row_len From user_tables Where table_name ='BIGEMP4'; |
| CPUCycles= | CPUCycles= | 434381502 | Est le nombre de cycles CPU. Pour l'obtenir il faut prendre la valeur de cpu_cost.Explain plan set statement_id='ptest' for select * from BIGEMP4; select id, cost, cpu_cost, io_cost from plan_table where statement_id = 'ptest'; |
| #SRDS= | #SRDS= | 0 | Est le nombre des lectures E/S en mode mono bloc (0 si multiblocks) |
| | _table_scan_cost_plus_one | 1 | Paramètre interne |
| #MRDS = | Blocks / MBRC= | 1007,125 | Est le nombre des lectures en mode multiblocs |
| sreadtm= | ioseektim + (db_block_size /iotfrspeed)= | 13,00 | Est le temps nécessaire à la lecture d'un bloc en mode mono bloc |
| mreadtm= | ioseektim+(MBRC *db_block_size/iotfrspeed)= | 27 | Est le temps de lecture en mode multiblocs |
| cout_io= | #SRDS + (#MRDS * mreadtm / sreadtm) + _table_scan_cost_plus_one= | 2092,72115 | Coût IO |
| cout_cpu= | #CPUCycles / (cpuspeed * sreadtm)= | 22,8706103 | Coût CPU |
| cost= | cout_io+cout_cpu= | 2115,59176 | Coût Total |
| cost : | Avec SET AUTOTRACE TRACEONLY | 2116 | set autotrace trace only; select * from bigemp4; |

# 5.3 LES HINTS ou suggestions

- **Le choix du mode de fonctionnement et/ou des chemins d'accès peut être forcé dans un Ordre SQL : SELECT, DELETE, UPDATE, INSERT, MERGE**

- **Syntaxe**

  <DELETE | SELECT | UPDATE | INSERT | MERGE]
  /*+ {HINT [*TEXT*] }*/
  [--+ {HINT [*TEXT*] }]

  - Le commentaire doit venir immédiatement après le DELETE, SELECT, INSERT, MERGE ou UPDATE.

  - "+" indique à Oracle d'interpréter le commentaire suivant comme une liste de HINTS

  - ***TEXT***
    - Texte d'explication du ou des HINTS

# 5.3 LES HINTS ou suggestions

- **Chaque ordre SQL ne peut avoir que un et seul commentaire mais plusieurs HINTS peuvent y apparaître**

- **Les suggestions ne s'appliquent qu'à l'ordre SQL ou elles apparaissent**

- **Si une requête utilise des Alias pour renommer les tables, les HINTS doivent faire référence à ces Alias plutôt qu'aux tables elles mêmes**

# 5.3 LES HINTS ou suggestions

## ■ Les différentes classes de HINTS

- **HINTS** Influençant le mode d'optimisation
- **HINTS** Influençant les chemins d'accès
- **HINTS** liées aux opérations de jointure
- **HINTS** pour déterminer l'ordre de jointure
- **HINTS** liées à la transformation des requêtes
- **HINTS** pour la migration d'application ONLINE
- **HINTS** d'activation d'une version de l'optimseur
- **HINTS** additionnels
- **HINTS** influençant les exécutions parallèles

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant le mode d'optimisation

- ALL_ROWS, FIRST_ROWS (n), RULE
- **Exemple**

**Forcer l'optimiseur de coûts en mode FIRST_ROWS_1**

**Alter session set optimizer_mode=rule;**

SELECT /*+FIRST_ROWS(1) */ empno, ename, sal, job
FROM emp WHERE empno=7566;

```
---------------------------------------------------------------------------
| Id  | Operation                    | Name   | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |        |     1 |    22 |     1   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP    |     1 |    22 |     1   (0)| 00:00:01 |
|*  2 |   INDEX UNIQUE SCAN          | PK_EMP |     1 |       |     0   (0)| 00:00:01 |
---------------------------------------------------------------------------
```

**Alter session set optimizer_mode=first_rows_1;**
**SELECT /*+RULE */ empno, ename, sal, job**
**FROM emp WHERE empno=7566;**

```
-----------------------------------------------
| Id  | Operation                     | Name   |
-----------------------------------------------
|   0 | SELECT STATEMENT              |        |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP    |
|*  2 |   INDEX UNIQUE SCAN           | PK_EMP |
-----------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant le chemin d'accès

| | |
|---|---|
| CLUSTER(tab1) | Accès via un cluster |
| FULL(tab1) | Balayage Full de table1 |
| HASH(table1) | Accès hachage dynamique |
| INDEX(tab1 [index1...]) | Accès via l'index |
| INDEX_ASC(tab1 [index1 …]) | Accès en ordre ascendant |
| INDEX_COMBINE(tab1[index1 …]) | Force l'accès via index bitmap |
| INDEX_DESC(tab1 [index1 ... ]) | Accès en ordre descendant |
| INDEX_FFS(tab1 [index1 …]) | Force l'index Fast full scan |
| INDEX_JOIN(tab1 [index1 …]) | Force la jointure d'index |
| INDEX_SS(tab1 [index1 …]) | Index concaténé utilisé en l'absence de la première colonne dans Where |
| INDEX_SS_ASC(tab1 [index1 …]) | Index skip scan ascendant |
| INDEX_SS_DESC(tab1 [index1 …]) | Index skip scan descendant |
| NATIVE_FULL_OUTER_JOIN | Use native full outer join (HASH) |
| NO_INDEX(table1 index1 …) | Pas d'accès via l'index |
| NO_INDEX_FFS(tab1 index1) | Pas d'index fast full scan |
| NO_INDEX_SS(tab1 index1) | No index skip scan |
| NO_NATIVE_FULL_OUTER_JOIN | No Use native full outer join |

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant le chemin d'accès

- **Exemple 1**
  - **Influencer l'usage d'un Index**

Create index idx_emp_comm on emp(comm);
execute Dbms_stats.gather_schema_stats('MOPOLO');
SELECT * FROM emp WHERE comm is not null;

```
-----------------------------------------------------------------------------------------
| Id  | Operation                   | Name        | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |             |     4 |   152 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| EMP         |     4 |   152 |     2   (0)| 00:00:01 |
|*  2 |   INDEX FULL SCAN           | IDX_EMP_COMM |     4 |       |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------
```

SELECT /*+ **NO_INDEX(emp idx_emp_comm)** */ *
FROM emp WHERE comm is not null;

```
-----------------------------------------------------------------------------
| Id  | Operation          | Name | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |      |     4 |   152 |     3   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| EMP  |     4 |   152 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant le chemin d'accès

- **Exemple 2**

**-- Forcer l'usage d'index bitmap**

```
Alter session set optimizer_mode=all_rows;
Set autotrace traceonly;

SELECT--+INDEX_COMBINE(CUSTOMERS)
        cust_last_name FROM  SH.CUSTOMERS
    WHERE ( CUST_GENDER= 'F' AND
    CUST_MARITAL_STATUS =  'single')
    OR     CUST_YEAR_OF_BIRTH BETWEEN '1917'
      AND '1920';
```

```
-----------------------------------------------------------------------------------------------
| Id  | Operation                    | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                     |  4663 | 93260 |   572   (1)| 00:00:07 |
|   1 |  TABLE ACCESS BY INDEX ROWID | CUSTOMERS           |  4663 | 93260 |   572   (1)| 00:00:07 |
|   2 |   BITMAP CONVERSION TO ROWIDS |                    |       |       |            |          |
|   3 |    BITMAP OR                 |                     |       |       |            |          |
|   4 |     BITMAP MERGE             |                     |       |       |            |          |
|*  5 |      BITMAP INDEX RANGE SCAN | CUSTOMERS_YOB_BIX   |       |       |            |          |
|   6 |     BITMAP AND               |                     |       |       |            |          |
|*  7 |      BITMAP INDEX SINGLE VALUE| CUSTOMERS_MARITAL_BIX |     |       |            |          |
|*  8 |      BITMAP INDEX SINGLE VALUE| CUSTOMERS_GENDER_BIX |      |       |            |          |
-----------------------------------------------------------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant le chemin d'accès

- **Exemple 2**
  - **Sans Forcer l'usage d'index bitmap**

```
SELECT cust_last_name
FROM  SH.CUSTOMERS
WHERE ( CUST_GENDER= 'F' AND
CUST_MARITAL_STATUS =  'single')
OR     CUST_YEAR_OF_BIRTH BETWEEN '1917'
   AND '1920';
```

```
-----------------------------------------------------------------------------
| Id  | Operation         | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |           |     2 |    40 |     2   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| CUSTOMERS |     2 |    40 |     2   (0)| 00:00:01 |
-----------------------------------------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS Influençant l'ordre de jointure

| | |
|---|---|
| LEADING(tab1[tab2…]) | Choix de la table directrice |
| ORDERED(tab1 tab2 ... ) | Joindre les tables dans l'ordre d'apparition |

## ■ HINTS pour les opérations de jointure

| | |
|---|---|
| USE_NL(tab1 tab2 …) | Force la jointure nested loop. Tab1 est la table directrice |
| USE_NL_WITH_INDEX | |
| USE_MERGE(tab1 tab2 …) | Force la jointure par tri-fusion |
| USE_HASH(tab1 tab2 …) | Force le hash join |
| NO_USE_NL | Pas de nested loop |
| NO_USE_MERGE | Pas de tri-fusion |
| NO_USE_HASH | Pas de hash join |

# 5.3 LES HINTS ou suggestions

## ■ Exemple Influençant l'ordre de jointure

Create index idx_emp_deptno on emp(deptno);

Execute Dbms_stats.gather_schema_stats('MOPOLO');

select dname,ename, job, sal

from dept, emp where emp.deptno=dept.deptno;

```
-----------------------------------------------------------------------------------
| Id  | Operation                    | Name    | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |         |    14 |   476 |     6  (17)| 00:00:01 |
|   1 |  MERGE JOIN                  |         |    14 |   476 |     6  (17)| 00:00:01 |
|   2 |   TABLE ACCESS BY INDEX ROWID| DEPT    |     4 |    52 |     2   (0)| 00:00:01 |
|   3 |    INDEX FULL SCAN           | PK_DEPT |     4 |       |     1   (0)| 00:00:01 |
|*  4 |   SORT JOIN                  |         |    14 |   294 |     4  (25)| 00:00:01 |
|   5 |    TABLE ACCESS FULL         | EMP     |    14 |   294 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------
```

select /*+USE_NL(emp dept)*/dname,ename, job, sal

from dept, emp where emp.deptno=dept.deptno;

```
-----------------------------------------------------------------------------------
| Id  | Operation                    | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |               |    14 |   476 |     6   (0)| 00:00:01 |
|   1 |  NESTED LOOPS                |               |       |       |            |          |
|   2 |   NESTED LOOPS               |               |    14 |   476 |     6   (0)| 00:00:01 |
|   3 |    TABLE ACCESS FULL         | DEPT          |     4 |    52 |     3   (0)| 00:00:01 |
|*  4 |    INDEX RANGE SCAN          | IDX_EMP_DEPTNO |     5 |       |     0   (0)| 00:00:01 |
|   5 |   TABLE ACCESS BY INDEX ROWID| EMP           |     4 |    84 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS pour la migration d'application ONLINE

| | |
|---|---|
| CHANGE_DUPKEY_ERROR_INDEX | En cas de violation de la clé, provoque l'erreur ORA-38911 au lieu ORA-001 (insert et update) |
| IGNORE_ROW_ON_DUPKEY_INDEX | Ignore la violation de clés primaires lors d'insert |
| RETRY_ON_ROW_CHANGE | Répéter l'opération en cas ORA_ROWSCN |

## ■ HINTS d'activation d'une version de l'optimseur

| | |
|---|---|
| Optimizer_features_enable | Choix de l'optimizer d'une version donnée optimizer_features_enable('11.1.0.6') |

# 5.3 LES HINTS ou suggestions

## ■ HINTS pour la transformation des requêtes

| | |
|---|---|
| FACT(tab1) | Indique la table des faits |
| MERGE | Fusion des vues pour chaque requête |
| NO_EXPAND | Evite les expansion OR |
| NO_FACT (tab1) | Indique que la table n'est pas une table de faits |
| NO_MERGE | Interdit la fusion des vues |
| NO_QUERY_TRANSFORMATION | Interdit la transformation |
| NO_REWRITE | interdit la réécriture de la requête |
| NO_STAR_TRANSFORMATION | No use bitmap for star transformation |
| NO_UNNEST | Forces Oracle not to Unnest sub-queries. |
| REWRITE | Force la réécriture de la requête |
| STAR_TRANSFORMATION | Use bitmap indexes for a Star Transformation execution path. |
| UNNEST | A global panacea for badly written sub-queries |
| USE_CONCAT ? | Réécriture des OR en UNION ALL et désactive les INLIST |

# 5.3 LES HINTS ou suggestions

## ■ HINTS pour la transformation des requêtes

- **Exemple**

select dname, job, sum(sal)
from dept4, EMP4 where
emp4.deptno=dept4.deptno group by dname, job;

```
-----------------------------------------------------------------------------------------
| Id  | Operation                    | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                 |     9 |   207 |     3   (0)| 00:00:01 |
|   1 |  MAT_VIEW REWRITE ACCESS FULL| MV_MASSE_SALARIALE |     9 |   207 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------
```

select /*+NO_REWRITE*/ dname, job, sum(sal)
from dept4, emp4 where emp4.deptno=dept4.deptno
group by dname, job;

```
-----------------------------------------------------------------------------------------
| Id  | Operation                    | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |          |    11 |   407 |  2303   (6)| 00:00:28 |
|   1 |  HASH GROUP BY               |          |    11 |   407 |  2303   (6)| 00:00:28 |
|   2 |   MERGE JOIN                 |          |    11 |   407 |  2302   (6)| 00:00:28 |
|   3 |    TABLE ACCESS BY INDEX ROWID| DEPT4    |     4 |    52 |     2   (0)| 00:00:01 |
|   4 |     INDEX FULL SCAN          | PK_DEPT4 |     4 |       |     1   (0)| 00:00:01 |
|*  5 |    SORT JOIN                 |          |    11 |   264 |  2300   (6)| 00:00:28 |
|   6 |     VIEW                     | VW_GBC_5 |    11 |   264 |  2299   (6)| 00:00:28 |
|   7 |      HASH GROUP BY           |          |    11 |   165 |  2299   (6)| 00:00:28 |
|   8 |       TABLE ACCESS FULL      | EMP4     | 1300K|   18M|  2215   (2)| 00:00:27 |
-----------------------------------------------------------------------------------------
```

# 5.3 LES HINTS ou suggestions

## ■ HINTS additionnels

| | |
|---|---|
| APPEND | Impose le direct path pour insert into select … |
| APPEND_VALUES | Impose le direct path pour insert into values( …) bulk insert |
| CACHE(tab1) | Cache blocks from Full Table Scan |
| PUSH_PRED | Push a join predicate between a view (or inline view) and a table into the view. |
| PUSH_SUBQ | Force Oracle to evaluate sub-query before other non-indexed predicates. |
| QB_NAME | Query block name |
| CURSOR_SHARING_EXACT | Permet le partage de plan s'ils sont égaux |
| DRIVING_SITE | Force le site de traitement la requête |
| DYNAMIC_SAMPLING | Echantillonnage dynamqique |
| MODEL_MIN_ANALYSIS | Permet de réduire le temps de compilation |
| NOCACHE(tab1) | Do not cache blocks from a Full Table Scan |
| NO_PUSH_PRED | Stop Oracle from pushing join predicates. |

# 5.3 LES HINTS ou suggestions

## ■ HINTS influençant le parallélisme

| | |
|---|---|
| PARALLEL | Impose l'exécution de la requête en parallèle |
| PARALLEL_INDEX | Impose l'accès à l'index en parallèle |
| PQ_DISTRIBUTE | Organise la distribution des lignes entre producteurs |
| NO_PARALLEL | Impose l'exécution de la requête en série |
| NO_PARALLEL_INDEX | Impose l'accès à l'index en série |
| | |

# 5.3 LES HINTS ou suggestions

## Exercice 5.1

- Ajouter dans la table des EMP une colonne SEXE de type CHAR(1) qui peut prendre l'une des valeurs suivantes :
  - F : pour Féminin
  - M : Masculin

- Modifier les lignes existantes et faire en sorte que 50% des Employés soient de type F et l'autre moitié de type M

- Créer un index sur la colonne SEXE

- Analyser la table avec la commande suivante
  EXECUTE DBMS_STATS.Gather_table_stats('EMP')

- Ecrire une requête qui renvoit les informations sur les employés de sexe masculin

- En cas d'utilisation des index utiliser les HINTS pour désactiver l'index. En cas de non utilisation faire l'inverse.

- Influencer le mode d'optimisation en choisissant le mode RULE

- Refaire l'exercice avec les tables BIGEMP2 (environ 130000 lignes). Installer pour cela le script BigEmp2_2010.sql

# 5.4 Calcul des statistiques

## ■ Généralités

- **Les statistiques sont essentielles pour le bon fonctionnement de l'optimiseur statistique**

- **Les objets concernés par les statistiques sont** : Les TABLES, les INDEX, les CLUSTERS et les PARTITIONS

- **L'optimiseur statistique utilise les statistiques pour estimer :**
  - Le taux de sélectivités des prédicats
  - Le coût des plans d'exécutions
  - Les méthodes d'accès et les méthodes de jointure
  - Le coût CPU et le coût des entrées/sorties

# 5.4 Calcul des statistiques

## ■ Généralités

- **Voici les différentes approches pour calculer MANUELLEMENT les statistiques :**
  - La commande ANALYZE
  - **Le package DBMS_STATS (conseillé)**
- **Voici les différentes approches pour calculer AUTOMATIQUEMENT les statistiques** :
  - **Le job GATHER_STATS_JOB (Oracle 10G)**
  - **La tâche « auto optimizer stats collection » (Oracle 11g et plus)**
  - L'Echantillonnage dynamique. Si aucune statistique n'est fournie.

# 5.4.1 La commande ANALYZE

## La commande ANALYZE

- La commande ANALYZE permet pour l'Index, la table, le cluster ou la partition :
  - de mettre à jour et *supprimer les informations sur les statistiques*,
  - vérifier l'*intégrité et la validité des données*

- La commande ANALYZE permet pour la table, le cluster ou la partition
  - d'identifier les *lignes chaînées*.

- **Nota**
  - **Dans la pratique, on utilisera les fonctions du package DBMS_STATS pour calculer les statistiques**

# 5.4.1 La commande Analyze

## ■ Vérification de la structure

- **Syntaxe**

  ANALYZE <

  INDEX index VALIDATE STRUCTURE[INTO nom_table] |

  <TABLE table | CLUSTER cluster | PARTITION>

  VALIDATE STRUCTURE [CASCADE] [INTO nom_table] >

  – . vérifie si la structure d'une table, d'un index ou d'un cluster est corrompue,

  – . **ANALYZE INDEX** index **VALIDATE STRUCTURE** correspond à l'ancien VALIDATE INDEX, permet de récolter des statistiques sur les index et met à jours les tables :
    - INDEX_HISTOGRAM (distribution des clés)
    - INDEX_STATS (statistiques sur la structure physique d'un index).

  – **Exemple**

  sql>ANALYZE INDEX idx_ename

  VALIDATE STRUCTURE ;

# 5.4.1 La commande Analyze

## ■ Vérification de la structure

- **La clause INTO nom_table**
  - Cette clause permet en cas d'inconsistance de rejeter dans la table *nom_table* les iditenfiants des lignes en cause.
  - ***Nom_table*** par défaut est **INVALID_ROWS**. Elle est obtenue en exécutant le script **UTLVALID.SQL**
  - SQL> DESC INVALID_ROWS

| Name | Null? | Type |
|---|---|---|
| OWNER_NAME | | VARCHAR2(30) |
| TABLE_NAME | | VARCHAR2(30) |
| PARTITION_NAME | | VARCHAR2(30) |
| HEAD_ROWID | | ROWID |
| ANALYZE_TIMESTAMP | | DATE |

# 5.4.1 La commande Analyze

## ■ Vérification de la structure

- **Exemple**

```
Sql> CREATE TABLE PEMP (EMPNO
      NUMBER(4) NOT NULL, ENAME
      VARCHAR2(10),
       JOB VARCHAR2(9),
       MGR NUMBER(4),
       HIREDATE DATE,
       SAL NUMBER(7,2),
       COMM NUMBER(7,2),
       DEPTNO NUMBER(2))
PARTITION BY RANGE(job)(
PARTITION p1 VALUES LESS THAN ('CLERK')
TABLESPACE ts_ora_users,
PARTITION p2 VALUES LESS THAN ('PRESIDENT')
TABLESPACE tools,
PARTITION p3 VALUES LESS THAN (MAXVALUE)
TABLESPACE ts_ora_users );
Sql> >@c:\orant\rdbms\admin\utlvalid.sql
Sql>ANALYZE TABLE pemp PARTITION (p1)
 VALIDATE STRUCTURE INTO INVALID_ROWS;
```

**Note :** - la table et/ou l'index doivent être partitionnés

# 5.4.1 La commande Analyze

## ■ Identification des chaînages de lignes

- Le chaînage d'une ligne sur plusieurs blocs physiques d'une table ou d'un cluster peut faire *baisser les performances des requêtes*. Il est nécessaire d'identifier ces lignes afin de remédier au chaînage.

- **Syntaxe**

  **sql>ANALYZE <TABLE table | CLUSTER cluster>**
  **LIST CHAINED ROWS** [INTO nom_table];

  – les numéros de lignes chaînées sont rejetées dans la table NOM_TABLE. Par défaut son nom CHAINED_ROWS.

  **–La Table CHAINED_ROWS**

  sql>@%ORACLE_HOME%\rdbms\admin\utlchain.sql
  SQL> desc chained_rows

| Name | Null? | Type |
|---|---|---|
| OWNER_NAME | | VARCHAR2(30) |
| TABLE_NAME | | VARCHAR2(30) |
| CLUSTER_NAME | | VARCHAR2(30) |
| PARTITION_NAME | | VARCHAR2(30) |
| HEAD_ROWID | | ROWID |
| ANALYZE_TIMESTAMP | | DATE |

# 5.4.1 La commande Analyze

## ■ Identification des chaînages de lignes

- **Exemple**

**1)**

sql>ANALYZE TABLE emp
LIST CHAINED ROWS INTO chained_rows;
**-- les statistiques doivent avoir été calculées**

**2)**

sql>CREATE TABLE emp2 as SELECT emp.*
FROM emp, chained_rows
WHERE emp.rowid=head_rowid;
**-- sauvegarde des lignes chaînées**

3)DELETE FROM emp WHERE rowid in
(SELECT head_rowid FROM chained_rows);
**-- suppression des lignes chaînées**

4)INSERT INTO emp
SELECT * FROM emp2;
**-- réinsertion des lignes dans emp**

# 5.4.1 La commande Analyze

## ■ Mise à jour des statistiques

- **Syntaxe**

  ANALYZE<<INDEX index |TABLE
  table>[PARTITION(nom)] | CLUSTER cluster>
  <COMPUTE STATISTICS [for_clause]
  | ESTIMATE STATISTICS [for_clause]
  [SAMPLE valeur <ROWS | PERCENT>
  | DELETE> STATISTICS

  – **Signifacation des clauses**
    - **COMPUTE** = analyse complète sur toutes les données
    - **ESTIMATE** = analyse sur un échantillon avec :
    - **ROWS** : NB lignes de l'échantillon
    - **PERCENT** : pourcentage de lignes de l'échantillon
    - **PARTITION (nom)** : nom d'une partition

  – **Exemple :**

  Sql>ANALYZE TABLE EMP COMPUTE STATISTICS;
  -- calcule les stats sur la table emp et ses index

  – NOTA :
    - Les clauses COMPUTE et ESTIMATE ne sont maintenues que pour des besoins de compatibilités. Elles seront abandonnées un jour. Utiliser à la place DBMS_STATS
    - Les clauses qui ne calculent pas les statistiques restent valides

# 5.4.1 La commande Analyze

## *Mise à jour des statistiques*

- **Note :**
  - produit des statistiques sur la structure physique et la distribution de l'index, la table ou le cluster
  - Par défaut, l'analyse estimée se fait sur 1064 lignes. Si ce nombre dépasse la moitié des lignes ou des clés, l'estimation se fait sur la table entière.

- **La clause FOR**
  - Permet d'affiner l'analyse sur une TABLE.
  - **Syntaxe**

    For-clause ::=

    [FOR TABLE – statistique sur la table seule

    | FOR ALL [INDEXED] COLUMNS [**SIZE** entier]

    | FOR COLUMNS [SIZE entier] colonne1…[SIZE entier

    | FOR ALL [**LOCAL**] INDEXES]

    **SIZE** : donne la plage de distribution dans ***_histograms***

    **LOCAL**: Lié à une partition

# 5.4.1 La commande Analyze

## *Mise à jour des statistiques*

- **La clause FOR**

  - **Exemple**

    - SQL>ANALYZE TABLE emp COMPUTE STATISTICS FOR TABLE;
      -- calcule les statistiques de la table ***emp*** seule
      -- sans celles sur ses index

    - SQL>ANALYZE TABLE emp COMPUTE STATISTICS FOR ALL INDEXED COLUMNS;
      -- calcule les statistiques pour toutes les colonnes indexées

    - SQL>ANALYZE TABLE emp COMPUTE STATISTICS FOR COLUMNS sal SIZE 20;
      -- **SIZE** Permet de fournir à l'optimiseur des informations sur la distributivité de l'index sur 20 Plages. UTILE UNIQUEMENT EN CAS D'UNE MAUVAISE REPARTITION DES CLES.

# 5.4.1 La commande Analyze

## *Résultat de l'Analyse d'un Index*

- **Le résultat se retrouve dans les vues, User_Indexes, dba_indexes *_*IND_STATISTICS* et all_indexes.**

- **Les colonnes mis à jour sont :**

| **NOM COLONNE** | **DESCRIPTION** |
|---|---|
| BLEVEL | Hauteur de l'arbre * |
| LEAF_BLOCKS | Nombre de blocs feuilles |
| DISTINCT_KEYS | Nbre de clés distincts |
| AVG_LEAF_BLOCKS_PER_KEY | Nbre moyen de blocs feuilles par clés |
| AVG_DATA_BLOCKS_PER_KEY | Nombre de blocs de données moyens pointés par une clé |
| CLUSTERING_FACTOR | Dimensionnement des données<br>Toujours évalué exactement |

- **Exemple**

```
sql>ANALYZE INDEX idx_deptno COMPUTE STATISTICS
ou
sql>ANALYZE INDEX idx_deptno ESTIMATE
   STATISTICS SAMPLE 600 ROWS STATISTICS;
```

# 5.4.1 La commande Analyze

- *Résultat de l'Analyse d'une table*
  - **_TABLES*
  - **_OBJECT_TABLES*
  - **_tab_columns*
  - **_TAB_STATISTICS*
  - **_TAB_COL_STATISTICS*
  - **_TAB_HISTOGRAMS*
  - **_TAB_PARTITIONS*
  - **_TAB_SUBPARTITIONS*
  - **_IND_PARTITIONS*
  - **_IND_SUBPARTITIONS*
  - **_PART_COL_STATISTICS*
  - **_PART_HISTOGRAMS*
  - **_SUBPART_COL_STATISTICS*
  - **_SUBPART_HISTOGRAMS*

*(ou * représente user, all , dba ou cdb)*

# 5.4.1 La commande Analyze

- *Résultat de l'Analyse d'une table*

  - **Dans les vues *_tables**

| Nom colonne | Désignation |
|---|---|
| num_rows | Nombre de lignes |
| Blocks | Nombre de blocs alloués à la table ayant des données |
| empty_blocs | Nombre de blocs vides alloués à la table |
| avg_space | Moyenne de l'espace libre/bloc |
| chain_cnt | Nbre de lignes chaînées |
| avg_row_len | Moyenne de la longueur des lignes |

  - **Dans les vues *_tab_columns**

| Nom colonne | Désignation |
|---|---|
| num_distinct | NB de valeurs distinctes pour chaque colonne. |
| low_value | Valeur la plus petite (2ème si plus 3 lignes) |
| High_value | Valeur plus grande (avant dernière si plus de 3 lignes) |

# 5.4.1 La commande Analyze

## *Résultat de l'Analyse d'un cluster*

- L'information importante pour un cluster est la moyenne du nombre de blocs pour une clé cluster.

- Le résultat de l'analyse d'un cluster se trouve dans les vues :
- *_CLUSTERS

- ***Dans les vues *_CLUSTERS***

| Nom colonne | Désignation |
|---|---|
| AVG_BLOCKS_PER_KEY | Moyenne du nombre de blocs pour une clé de cluster. |

- **Nota**
  - Les TABLES contenant les résultats du calcul des statistiques avec ANALYZE seront lors du calcul avec le package DBMS_STATS

# 5.4.1 La commande Analyze

- *Exercice 5.2*

  - Calculer les statistiques sur les colonnes de la table EMP et ces index en utilisant la commande ANALYZE

  - Afficher via des requêtes SQL les informations concernant les statistiques

    - Sur la table EMP
    - Sur les colonnes de la table EMP
    - Sur les Index de la table EMP

    - Nota : Bien identifier les tables ou trouver les statistiques

# 5.4.1 La commande Analyze

- *Exercice 5.3*

L'objectif de cet exercice est de provoquer et identifier les chaînages dans une table et d'y remédier.

```
-- suppression de la table Etudiant si elle existe
DROP TABLE ETUDIANT;

-- Création de la table étudiant
CREATE TABLE ETUDIANT(
ETU# NUMBER (10) constraint pk_etu primary key,
NOM          VARCHAR2(30),
CV           VARCHAR2(1000)
)
pctfree 0;
-- pourquoi PCTFREE est –il mis à 0  ?

-- Supprimer la séquence si elle existe
DROP SEQUENCE Seq_ETU_NUMERO;

-- Créer la séquence
CREATE SEQUENCE Seq_ETU_NUMERO start with 1;

-- Supprimer un type Array de 20 noms s'il existe
DROP TYPE TabNoms_T;

-- Définir un type Array de 20 noms
CREATE OR REPLACE TYPE TabNoms_T as Varray(20) of varchar2(20)
/
```

# 5.4.1 La commande Analyze

- *Exercice 5.3*

```
-- Exécuter le programme ci-dessous pour insérer des lignes dans
-- la table étudiant
DECLARE
   CV1          ETUDIANT.CV%TYPE:='UN CV PLEIN, UN CV
PLEIN, UN CV PLEIN, UN CV PLEIN, UN CV PLEIN, UN CV
PLEIN, UN CV PLEIN,  UN CV PLEIN, UN CV PLEIN, UN CV
PLEIN,  UN CV PLEIN, UN CV PLEIN, UN CV PLEIN,  UN CV
PLEIN, UN CV PLEIN, UN CV PLEIN, UN CV PLEIN, UN CV
PLEIN, UN CV PLEIN, UN CV PLEIN, UN CV PLEIN';
lesNoms       tabNoms_t:= tabNoms_t('Dupont', 'Durand', 'Foudil',
'Foudelle', 'Akim', 'Bleck', 'Zembla', 'Tintin', 'Milou', 'Mopolo',
'Malik', 'Amidou', 'Mamadou', 'Mariama', 'Marine', 'Mouloud',
'Chang', 'Li', 'Bruce', 'Balak');
j                 number:=1;
BEGIN
For i In 1 .. 10000
loop
If j>lesNoms.limit THEN -- j ne doit pas dépasser la table l'ARRAY
j:=1;
END IF;
 INSERT INTO ETUDIANT
VALUES (tpors.seq_etu_numero.nextval,lesNoms(j), cv1);
j:=j+1;          -- incrémenter j
End loop;
COMMIT;
END;
/
```

# 5.4.1 La commande Analyze

- *Exercice 5.3*

  - Calculer les statistiques sur la table et les index avec la commande ANALYZE
  - Vérifier s'il ya des chaînages
  - Provoquer les chaînages et assurer vous de leur présence
  - S'il y'a des chaînages, faire en sorte de les supprimer

# 5.4.2 Le package DBMS_STATS

- **Le package `DBMS_STATS` peut être utilisé pour :**
  - **Générer et gérer les statistiques utilisées par l'optimiseur**
  - **Collecter, modifier, visualiser, exporter, importer et supprimer** les statistiques
  - **Identifier ou nommer les statistiques collectées**
  - **Les statistiques sont collectées sur** :
    - Les Index, les tables, les colonnes, et les partitions
    - Tous les objets d'un schéma (utilisateur) ou la base entière
  - **La collecte des statistiques peut être sérialisée ou parallélisée**
  - **Nota :** Il est recommandé d'utiliser ce package en cas de calcul manuel des statistiques

# 5.4.2 Le package DBMS_STATS

## ■ Le package dbms_stats

- Ce package contient plusieurs procédures et fonctions pour calculer et manipuler des statistiques avec un granule très fin

- **Analyse des statistiques sur une table** (table, colonne, index)

```
BEGIN
DBMS_STATS.GATHER_table_STATS
   (OWNNAME => ‘SCOTT', TABNAME =>
   ‘EMP',
METHOD_OPT => 'FOR COLUMNS SIZE 10
   quantity_on_hand');
END;
/
```

# 5.4.2 Le package DBMS_STATS

## Le package dbms_stats

- **Analyse des statistiques sur les objets d'un utilisateur**

```
EXECUTE
   DBMS_STATS.GATHER_SCHEMA_STATS(
 'SCOTT',DBMS_STATS.AUTO_SAMPLE_SIZE);
```

- **Analyse des statistiques sur un index**
  - GATHER_INDEX_STATS

- **Analyse des statistiques sur le dictionnaire de données Oracle**
  - GATHER_DICTIONARY_STATS

- **Analyse des statistiques sur tous les objets de la base**
  - GATHER_DATABASE_STATS

## 5.4.3 Collecte automatique des statistiques

- **A partir d'Oracle 10G, Oracle automatise la collecte des statistiques :**
  - **Les statistiques sont collectées automatiquement sur tous les objets de la base**
  - Le tâche « **auto optimizer stats collection** » est utilisé pour la collecte et maintenir les statistiques. EN 10g c'était DBMS_STATS_JOB
  - Plusieurs Packages sont utilisés pour gérer cette tâche :
    - DBMS_AUTO_TASK_ADMIN
    - DBMS_AUTO_TASK_EXPORT
    - DBMS_AUTO_TASK
    - DBMS_AUTOTASK_PRVT
    - DBMS_AUTO_TASK_IMMEDIATE
    - DBMS_AUTOTASK_PRVT_LIB
    - DBMS_SCHEDULER

## 5.4.3 Collecte automatique des statistiques

- **A partir d'Oracle 10G, Oracle automatise la collecte des statistiques :**

  - Plusieurs vues permettent de visualiser les informations sur cette tâche

    DBA_AUTOTASK_CLIENT,
    DBA_AUTOTASK_CLIENT_HISTORY,
    DBA_AUTOTASK_CLIENT_JOB,
    DBA_AUTOTASK_JOB_HISTORY,
    DBA_AUTOTASK_OPERATION,
    DBA_AUTOTASK_SCHEDULE,
    DBA_AUTOTASK_TASK,
    DBA_AUTOTASK_WINDOW_CLIENTS,
    DBA_AUTOTASK_WINDOW_HISTORY)

# 5.4.3 Collecte automatique des statistiques

- **Administration de la tâche « auto optimizer stats collection »**

  - Le package DBMS_AUTO_TASK_ADMIN fournnit les fonctions nécessaires
    - Procédure DISABLE : désactive la collecte automatique

```
BEGIN
  DBMS_AUTO_TASK_ADMIN.DISABLE(
   client_name => 'auto optimizer stats collection',
   operation => NULL,  window_name => NULL);
END;
/
```

    - Procédure ENABLE : active la collecte automatique

```
BEGIN
  DBMS_AUTO_TASK_ADMIN.ENABLE(
   client_name => 'auto optimizer stats collection',
   operation => NULL,  window_name => NULL);
END;
/
```

# 5.4.3 Collecte automatique des statistiques

- **Administration de la tâche « auto optimizer stats collection »**

  - Vérification en consultant la vue

select CLIENT_NAME, OPERATION_NAME, STATUS from DBA_AUTOTASK_OPERATION;

```
CLIENT_NAME                      OPERATION_NAME                 STATUS
-------------------------------- ------------------------------ --------
auto optimizer stats collection  auto optimizer stats job       ENABLED
auto space advisor               auto space advisor job         ENABLED
sql tuning advisor               automatic sql tuning task      ENABLED
```

select CLIENT_NAME, OPERATION_NAME, STATUS from DBA_AUTOTASK_TASK;

```
CLIENT_NAME                      OPERATION_NAME                 STATUS
-------------------------------- ------------------------------ --------
sql tuning advisor               automatic sql tuning task      ENABLED
auto optimizer stats collection  auto optimizer stats job       ENABLED
auto space advisor               auto space advisor job         ENABLED
```

# 5.4.3 Collecte automatique des statistiques

- **Gestion des fenêtres de maintenance des statistiques**
  - En 11g la configuration de la collecte par **défaut** se fait chaque jour de la semaine.
    - Lundi à Vendredi : 22h à 2h
    - Samedi et dimanche : 6h à 20h
  - **Vérification de la configuration actuelle**

```
select window_name, repeat_interval,
duration from dba_scheduler_windows
order by window_name;
```

| WINDOW_NAME | REPEAT_INTERVAL | DURATION |
|---|---|---|
| FRIDAY_WINDOW | freq=daily;byday=FRI;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 |
| MONDAY_WINDOW | freq=daily;byday=MON;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 |
| SATURDAY_WINDOW | freq=daily;byday=SAT;byhour=6;byminute=0; bysecond=0 | +000 20:00:00 |
| SUNDAY_WINDOW | freq=daily;byday=SUN;byhour=6;byminute=0; bysecond=0 | +000 20:00:00 |
| THURSDAY_WINDOW | freq=daily;byday=THU;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 |
| TUESDAY_WINDOW | freq=daily;byday=TUE;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 |
| WEDNESDAY_WINDOW | freq=daily;byday=WED;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 |
| WEEKEND_WINDOW | freq=daily;byday=SAT;byhour=0;byminute=0;bysecond=0 | +002 00:00:00 |
| WEEKNIGHT_WINDOW | freq=daily;byday=MON,TUE,WED,THU,FRI;byhour=22;byminute=0; bysecond=0 | +000 08:00:00 |

# 5.4.3 Collecte automatique des statistiques

## Gestion des fenêtres de maintenance des statistiques

- **Modification des fenêtres de collecte**

```
EXECUTE
DBMS_SCHEDULER.SET_ATTRIBUTE('MONDAY_WINDOW',
'repeat_interval','freq=daily;byday=MON;byhour=05;bymin
ute=0; bysecond=0');
EXECUTE DBMS_SCHEDULER.SET_ATTRIBUTE('TUESDAY_WINDOW',
'repeat_interval','freq=daily;byday=TUE;byhour=05;bymin
ute=0; bysecond=0');
EXECUTE
DBMS_SCHEDULER.SET_ATTRIBUTE('WEDNESDAY_WINDOW','repeat
_interval','freq=daily;byday=WED;byhour=05;byminute=0;
bysecond=0');
EXECUTE DBMS_SCHEDULER.SET_ATTRIBUTE('THURSDAY_WINDOW',
'repeat_interval','freq=daily;byday=THU;byhour=05;bymin
ute=0; bysecond=0');
EXECUTE DBMS_SCHEDULER.SET_ATTRIBUTE('FRIDAY_WINDOW',
'repeat_interval','freq=daily;byday=FRI;byhour=05;bymin
ute=0; bysecond=0');
EXECUTE DBMS_SCHEDULER.SET_ATTRIBUTE('SATURDAY_WINDOW',
'repeat_interval','freq=daily;byday=SAT;byhour=13;bymin
ute=0; bysecond=0');
EXECUTE DBMS_SCHEDULER.SET_ATTRIBUTE('SUNDAY_WINDOW',
'repeat_interval','freq=daily;byday=SUN;byhour=13;bymin
ute=0; bysecond=0');
```

# 5.4.3 Collecte automatique des statistiques

- **Gestion des fenêtres de maintenance des statistiques**
  - **Vérification de la configuration modifiée**

```
select window_name, repeat_interval,
    duration from dba_scheduler_windows
    order by window_name;
```

| WINDOW_NAME | REPEAT_INTERVAL | DURATION |
|---|---|---|
| FRIDAY_WINDOW | freq=daily;byday=FRI;byhour=05;byminute=0; bysecond=0 | +000 04:00:00 |
| MONDAY_WINDOW | freq=daily;byday=MON;byhour=05;byminute=0; bysecond=0 | +000 04:00:00 |
| SATURDAY_WINDOW | freq=daily;byday=SAT;byhour=13;byminute=0; bysecond=0 | +000 20:00:00 |
| SUNDAY_WINDOW | freq=daily;byday=SUN;byhour=13;byminute=0; bysecond=0 | +000 20:00:00 |
| THURSDAY_WINDOW | freq=daily;byday=THU;byhour=05;byminute=0; bysecond=0 | +000 04:00:00 |
| TUESDAY_WINDOW | freq=daily;byday=TUE;byhour=05;byminute=0; bysecond=0 | +000 04:00:00 |
| WEDNESDAY_WINDOW | freq=daily;byday=WED;byhour=05;byminute=0; bysecond=0 | +000 04:00:00 |
| WEEKEND_WINDOW | freq=daily;byday=SAT;byhour=0;byminute=0;bysecond=0 | +002 00:00:00 |
| WEEKNIGHT_WINDOW | freq=daily;byday=MON,TUE,WED,THU,FRI;byhour=22;byminute=0; bysecond=0 | +000 08:00:00 |

# 5.4.3 Collecte automatique des statistiques

- **Vérification le jour suivant de l'exécution des tâches**

```
col client_name format a35
col job_status format a15
col job_start_time format a32
col job_duration format a20
set linesize 200
select client_name, job_status, job_start_time,
job_duration from dba_autotask_job_history order by
job_start_time;
```

| CLIENT_NAME | JOB_STATUS | JOB_START_TIME | JOB_DURATION |
|---|---|---|---|
| auto optimizer stats collection | SUCCEEDED | 09/11/14 06:21:29,827000 +01:00 | +000 00:01:32 |
| auto space advisor | SUCCEEDED | 09/11/14 06:21:29,827000 +01:00 | +000 00:00:42 |
| sql tuning advisor | SUCCEEDED | 09/11/14 06:21:29,840000 +01:00 | +000 00:00:12 |
| auto optimizer stats collection | SUCCEEDED | 09/11/14 11:31:23,005000 +01:00 | +000 00:00:34 |
| auto space advisor | SUCCEEDED | 09/11/14 11:31:23,008000 +01:00 | +000 00:00:30 |

# 5.4.3 Collecte automatique des statistiques

## ■ Collecte immédiate des statistiques

- Le package DBMS_AUTO_TASK_IMMEDIATE permet de collecter immédiatement les statistiques sans attendre la fenêtre de collecte. Il possède une procédure :

  – GATHER_OPTIMIZER_STATS

- Utilisation de procédure

```
BEGIN
        DBMS_AUTO_TASK_IMMEDIATE.
GATHER_OPTIMIZER_STATS;

END;
/
```

# 5.4.3 Collecte automatique des statistiques

- **Configuration de la collecte automatique des statistiques**

  - Monitoring des ordres LMD
  - Echantillonnage
  - Parallélisation de la collecte
  - Les histogrammes
  - Cascade

# 5.4.3 Collecte automatique des statistiques

## Monitoring des ordre LMD

- Permet de surveiller les ordres SQL et les tables tronquées
- Permet à la collecte automatique d'identifier les segments avec les statistiques anciennes
- Le MONITORING est activée par défaut si le paramètre d'initialisation
  - `STATISTICS_LEVEL` est `TYPICAL` ou `ALL`
- Activation manuelle du monitoring d'une table
  - **Alter table nomTable monitoring;**

# 5.4.3 Collecte automatique des statistiques

## ■ Monitoring des ordre LMD

- **Visualisation des tables monitorées**

select table_name, monitoring
from user_tables order by table_name;

| | TABLE_NAME | MONITORING |
|---|---|---|
| 1 | APPARAIT | YES |
| 2 | ARTISTE | YES |
| 3 | AVION | YES |
| 4 | BONUS | YES |
| 5 | DEPT | YES |
| 6 | DUMMY | YES |
| 7 | EMP | YES |
| 8 | OUVRAGE | YES |
| 9 | PILOTE | YES |
| 10 | SALGRADE | YES |
| 11 | SALLE | YES |
| 12 | TABLEAU | YES |
| 13 | VOL | YES |

# 5.4.3 Collecte automatique des statistiques

- **Monitoring des ordre LMD**

  - Visualisation des changements sur les tables dûs aux ordres LMD dans la vue : `USER_TAB_MODIFICATIONS`

  - `select * from USER_TAB_MODIFICATIONS;`

| | TABLE_NAME | PARTITION_NAME | SUBPARTITION_NAME | INSERTS | UPDATES | DELETES | TIMESTAMP | TRUNCATED | DROP_SEGMENTS |
|---|---|---|---|---|---|---|---|---|---|
| 1 | PILOTE | (null) | (null) | 0 | 1 | 0 | 18/03/22 | NO | 0 |

  - Mise à jour des données de la vue USER_TAB_MODIFICATION avec des informations récentes, il faut invoquer la procédure suivante

execute
DBMS_STATS.FLUSH_DATABASE_MONITORING_INFO;

# 5.4.3 Collecte automatique des statistiques

## ■ Monitoring des ordres LMD

- Pour mettre à jour manuellement les statitisques des tables ayant de vielles statistiques, il faut appeler :

  - `GATHER_DATABASE_STATS` ou `GATHER_SCHEMA_STATS`
  - Si les `OPTIONS` sont mises à `GATHER STALE` ou `GATHER AUTO`

# 5.4.3 Collecte automatique des statistiques

## ■ Echantillonage

- La collecte des statistiques peut se **calculer sur un échantillon** pour minimiser la consommation des ressources

- **Manuellement il faut utiliser le paramètre `ESTIMATE_PERCENT`** des procédures du package `DBMS_STATS` pour modifier le pourcentage d'échantillonnage

- Pour automatiser l'échantillonnage, fixer le paramètre `DBMS_STATS.AUTO_SAMPLE_SIZE` (default)

  – **Ce paramètre permet à Oracle de calculer le bon échantillon pour chaque objet automatiquement**

```
EXECUTE DBMS_STATS.GATHER_SCHEMA_STATS
('TPORS',DBMS_STATS.AUTO_SAMPLE_SIZE);
```

# 5.4.3 Collecte automatique des statistiques

- **Dégré de parallélisme pour la collecte automatique des statistiques**

  - Cette collecte peut se faire en mode parallèle ou en mode série

  - Par défaut le degré de parallélisme se fait automatiquement

  - Il est possible de fixer manuellement le dégré de parallélisme grâce à l'argument DEGREE des procédures du package `DBMS_STATS`

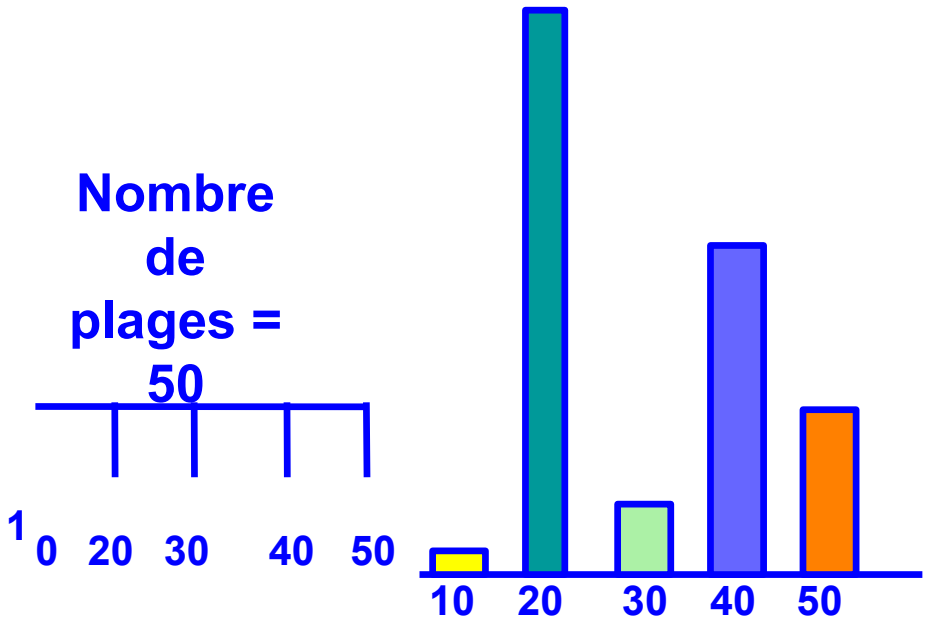
  - Fixer le paramètre DEGREE à `DBMS_STATS.AUTO_DEGREE` (default) permet à Oracle de sélectionner lui-même le bon dégré de parallélisme même si la collecte est faite manuellement.

# 5.4.3 Collecte automatique des statistiques

## Calcul automatique des histogrammes

- Permet à l'optimiseur de produire de meilleurs plan (plus précis)
- Améliore le taux de sélectivité par une meilleure connaissance des données
- Evite la répartition uniforme des lignes sur les clés d'une colonne

| Valeur de la colonne | Nombre de lignes |
|---|---|
| 10 | 10 |
| 20 | 1050 |
| 30 | 126 |
| 40 | 567 |
| 50 | 248 |




**Note** : il existe 2 types d'histogrammes

-**Frequency histograms** : par défaut le nombre de plages = nombres de valeurs distinctes (répartition des lignes exacts par plage)

-**Hight balanced histograms** : répartition équitable des lignes par plage

# 5.4.3 Collecte automatique des statistiques

## Création des histogrammes

- Le mécanisme de récolte automatique des statistiques créés les histogrammes par défaut.

- Il est possible de modifier les valeurs par défaut grâce aux fonctions du packages : `DBMS_STATS`

- Le package `DBMS_STATS` permet aussi de créer les histogrammes manuellement

- L'exemple ci-dessous permet de créer histogrammes avec 50 plages sur la colonne `PROD_LIST_PRICE`

```
EXECUTE dbms_stats.gather_table_stats
        ('sh','products',
         method_opt => 'for columns size 50
                       prod_list_price');
```

- L'exemple ci-dessous permet de fixer le calcul automatique des histogrammes pour toutes les colonnes

  - `Exec dbms_stats.set_param (method_opt => 'for all columns size auto');`

# 5.4.3 Collecte automatique des statistiques

- **Consultation des histogrammes**

```
SELECT column_name, num_distinct, num_buckets,
    histogram
  FROM USER_TAB_COL_STATISTICS
WHERE histogram <> 'NONE';

SELECT column_name, num_distinct, num_buckets,
    histogram
  FROM USER_TAB_COL_STATISTICS
WHERE column_name = 'PROD_LIST_PRICE';
```

# 5.4.3 Collecte automatique des statistiques

- **Conseils sur les histogrammes**

  - **L'option par défaut est** : `DBMS_STATS METHOD_OPTS is FOR ALL COLUMNS SIZE AUTO`, qui permet la création automatique des histogrammes.

  - **Il est possible d'en créer manuellement :**
    - Sur les colonnes souvent utilisées dans les clauses `WHERE`
    - Sur les colonnes qui ont une mauvaise distribution des données (skewed columns)

  - **Eviter les histogrammes s'ils font baisser les performances.**
    - Plus de consommation de place
    - Si à cause du ré-calcule des histogrammes, la distribution des données évolue constamment
    - Sur **les requêtes avec les Bind Variables** (variables liées)

# 5.4.3 Collecte automatique des statistiques

## ■ Accès aux valeurs des Bind Variables

- L'optimiseur accède aux valeurs des bind variable à la première invocation du curseur
- Cela est fait pour déterminer la sélectivité du prédicat
- Il n'y aura plus d'accès lors des prochaines invocation du CURSEUR
- Les curseurs sont des objets partagés par plusieurs transactions (voir les règles de partage)

# 5.4.3 Collecte automatique des statistiques

## Calcul des statistiques sur les index en CASCADE

- Le mécanisme de récolte automatique des statistiques est configuré par défaut pour collecter les statisques sur les index au moment de la collecte des statistiques sur les tables

- Il est possible de changer le comportement par défaut en modifiant l'option CASCADE du package DBMS_STATS

- Mettre l'option `CASCADE` à :
  - `TRUE` pour collecter les statistiques sur les index
  - `DBMS_STATS.AUTO_CASCADE` permet à Oracle de déterminer s'il faut collecter les statistiques ou non sur les index

# 5.4.3 Collecte automatique des statistiques

## ■ Exemple de calcul de statistiques

```
dbms_stats.gather_table_stats
('sh'              -- schema
,'customers'       -- table
, null             -- partition
, 20               -- sample size(%)
, false            -- block sample?
,'for all columns' -- column spec
, 4                -- degree of parallelism
,'default'         -- granularity
, true ); -- cascade to indexes
```

```
dbms_stats.set_param('CASCADE',
                     'DBMS_STATS.AUTO_CASCADE');

dbms_stats.set_param('ESTIMATE_PERCENT','5');

dbms_stats.set_param('DEGREE','NULL');
```

# 5.4.3 Collecte automatique des statistiques

## ■ Quand calculer manuellement les statistiques

- Cela dépend du rythme de collecte automatique
- Pour modifier les fréquences de la collecte automatique afin de s'adapter aux besoins
- La collecte manuelle se fera :
  - Pour les objets volatiles
  - Pour les objets modifiés dans les opérations Batch

# 5.4.3 Collecte automatique des statistiques

- **Quand calculer manuellement les statistiques**

  - **Collecte manuelle de statistiques**

```
BEGIN
   DBMS_STATS.GATHER_TABLE_STATS('OE','ORDERS');
   DBMS_STATS.LOCK_TABLE_STATS('OE','ORDERS'); END;
```

  - Pour **les objets modifiés dans les opérations batchs calculer les statistiques dans le batch**

  - Pour **les nouveaux objets calculer les statistiques immédiatement après leur création**

# 5.4.3 Collecte automatique des statistiques

## ■ Echantillonnage dynamique

- L'échantillonnage dynamique est utilisé automatiquement si :
  - Le coût de la collecte des statistiques est minimal comparé au temps d'exécution
  - La requête est exécutée plusieurs fois

# 5.4.3 Collecte automatique des statistiques

## Verrouillage des statistiques

- Empêche la collecte automatique

- Est utilisé en priorité pour les tables volatiles
  - Un verrouillage sans statistiques implique un échantillonnage dynamique
  - Un verrouillage avec statistiques est fait pour les données représentatives

```
EXECUTE DBMS_STATS.LOCK_TABLE_STATS
('owner name', 'table name');

EXECUTE DBMS_STATS.LOCK_SCHEMA_STATS
('owner name');

SELECT stattype_locked
FROM dba_tab_statistics;
```

# 5.4.3 Collecte automatique des statistiques

- **Vérification des statistiques sur les tables**

```
SELECT last_analyzed analyzed, sample_size,
        monitoring,table_name
FROM dba_tables
   WHERE table_name ='EMPLOYEES';
```

| ANALYZED | SAMPLE_SIZE | MON | TABLE_NAME |
|---|---|---|---|
| 09-FEB-04 | 2000 | YES | EMPLOYEES |

# 5.4.3 Collecte automatique des statistiques

## ■ Vérification des statistiques sur les colonnes

```
Col column_name format a20
Set pagesize 100
SELECT column_name, num_distinct,histogram,
    num_buckets, density, last_analyzed analyzed
FROM dba_tab_col_statistics
   WHERE table_name  ='SALES'
   AND owner='SH'
   ORDER BY column_name;
```

```
COLUMN_NAME          NUM_DISTINCT HISTOGRAM       NUM_BUCKETS    DENSITY ANALYZED
-------------------- ------------ --------------- ----------- ---------- --------

AMOUNT_SOLD                  1887 NONE                      1 ,000529942 02/01/10
CHANNEL_ID                      4 NONE                      1        ,25 02/01/10
CUST_ID                      5967 NONE                      1 ,000167588 02/01/10
PROD_ID                        73 FREQUENCY                72 5,4199E-07 02/01/10
PROMO_ID                        4 NONE                      1        ,25 02/01/10
QUANTITY_SOLD                   1 NONE                      1          1 02/01/10
TIME_ID                      1302 NONE                      1 ,000768049 02/01/10

7 ligne(s) sélectionnée(s).
```

# 5.4.3 Collecte automatique des statistiques

## ■ Vérification des statistiques sur les indexes

```
Execute dbms_stats.gather_schema_stats('HR');
Col index_name format A20
SELECT index_name, num_rows,
last_analyzed, distinct_keys, leaf_blocks l_b,
avg_leaf_blocks_per_key,  join_index
FROM dba_indexes
WHERE table_name = 'EMPLOYEES'
ORDER BY index_name;
```

| INDEX_NAME | NUM_ROWS | LAST_ANA | DISTINCT_KEYS | L_B | AVG_LEAF_BLOCKS_PER_KEY | JOI |
|---|---|---|---|---|---|---|
| EMP_DEPARTMENT_IX | 106 | 06/01/10 | 11 | 1 | 1 | NO |
| EMP_EMAIL_UK | 107 | 06/01/10 | 107 | 1 | 1 | NO |
| EMP_EMP_ID_PK | 107 | 06/01/10 | 107 | 1 | 1 | NO |
| EMP_JOB_IX | 107 | 06/01/10 | 19 | 1 | 1 | NO |
| EMP_MANAGER_IX | 106 | 06/01/10 | 18 | 1 | 1 | NO |
| EMP_NAME_IX | 107 | 06/01/10 | 107 | 1 | 1 | NO |

6 ligne(s) sélectionnée(s).

## 5.4.3 Collecte automatique des statistiques

- **Gestion de l'historique des statistiques, via les fonctions suivantes du package dbms_stats:**
  - `RESTORE_*_STATS()`
  - `PURGE_STATS()`
  - `ALTER_STATS_HISTORY_RETENTION()`

# 5.4.3 Collecte automatique des statistiques

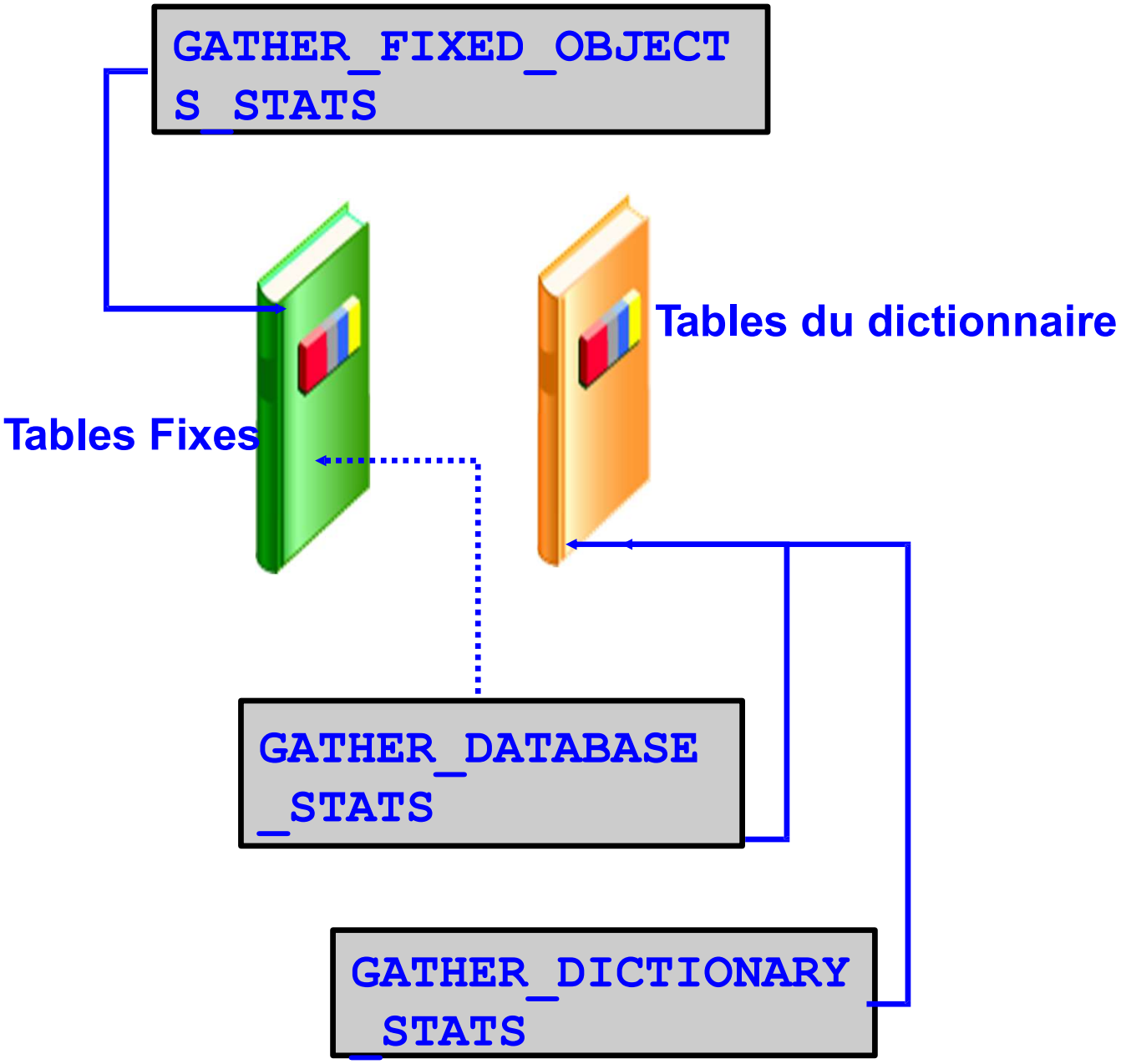
## ■ Génération des statistiques systèmes

- Statistiques sur les entrées / sorties (I/O)
- Statistiques sur la CPU

```
BEGIN
dbms_stats.gather_system_stats(
        gathering_mode => 'interval',
        interval => 720,
        stattab => 'mystats',
        statid => 'oltp');
END;
/
```

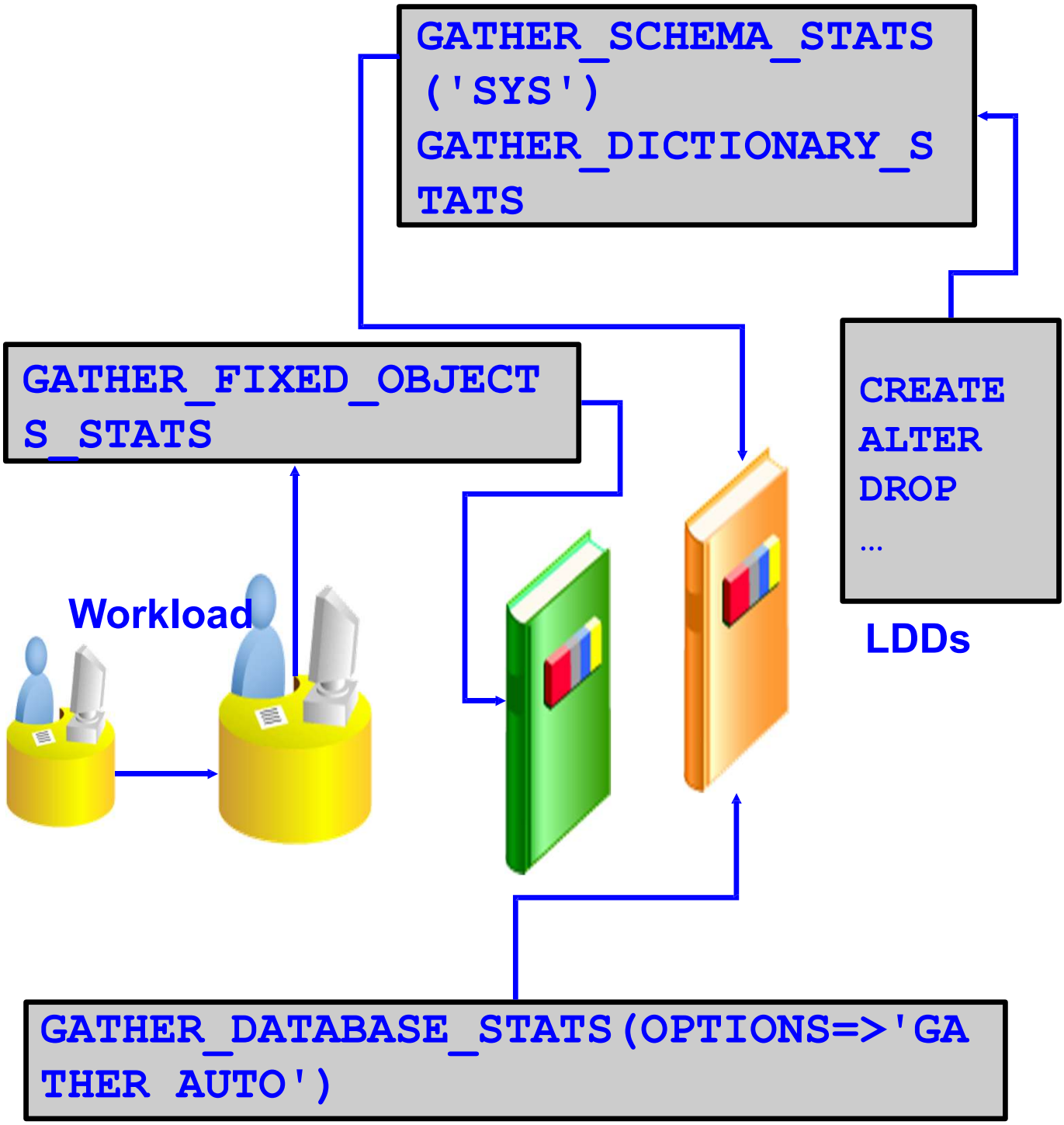
# 5.4.3 Collecte automatique des statistiques

- **Calcul des statistiques sur les objets du dictionnaire Oracle**



# 5.4.3 Collecte automatique des statistiques

- **Calcul des statistiques sur les objets du dictionnaire Oracle : meilleures pratiques**



# 5.4.3 Collecte automatique des statistiques

## Exercice 5.4 : Utilisation du package dbms_stats et recolte automatique

- Calculer manuellement les statistiques en utilisant le package DBMS_STATS
  - Sur un utilisateur (exemple : TPORS)
  - Sur la base entière
- Vérifier la présence des statistiques dans le dictionnaire de données Oracle
- Supprimer les statistiques de l'utilisateur TPORS et mettre en évidence leur absence. Puis recalculer les et mettre en évidence leur présence
- Exporter les statistiques de l'utilisateur TPORS
- Vérifier que Oracle collecte automatiquement les statistiques sur votre base
  - Comment activer ou désactiver la collecte des statistiques
  - Quels sont les paramètres de configuration actuels de la collecte

# 5.5 Autres exercices sur le CBO

## ■ Exercice 5.5

- Mettre en évidence que l'optimiseur de statistique et l'optimiseur de règles ne prennent pas souvent les mêmes décisions

  – Soient les tables Emp et Dept et les index sur ename, deptno dans emp et deptno dans dept. Générer les plans des deux requêtes ci-dessous et comparer. Il est à savoir que l'optimiseur de règles en cas d'index sur les deux colonnes de jointures, il choisit la table la plus à droite de la clause FROM comme première table (elle est lue séquentiellement)

```
Alter session set OPTIMIZER_MODE=rule;
SELECT EMPNO, ENAME, dept.DEPTNO, DNAME
   FROM dept, emp
WHERE emp.deptno=dept.deptno ;

Alter session set OPTIMIZER_MODE=first_rows_1;
SELECT EMPNO, ENAME, dept.DEPTNO, DNAME
FROM dept, emp
WHERE emp.deptno=dept.deptno ;
```

# 6. Etude de cas

## *Etude de cas*

- Générer les plans d'exécutions et mesurer les temps accès aux
- enregistrements de deux tables selons différents types de jointures :
- 1) Jointure sans tables en cluster (CBO)
- 2) Jointure avec tables en cluster index (CBO)
- 3) Jointure avec tables en cluster haché (CBO)

- Pour cela il faut créer les tables suivantes :

- 1) création et insertion de lignes dans les tables EMP4 et DEPT4.
- Y insérer plusieurs millions de lignes
- 2) création et insertion de lignes dans les tables IEMP4
- et IDEPT4 (
- cluster indexé)

## Etude de cas

- 3) création et insertion de lignes dans les tables HEMP4 et HDEPT4 (
- cluster haché)

- Les requêtes à exécuter sont les suivantes:

- SELECT dname, loc, ename, job, hiredate
- FROM EMP4 e, DEPT4 d
- WHERE e.deptno=d.deptno;

- -- Jointure avec tables en cluster indexé
- SELECT dname, loc, ename, job, hiredate
- FROM IEMP4 e, IDEPT4 d
- WHERE e.deptno=d.deptno;

## Etude de cas

- -- Jointure avec tables en cluster haché
- SELECT dname, loc, ename, job, hiredate
- FROM HEMP4 e, HDEPT4 d
- WHERE e.deptno=d.deptno;

- select count(*) from emp4;
- select count(*) from iemp4;
- select count(*) from hemp4;

- SELECT dname, loc, ename, job, hiredate
- FROM EMP4 e, DEPT4 d
- WHERE e.deptno=d.deptno and e.deptno=10;

- SELECT dname, loc, ename, job, hiredate
- FROM IEMP4 e, IDEPT4 d
- WHERE e.deptno=d.deptno and e.deptno=10;

## Etude de cas

- SELECT dname, loc, ename, job, hiredate
- FROM HEMP4 e, HDEPT4 d
- WHERE e.deptno=d.deptno and e.deptno=10;

- SELECT ename, job, hiredate
- FROM EMP4 e
- WHERE e.deptno=10;

- SELECT ename, job, hiredate
- FROM IEMP4 e
- WHERE e.deptno=10;

- SELECT ename, job, hiredate
- FROM HEMP4 e
- WHERE e.deptno=10;

# 6. Etude de cas

## ■ *Etude de cas*

- Mettre en œuvre la réécriture de requêtes à travers une vue matérialisée.

## Etude de cas

- Ces requêtes sont à exécuter avec l'optimiseur statistique en mode First_rows_1
- et en mode All_rows

- Utiliser pour cela autotrace pour produire aussi bien les
- temps d'exécutions et les plans d'exécutions;

# 7. Annexes

## Plan


- Annexe 7.1 Le fichier affreq.sql
- Annexe 7.2 Le fichier Genreq.sql
- Annexe 7.3 Le package DBSM_STATS
- Annexe 7.4 Le package DBSM_STATS

# Annexe 7.1 Le fichier affreq.sql

- **Ce fichier permet d'afficher le plan d'exécution d'une requête.**

```
col operation format a15
col options format a15
col object_name format a12
col id format 99
col pere format 99
col position format 99
col orig_pos format 99
col optimizer format a20
col card format 9999
col cost format 9999

SELECT OPERATION, OPTIONS, OBJECT_NAME, ID, parent_id
    "PERE", position "pos",
OBJECT_INSTANCE "ORIG_POS", optimizer, cost, cardinality
    "card"
FROM PLAN_TABLE
WHERE STATEMENT_ID='SCOTT'
CONNECT BY PRIOR ID=PARENT_ID
AND STATEMENT_ID='SCOTT' START
WITH ID=0
AND STATEMENT_ID='SCOTT'
ORDER BY ID;
```

# Annexe 7.2 Le fichier Genreq.sql

- **Ce fichier permet de supprimer un plan existant identifié par USERNAME et de gérérer un nouveau. Six lignes de requêtes peuvent être saisies d'un trait.**

```
DELETE FROM PLAN_TABLE
WHERE STATEMENT_ID='USERNAME'
/
EXPLAIN PLAN SET
   STATEMENT_ID='USERNAME'
INTO PLAN_TABLE  FOR &requete1_part1
&requete1_part2
&requete1_part3
&requete1_part4
&requete1_part5
&requete1_part6
/
```

# Annexe 7.3 Le package DBSM_STATS

- **Le package `DBMS_STATS` peut être utilisé pour :**
  - **Générer et gérer les statistiques utilisées par l'optimiseur**
  - **Collecter, modifier, visualiser, exporter, importer et supprimer** les statistiques
  - **Identifier ou nommer les statistiques collectées**
  - **Les statistiques sont collectées sur** :
    - Les Index, les tables, les colonnes, et les partitions
    - Tous les objets d'un schéma (utilisateur) ou la base entière
  - **La collecte des statistiques peut être sérialisée ou parallélisée**
- **Note**
  - Voir le manuel Oracle **PL/SQL Packages and Types Reference** pour plus de détails

# Annexe 7.3 Le package DBSM_STATS

## ■ Fonction de gestion de la table STATTAB

| Fonction | Description |
|---|---|
| CREATE_STAT_TABLE | Permet de créer une table pour stocker des statistiques. Cette table s'appelle STATTAB. Elle est utile pour exporter et importer des statistiques |
| DROP_STAT_TABLE | Supprime la table des statistiques créée avec la fonction CREATE_STAT_TABLE |
| UPGRADE_STAT_TABLE | Upgrader la table de statistiques |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions de suppression des statistiques

| Fonction | Description |
|---|---|
| DELETE_COLUMN_STATS | Supprime les statistiques sur une colonne |
| DELETE_DATABASE_STATS | Supprime les statistiques sur une base entière |
| DELETE_DICTIONARY_STATS | Supprime les statistiques sur le dictionnaire d'Oracle (objets de SYS, SYSTEM et d'autres utilisateurs système) |
| DELETE_FIXED_OBJECTS_STATS | Supprime les statistiques sur les tables fixes (table commençant x$... Permettant de gérer des structures internes du noyau Oracle) d'Oracle |
| DELETE_INDEX_STATS | Supprime les statistiques sur un index |
| DELETE_SCHEMA_STATS | Supprime les statistiques sur un schéma |
| DELETE_SYSTEM_STATS | Supprime les statistiques systèmes |
| DELETE_TABLE_STATS | Supprime les statistiques sur une table |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions d'Export des statistiques

| Fonction | Description |
|---|---|
| EXPORT_COLUMN_STATS | Exporte les statistiques sur une colonne |
| EXPORT_DATABASE_STATS | Exporte les statistiques sur une base entière. Les statistiques sont stockées dans la table STATTAB créée avec la fonction CREATE_STAT_TAB |
| EXPORT_DICTIONARY_STATS | Exporte les statistiques sur le dictionnaire Oracle |
| EXPORT_FIXED_OBJECTS_STATS | Exporte les statistiques sur les tables fixes Oracle |
| EXPORT_INDEX_STATS | Exporte les statistiques sur les index |
| EXPORT_SCHEMA_STATS | Exporte les statistiques sur un schéma |
| EXPORT_SYSTEM_STATS | Exporte les statistiques systèmes |
| EXPORT_TABLE_STATS | Exporte les statistiques sur les tables |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions de collecte des statistiques

| Fonction | Description |
|---|---|
| GATHER_DATABASE_STATS | Collecte des statistiques sur la base entière |
| GATHER_DICTIONARY_STATS | Collecte des statistiques sur le dictionnaire Oracle |
| GATHER_FIXED_OBJECTS_STATS | Collecte des statistiques sur les tables fixes |
| GATHER_INDEX_STATS | Collecte des statistiques sur les index |
| GATHER_SCHEMA_STATS | Collecte des statistiques sur les objets d'un utilisateur |
| GATHER_SYSTEM_STATS | Collecte des statistiques systèmes |
| GATHER_TABLE_STATS | Collecte des statistiques sur une table |
| GENERATE_STATS | Génération des statistiques à partir des statistiques déjà collectées sur un objet |

# Annexe 7.3 Le package DBSM_STATS

- Les fonctions de récupération de statistiques depuis la table STATTAB

| Fonction | Description |
|---|---|
| GET_COLUMN_STATS | Récupération des statistiques sur une colonne |
| GET_INDEX_STATS | Récupération des statistiques sur un index |
| GET_SYSTEM_STATS | Récupération des statistiques système |
| GET_SCHEMA_STATS | Récupération des statistiques sur un schéma |
| | |
| GET_TABLE_STATS | Récupération des statistiques sur une table |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions d'Import des statistiques

| Fonction | Description |
|---|---|
| IMPORT_COLUMN_STATS | Importe les statistiques sur une colonne |
| IMPORT_DATABASE_STATS | Importe les statistiques sur une base entière. Les statistiques sont stockées dans la table STATTAB créée avec la fonction CREATE_STAT_TAbLE |
| IMPORT_DICTIONARY_STATS | Importe les statistiques sur le dictionnaire Oracle |
| IMPORT_FIXED_OBJECTS_STATS | Importe les statistiques sur les tables fixes Oracle |
| IMPORT_INDEX_STATS | Importe les statistiques sur les index |
| IMPORT_SCHEMA_STATS | Importe les statistiques sur un schéma |
| IMPORT_SYSTEM_STATS | Importe les statistiques systèmes |
| IMPORT_TABLE_STATS | Importe les statistiques sur les tables |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions de verrouillage des statistiques

| Fonction | Description |
|---|---|
| LOCK_SCHEMA_STATS | Verrouiller les statistiques sur un schéma |
| | |
| LOCK_TABLE_STATS | Verrouiller les statistiques sur une table |
| UNLOCK_SCHEMA_STATS | |
| UNLOCK_TABLE_STATS | Déverrouille les statistiques |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions de préparation des statistiques

| Fonction | Description |
|---|---|
| PREPARE_COLUMN_VALUES | Converti le minimum, maximum et l'histogramm définient par l'utilisateur pour type spécifique en format interne Oracle pour stockage futur via la fonction SET_COLUMN_STATS |
| PREPARE_COLUMN_VALUES _NVARCHAR2 | Converti le minimum, maximum et l'histogramm définient par l'utilisateur pour type spécifique en format interne Oracle pour stockage futur via la fonction SET_COLUMN_STATS |
| PREPARE_COLUMN_VALUES _ ROWID | Converti le minimum, maximum et l'histogramm définient par l'utilisateur pour type spécifique en format interne Oracle pour stockage futur via la fonction SET_COLUMN_STATS |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions de restauration des statistiques

| Fonction | Description |
|---|---|
| RESTORE_DICTIONARY_STATS | Restaure les statistiques sur le dictionnaire Oracle (table de SYS, SYSTEM et d'autres users systèmes) à partir d'une date donnée |
| RESTORE_SCHEMA_STATS | Restaure les statistiques sur un utilisateur à partir d'une date donnée |
| RESTORE_SYSTEM_STATS | Restaure les statistiques systèmes à partir d'une date donnée |
| RESTORE_TABLE_STATS | Restaure les statistiques sur une table à partir d'une date donnée |

# Annexe 7.3 Le package DBSM_STATS

## ■ Les fonctions d'assignation des statistiques

| Fonction | Description |
|---|---|
| SET_COLUMN_STATS | Assigne les statistiques sur une colonne |
| SET_INDEX_STATS | Assigne les statistiques sur un index |
| SET_SYSTEM_STATS | Assigne les statistiques système |
| SET_TABLE_STATS | Assigne les statistiques sur une table |

# Annexe 7.3 Choix de chemins d'accès : optimiseur de règles

## ■ Mode de choix d'un chemin d'accès dans l'approche règle (Optimizer_mode=rule)

| RANG | CHEMIN D'ACCES (1 plus rapide, 15 plus lent). |
|---|---|
| 1 | **ROWID = constante** (1 tuple au plus ramené) |
| 2 | **clé cluster entièrement renseignée = clé cluster d'une autre table** (1 tuple au plus ramené) |
| 3 | **Clé cluster haché avec index unique ou primaire** = constante (1 tuple au plus est ramené) |
| 4 | **index unique ou clé primaire entièrement renseignée = constante** (1 tuple au plus est ramené) |
| 5 | **clé cluster entièrement renseignée** = clé cluster d'une autre table (0 à N tuples sont ramenés) |
| 6 | **clé cluster hachée avec index unique ou primaire =** constante (0 à N tuples sont ramenés) |
| 7 | **clé cluster indexée = constante** |
| 8 | **Index composé** entièrement renseigné <=ou>= constante |
| 9 | **Index non unique simple** = constante |
| 10 | **Index concatené partiellement renseigné** à gauche Index (unique non unique) utilisé avec between ou like 'X%' |
| 11 | **Index (unique non unique)** <ou<= ou > ou >= constante |
| 12 | tri fusion (jointure de tables non clusterisées) |
| 13 | **MAX ou MIN** (colonne indexée) |
| 14 | **Order by colonne indexée** |
| 15 | **Balayage séquentiel** |

# Annexe 7.4 Choix de chemins d'accès : optimiseur de règles

- **Mode de choix d'un chemin d'accès dans l'approche règle(Optimizer_mode=rule)**
  - En présence de plusieurs chemins d'accès, l'optimiseur syntaxique choisit d'abord les chemins de fortes priorités(rang 1 à 15).

  - **Exemple**
    - **select** empno **from** emp
    - **where** ename='KING' **AND** sal >2500;
    - Considérons comme définis :
      - une clé primaire sur empno
      - un index sur ename nommé idx_ename
      - un index sur sal nommé idx_sal
    - chemins d'accès disponibles :
      - Index non unique simple=constante. Ou, ename= constante (rang 9)
      - Index < ou <= ou > ou >= constante : sal > 2500 (rang 11)
      - l'accès séquentiel à toute la table (rang 15).
    - *Choix*: l'optimiseur choisi le chemin idx_ename de rang 9